UNIDAD 2 — Redes Neuronales 1

Perceptrón Simple

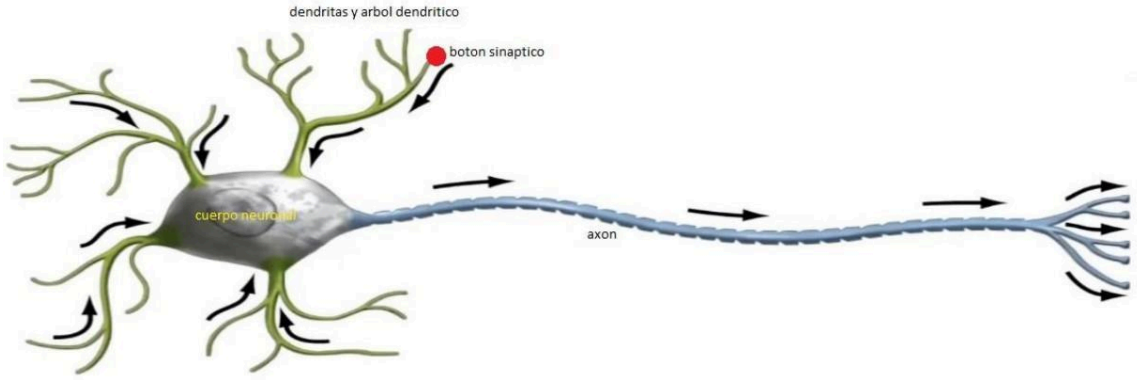
1- Fisiología básica de una neurona

Características de la neurona para poder modelar en una computadora:

- En el cerebro hay 10^11 neuronas, aunque actualmente hay modelos de esta cantidad no lo vamos a realizar.

- En las **redes neuronales**:
  - **N**o linealidad, un pequeño cambio en la entrada puede generar un gran cambio en la salida, o si dos entradas tiene dos salidas la suma de las entradas no es la suma de las salidas.
  - **P**aralelismo, hay muchas neuronas o unidades de proceso que convergen o concurren para resolver problemas complejos, pero cada unidad es muy sencilla o elemental.
  - **A**prendizaje, pueden aprender a partir de los datos, les damos ejemplos y ellas aprenden a resolver situaciones parecidas.
  - **G**eneralización, es la capacidad de resolver problemas nunca antes visto.
  - **A**daptabilidad, queremos que sean capaces de adaptarse a nuevas situaciones que no se vieron en el proceso de aprendizaje.
  - **R**obustez, la capacidad de funcionar bien aun cuando los datos tienen ruido o tiene faltantes.

Partes y fisiología de la neurona biológica:



- Cuerpo neuronal, que es donde se procesa la respuesta.
- Dendritas, que conforman un árbol dendrítico, es por donde ingresan los impulsos, las entradas.
- Axón, es por donde se comunica la respuesta, su salida.

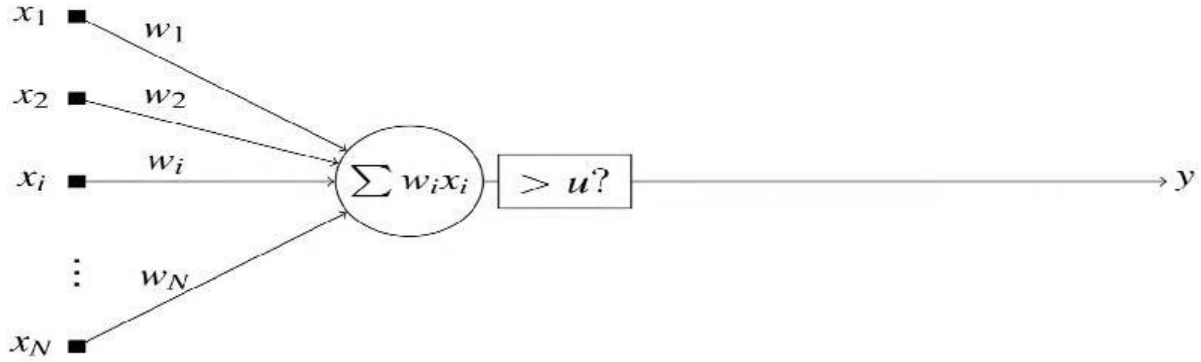
Comportamiento de una neurona:

- Botón sináptico, es donde se produce la sinapsis.
- **Sinapsis,** es la transmisión de los neurotransmisores.
  - Su finalidad es despolarizar la membrana de la neurona.
  - La membrana del núcleo tiene una polaridad, y cuando recibe una cierta cantidad de estímulos llega a despolarizarse y emite un único pulso por el axón.

    - Estos estímulos no son todos iguales, algunos son excitatorios y otros son inhibitorios, más o menos débil en cada caso, dependiendo del aprendizaje.
- Comportamiento todo o nada, solamente se envía el pulso cuando se logra la suficiente excitación de la neurona.
- Refuerzo de la sinapsis o aprendizaje, cuando se va aprendiendo se refuerzan los impulsos y los que eran débiles pasan a ser más fuertes y se va modificando la sinapsis a veces hasta se desconectan.

Modelo computacional de una neurona

- Modelo simplificado de la neurona o perceptrón simple 1:

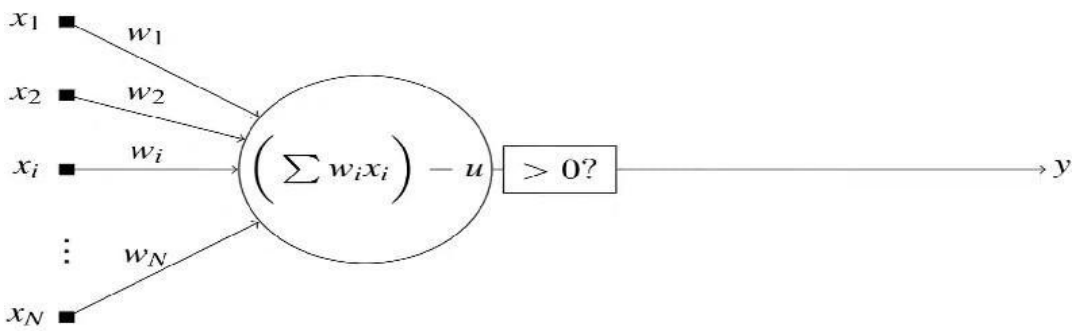


- Consideramos n entradas **xi = x1…xn** y cada una va a tener un peso sináptico **wi = w1…wn** que es un número real sin ninguna limitante. (si la sinapsis es excitatoria y grande, será un número alto, y viceversa)
  => a cada entrada **xi** se la pondera con el peso **wi** de acuerdo a la importancia que tengan para la despolarización de la neurona.

- En el cuerpo de la neurona:
    - Se hace una suma ponderada de las entradas con sus pesos.
    - Si la sumatoria es mayor al umbral definido **u**, se envía una respuesta o salida **y**.

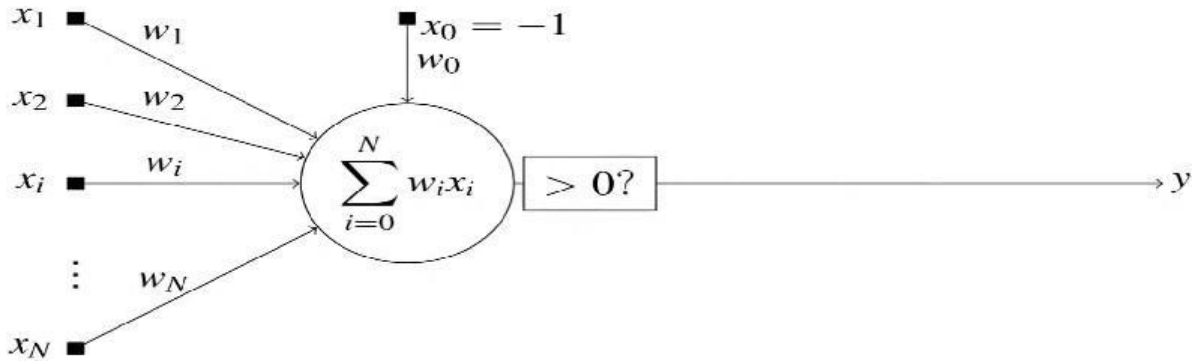
- Se realizan mayores simplificaciones asumimos que:
    - Las entradas **xi**, llegan simultáneamente.
    - Se evalúa de manera instantánea en el cuerpo neuronal si se supera el umbral u.

- Modelo simplificado de la neurona o perceptrón simple 2:



- Es una ecuación más sencilla, introduciendo el umbral como resta en el núcleo de la neurona.

- Modelo simplificado de la neurona o perceptrón simple 3:



- Una última simplificación, en la cual hacemos que el umbral **u** sea la multiplicación de una entrada **x0,** que siempre es **-1**, por un peso **w0**. Para incluirla, cambiamos el índice de la sumatoria, de 1 a cero, entonces estamos incluyendo la nueva entrada.

=> llegamos a lo que es un Perceptrón Simple:

Perceptrón simple:

- Es un producto interno entre las entradas y peso sináptico, denominado activación lineal **v**, y un umbral:

$$y = \phi(v - u) = \phi\left(\sum_{i=1}^{N} w_i x_i - u\right)$$

y con la entrada extendida: **x0 = -1, w0 = u:**

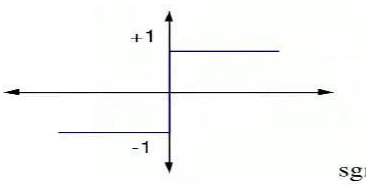
$$y = \phi\left(\sum_{i=0}^{N} w_i x_i\right) = \phi\left(\langle \mathbf{w}, \mathbf{x} \rangle\right)$$

Función de Activación:

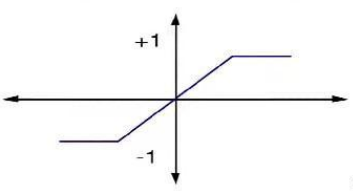
$$\emptyset(z) = y - \emptyset\left(\sum_{i=0}^{N} w_i\, x_i\right) = \emptyset(< w, x >)$$

?

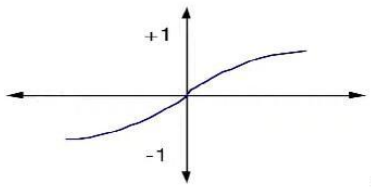
Podemos obtener muchas funciones de activación:



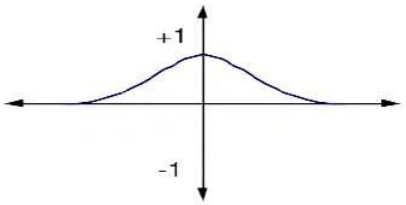
$$\text{sgn}(z) = \begin{cases} -1 & \text{si } z < 0 \\ +1 & \text{si } z > 0 \end{cases}$$

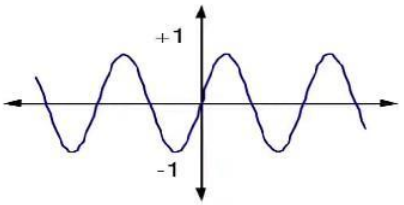


$$\text{sln}(z) = \begin{cases} -1 & \text{si } z < -a \\ \alpha z & \text{si } -a < z < 0 \\ +1 & \text{si } z \geq a \end{cases}$$

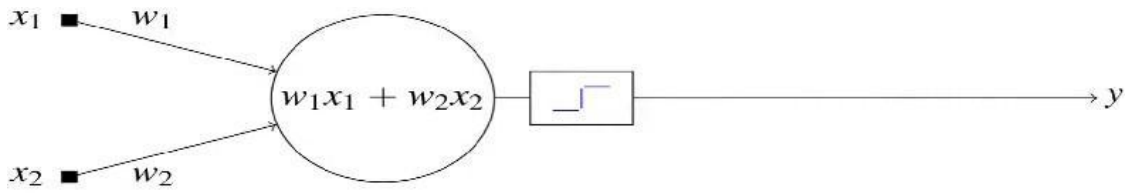


$$\text{sig}(z) = \frac{1 - e^{-az}}{1 + e^{-az}}$$





Un perceptrón simple con 2 entradas, y fda signo:

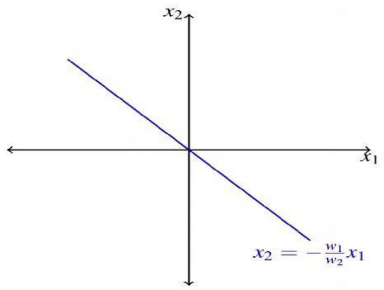


$$y = \text{sgn}(w_1x_1 + w_2x_2)$$

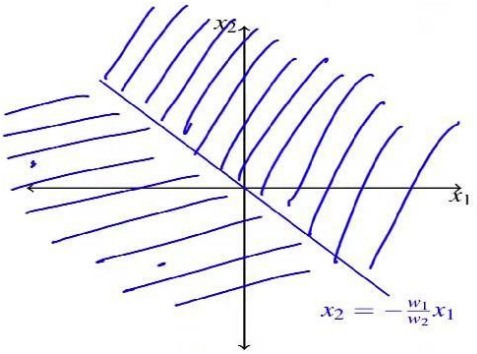
$w_1x_1 + w_2x_2 > 0?$ → <u>punto crítico de funcionamiento de la neurona:</u>

- <u>Representación gráfica</u> de la zona donde la suma ponderada da cero:

$w_1x_1 + w_2x_2 = 0$ → ecuación de una recta →

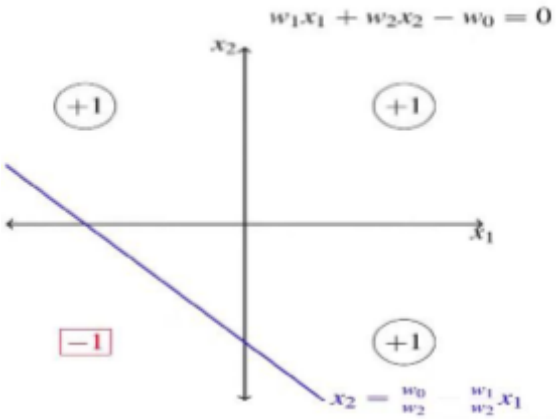


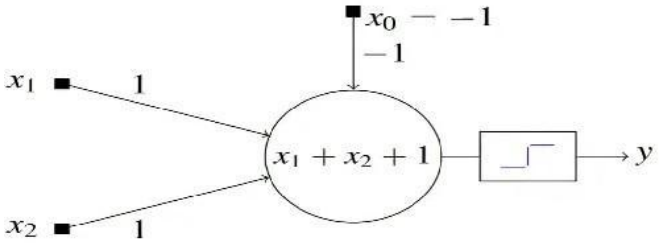
→ lo que nos delimita 2 zonas de comportamiento:



- Necesidad del bias/sesgo:

    - Cuando necesitamos que la recta NO pase por el orígen para delimitar las 2 zonas de comportamiento, agregamos el umbral **w0** a la ecuación:
    - Por ejemplo, en el comportamiento del OR:





Acá le pusimos nosotros los pesos sinápticos, pero la idea es que la neurona aprenda sola a asignar los pesos…

Algoritmos de aprendizaje: ¿Cómo aprende solo?

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas $y_d(n)$ y en cada caso se calcula:

$y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$ → incorporamos la **n** (la iteración en el proceso de aprendizaje, cada vez que le muestro un ejemplo):

- SI la salida de la red es **correcta:** $y(n) = y_d(n)$
  no se hacen cambios: *principio de mínima perturbación.*
- SI la salida de la red es **incorrecta:** $y(n) \neq y_d(n)$
  penalización: se *actualizan los pesos en el sentido opuesto* al cual con el que contribuyeron a la salida incorrecta:

  - **SI** $y(n) = +1$ **mientras** $y_d(n) = -1$
    $\Rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) - \eta\mathbf{x}(n)$

  => *restamos una proporción de las entradas que entraron.*

  - **SI** $y(n) = -1$ **mientras** $y_d(n) = +1$
    $\Rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) + \eta\mathbf{x}(n)$

  => *sumamos una proporción de las entradas que entraron*

O mejor, para unir todos los casos (correcto e incorrecto):

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\eta}{2}[y_d(n) - y(n)]\mathbf{x}(n)$$

=> *al peso actual le sumamos* **una proporción*(la diferencia entre la salida deseada y la salida de la neurona)*(entrada)**

(el ½, se pone para simplificar la dif, que puede dar +2 o -2)

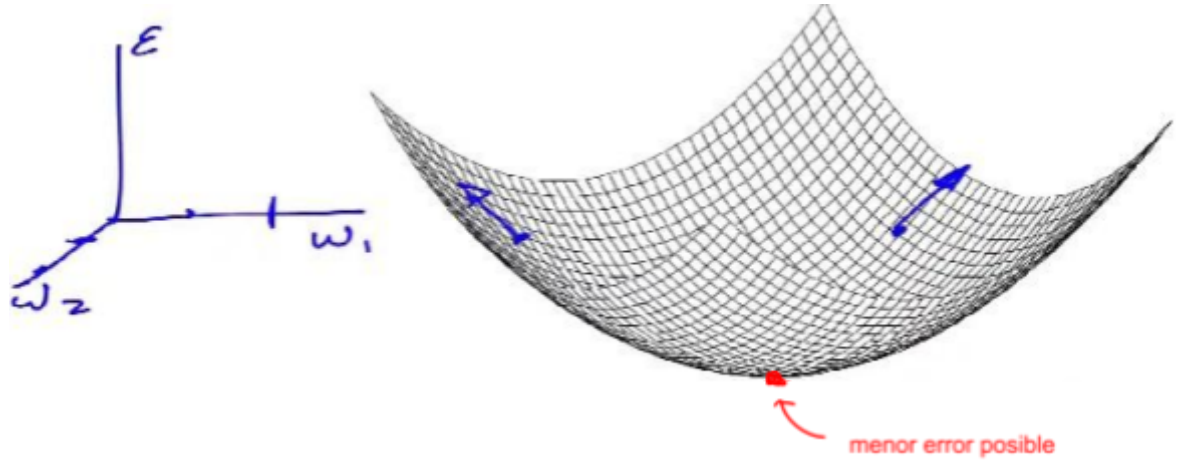
3. Volver a 2 hasta satisfacer algún criterio de finalización.

Entrenamiento por <u>métodos de gradiente</u>: perceptrón simple

→ Forma de encontrar los pesos sinápticos con un mayor sustento matemático

→ **Concepto**: Mover los pesos iteración a iteración en la dirección en que se reduce el error, <u>dirección que es opuesta a su gradiente con respecto a los pesos</u>

→ **Interpretación gráfica:**



→ **Ecuación básica:**

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu\nabla_w\xi(\mathbf{w}(n))$$

pesos en la it. sig. **=** peso actual - vel. de aprendizaje*gradiente de error respecto a los pesos

$$\xi(n) = \tfrac{1}{2}(d(n)-y(n))^2$$

½ para simplificar cuando derivamos
elevado al cuadrado para que siempre de positivo, error negativo no tendría sentido

→ <u>la vel. de aprendizaje hace que nos movamos más o menos rápido por la superficie</u>
→ si la sup. es suave puedo ir más rápido, y viceversa.

→ **Aplicación:**

- Caso sencillo: perceptrón simple (least mean squares)
- **Caso general: perceptrón multicapa (back-propagation)**

Perceptrón simple: método de gradiente (caso lineal)

- Medimos el error con el criterio de error instantáneo (d(n) = salida deseada)

$$e^2(n) = [d(n) - y(n)]^2 = [d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

- Caso lineal: **eliminamos** la función de activación, es una recta.

- Para calcular el gradiente, tenemos que derivar respecto a los pesos:

$$\nabla_w e^2(n) = 2\,[d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]\,(-\mathbf{x}(n))$$

$$\nabla_w e^2(n) = 2e(n)(-\mathbf{x}(n))$$

reemplazando en: $\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)}$$

→ lo único que cambió respecto al método anterior es que tenemos 2μ, donde antes teníamos η.

Ejemplo:

Supongamos que: $w_0 = +1 \quad x_0 = -1 \quad d = -1$
$w_1 = +1 \quad x_1 = +1$
$w_2 = +1 \quad x_2 = +1$

$$y = \text{sgn}(-1 + 1 + 1) \rightarrow 1$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)$$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + 2\mu(-1-1) \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

suponiendo que $\mu = \frac{1}{2}$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + \begin{bmatrix} +2 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} +3 \\ -1 \\ -1 \end{bmatrix}$$

Qué sucede si volvemos a poner: $\mathbf{x}(n+1) = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$?

$$y = \text{sgn}(-3 - 1 - 1) \rightarrow -1$$

$$\boxed{\text{ahora } e(n+1) = -1 - (-1) = 0}$$

UNIDAD 3 — Redes Neuronales 2

Perceptrón Multicapa

¿Cómo podríamos resolver el problema del XOR?

→ Lo podemos resolver con 3 perceptrones simples:

Primera capa: **perceptrones A y B**

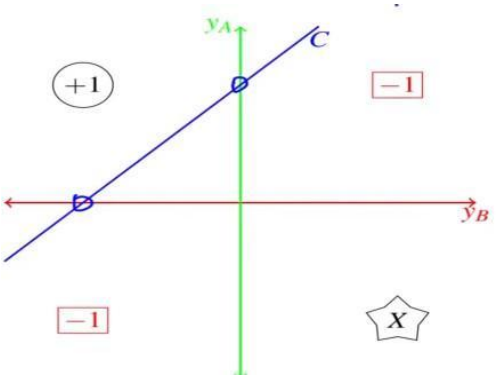


En la zona donde A da 1 y B da -1, deberíamos tener un perceptrón C que junte esas 2 salidas

Segunda capa: **perceptrón C**

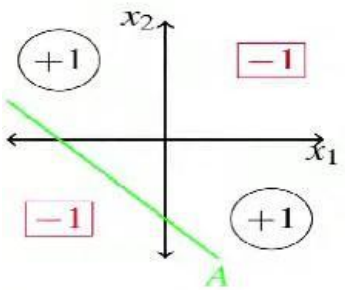
Vamos a considerar las salidas de A y B, $\mathbf{Y_A}$ e $\mathbf{Y_b}$ como entradas de C, y lo vamos a diseñar según la siguiente tabla de verdad:

y_A	y_B	y_C
+1	+1	−1
+1	−1	+1
−1	−1	−1



y el caso de $\mathbf{Y_A}$ = -1 e $\mathbf{Y_b}$ = +1 no va a darse nunca, entonces la región X no importa.

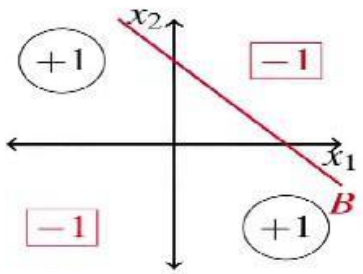
Combinación de perceptrones simples



Perceptrón A: $x_2 = -1 - x_1 = \frac{w_{A0}}{w_{A2}} - \frac{w_{A1}}{w_{A2}} x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{A0} = -1 \\ w_{A1} = +1 \\ w_{A2} = +1 \end{array} \right\} \rightarrow y_A = \text{sgn}(x_2 + x_1 + 1)$$

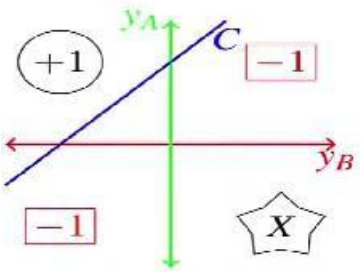
→ A suma las 2 entradas y a eso le suma 1.



Perceptrón B: $x_2 - +1 - x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{B0} = +1 \\ w_{B1} = +1 \\ w_{B2} = +1 \end{array} \right\} \rightarrow y_B = \mathrm{sgn}(x_2 + x_1 - 1)$$

→ B suma las 2 entradas y a eso le resta 1.

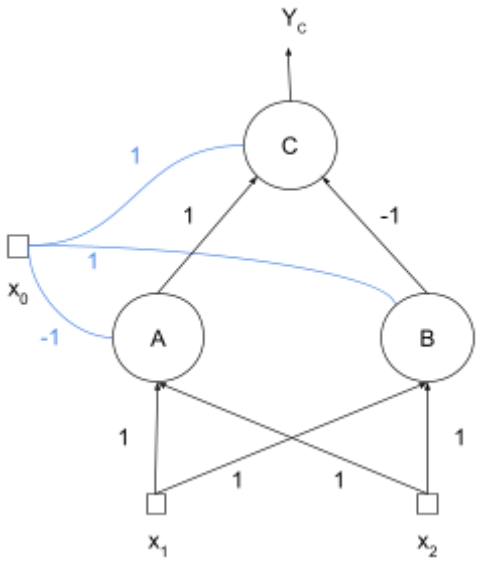


Perceptrón C: $y_A = +1 + y_B$

$$\rightarrow \left\{ \begin{array}{l} w_{C0} = +1 \\ w_{C1} = -1 \\ w_{C2} = +1 \end{array} \right\} \rightarrow y_C = \mathrm{sgn}(y_A - y_B - 1)$$

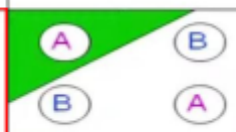
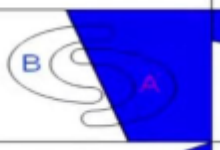
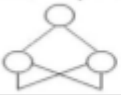
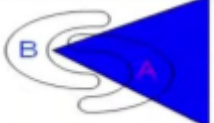
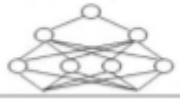
→ C resta Y_A menos Y_B y a eso le resta 1.

Arquitectura de esta red neuronal

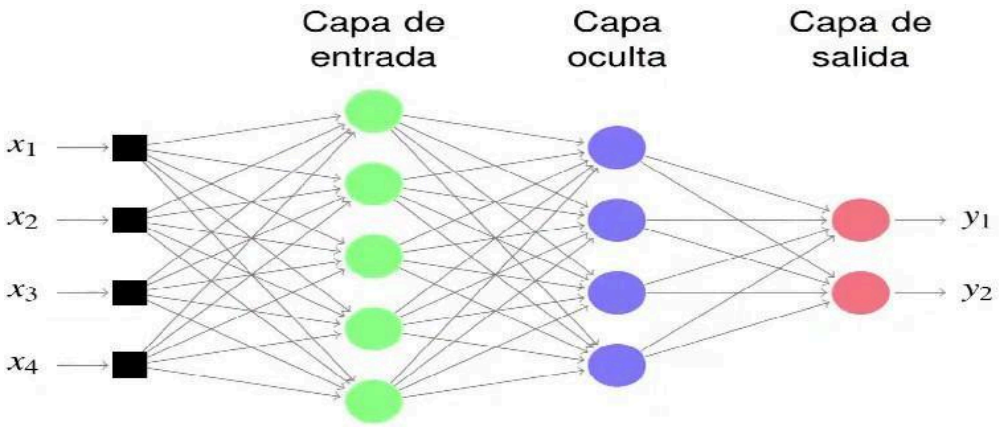


$$\left. \begin{array}{l} y_A = \mathrm{sgn}(x_2 + x_1 + 1) \\ y_B = \mathrm{sgn}(x_2 + x_1 - 1) \end{array} \right\} \rightarrow y_C = \mathrm{sgn}(y_A - y_B - 1)$$

Perceptrón Multicapa: Regiones de decisión y arquitectura

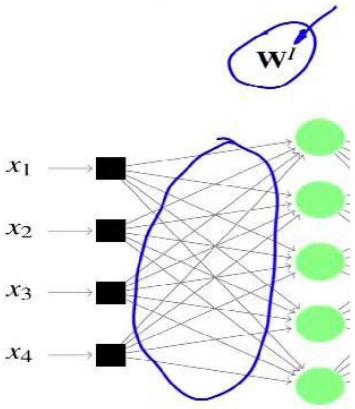
Estructura	Tipos de regiones de decisión	Problema XOR	Separación en clases	Formas regiones más generales
Una capa 	hemiplano limitado por hiperplano			
Dos capas 	Regiones convexas abiertas o cerradas			
Tres capas 	Arbitrarias (Complejidad limitada por N°. de Nodos)			

Arquitectura del perceptrón multicapa

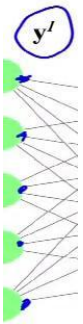


- En este caso tenemos datos de entrada en R^4
- Faltaría la entrada x_0 en el gráfico

→ **Notación vectorial:**



- Vamos a tener <u>MATRICES DE PESOS</u> **W** →
  (de tamaño: entradas x cant. percep. prox. capa)
- y <u>VECTORES DE SALIDAS</u> **y** →



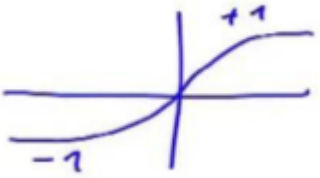
- Usamos superíndices para indicar el número de capa en el que estamos.

Cálculo de las salidas en cada capa (Propagación hacia adelante)

- Capa I:

$$v_j^I = \left\langle \mathbf{w}_j^I, \mathbf{x} \right\rangle = \sum_{i=0}^{N} w_{ji}^I x_i \quad \text{(completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x}\text{)}$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-bv_j^I}} - 1 \quad \text{(simétrica } \perp 1\text{)}$$



→ en lugar de la función signo, usamos la función sigmoide →

- Capa II:

$$v_j^{II} = \left\langle \mathbf{w}_j^{II}, \mathbf{y}^I \right\rangle \quad \rightarrow \quad y_j^{II} = \phi(v_j^{II})$$

- Capa III:

$$v_j^{III} = \left\langle \mathbf{w}_j^{III}, \mathbf{y}^{II} \right\rangle \quad \rightarrow \quad y_j^{III} = \phi(v_j^{III}) = y_j$$

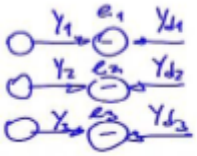
Propagación hacia atrás: caso general y capa de salida

→ Es lo que permite actualizar los pesos, hacer que la red neuronal aprenda iteración a iteración.

→ Definimos un **criterio de error**:

Suma del error cuadrático instantáneo

Es un error para el instante n, para la iteración n.



$$\xi(n) = \frac{1}{2} \sum_{j=1}^{M} e_j^2(n)$$

Restamos cada una de las salidas, elevamos al cuadrado, y luego sumamos.

→ (así siempre la suma va creciendo cuando hay un error)

Aplicación del gradiente (caso general)

Modificación de los pesos:

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

- En el sentido opuesto al gradiente del error.
- La **velocidad de aprendizaje** μ es qué tan rápido nos movemos sobre la superficie de error:
  - Si es muy grande: se va a aprender el ejemplo que le acabo de mostrar, pero olvidar todo lo que aprendió antes.

- Si es muy pequeña: voy a tener que mostrarle muchas más veces las cosas para que las aprenda

- El **gradiente de error** es la derivada parcial del criterio de error cuadrático que definimos con respecto a los **pesos** w_{ij}.

  Tenemos que derivar a una función de una función de una función:
  - El criterio de error cuadrático medio **ξ** depende del error cuadrático medio e_j de la neurona asociada al peso **j.**
  - Ese error e_j depende de la salida no lineal y_j que haya dado esa neurona (la salida después de pasar por la función de activación.
  - Esa salida y_j depende de la salida lineal v_j (la salida antes de pasar por la función de activación).
  - Esa salida v_j es la que finalmente depende de los pesos w_{ij}.

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial \sum_{i=0}^{N} w_{ji}(n) y_i(n)}{\partial w_{ji}(n)} = y_i(n)$$

→ Observamos que para todos los valores de i, salvo cuando i valga el i de $w_{ij}(n)$, tenemos valores constantes respecto de lo que estamos derivando (sería la variable $w_{ij}(n)$).

Entonces nos quedará la dd de una constante ($y_i(n)$) por la variable ($w_{ij}(n)$)

OBS → y_i , con subíndice **i**, es la **ENTRADA** a la neurona, que fue pesada con el peso w_i.

→ y_j , con subíndice **j**, es la SALIDA no lineal de la neurona.

=> la modificación nos queda:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \boxed{\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}} y_i(n)$$

Para seguir simplificando la expresión, definimos:

$$\text{Gradiente de error local instantáneo: } \delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

- "Local" → porque es de la neurona **j**.
- "Instantáneo" → porque está calculado en un instante **n** de esta iteración.
- Es una parte del gradiente global.

- Es una versión simplificada del gradiente respecto de la salida, a diferencia de la primera expresión donde estamos pasando a través del error $\mathbf{e_j}$.

→ Si analizamos la segunda parte del $\boldsymbol{\delta_j}$:

$$\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \boxed{\frac{\partial y_j(n)}{\partial v_j(n)}}$$

→ Es la derivada de la función de activación $\mathbf{y_j}$. (simétrica)

- **Desarrollo de la derivada de la sigmoidea:**

$$\begin{aligned}
\boxed{\frac{\partial y_j(n)}{\partial v_j(n)}} &= \frac{\partial \left\{ \frac{2}{1+e^{-v_j(n)}} - 1 \right\}}{\partial v_j(n)} \\
&= 2\frac{e^{-v_j(n)}}{(1+e^{-v_j(n)})^2} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \frac{e^{-v_j(n)}}{1+e^{-v_j(n)}} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \frac{\overbrace{-1+1}^{0} + e^{-v_j(n)}}{1+e^{-v_j(n)}} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \left(\frac{-1}{1+e^{-v_j(n)}} + \frac{1+e^{-v_j(n)}}{1+e^{-v_j(n)}} \right)
\end{aligned}$$

$$\begin{aligned}
\frac{\partial y_j(n)}{\partial v_j(n)} &= 2\frac{1}{1+e^{-v_j(n)}} \left(1 - \frac{1}{1+e^{-v_j(n)}} \right) \\
&- 2\frac{y_j(n)+1}{2} \left(1 - \frac{y_j(n)+1}{2} \right) \\
&= (y_j(n)+1) \left(1 - \frac{y_j(n)+1}{2} \right) \\
&= (y_j(n)+1) \left(\frac{2 - y_j(n) - 1}{2} \right) \\
&= \boxed{\frac{1}{2}(y_j(n)+1)(y_j(n)-1)}
\end{aligned}$$

→ obtenemos una ecuación muy simple

=> volviendo a la definición del $\boldsymbol{\delta_j}$:

$$\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

→ Nos quedaría por calcular una única derivada, la cual va a depender de la capa en la que estemos parados

=> la modificación nos queda:

$$\boxed{\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)}$$

Propagación hacia atrás en la capa III (salida)

La modificación de los pesos de la última capa es:

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

Tenemos que calcular el gradiente de error instantáneo δ_j^{III}.

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

- Calculamos la derivada:

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

  - Reemplazamos la expresión de **ξ,** y de $\mathbf{e_j}$:

$$\delta_j^{III}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_j e_j^2(n) \right\}}{\partial e_j(n)} \cdot \frac{\partial \left\{ d_j^{III}(n) - y_j^{III}(n) \right\}}{\partial y_j^{III}(n)} \cdot \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

  - Derivamos:

$$\delta_j^{III}(n) = \frac{1}{2} e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n)) \bigstar$$

=> la modificación queda:

$$\Delta w_{ji}^{III}(n) = \eta e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n)) y_i^{II}(n)$$

(η absorbe todas las constantes que hay por ahí, y la velocidad de aprendizaje)

ΔW_{ji}^{III} = eta * error * dd de la f. de activación * entrada

Propagación hacia atrás en la capa II (oculta)

- Para calcular la derivada dentro del δ_j^{II} en este caso, vemos que <u>al error **ξ** lo estamos midiendo en la capa de salida</u> ($\mathbf{e_j}$ depende de $\mathbf{y_j}$), pero ahora queremos derivar con respecto a la capa oculta, osea que tenemos que atravesar ambas para hacerlo:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^{I}(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_j^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

y_j^{III}

(llamamos **k** al índice que recorre las neuronas de la capa de salida (III), y **j** es el de la capa actual (II), e **i** recorre a las salidas de la capa anterior (I))

- La dd de la sumatoria es la sumatoria de las dd:

$$\delta_j^{II}(n) = -\frac{1}{2}\sum_k \frac{\partial e_k^2(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n)\frac{\partial e_k(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

- Ahora sí vamos a recorrer todo lo que hay entre el error en la capa de salida y la neurona en la capa oculta contra la que queremos obtener la dd:

$$\delta_j^{II}(n) = -\sum_k e_k(n)\frac{\partial e_k(n)}{\partial y_k^{III}(n)}\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)}\frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n)\cdot\frac{\partial\{d_k^{III}(n)-y_k^{III}(n)\}}{\partial y_k^{III}(n)}\cdot\frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n))\cdot$$
$$\cdot\frac{\partial\left\{\sum_j w_{kj}^{III}y_j^{II}(n)\right\}}{\partial y_j^{II}(n)}\cdot\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

dd de la función de activación en la capa de salida, y en la capa oculta

$$\delta_j^{II}(n) = \sum_k e_k(n)\cdot\frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n))\cdot w_{kj}^{III}\cdot$$
$$\cdot\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

  - Pero vemos que dentro de δ_j^{II} está metida la fórmula de δ_j^{III}, así que podemos reemplazarlo:

$$\delta_j^{II}(n) = \left[\sum_k \delta_k^{III}(n)w_{kj}^{III}\right]\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

→ DE ESTA PARTE SURGE EL CONCEPTO DE "RETROPROPAGACIÓN":

- Tenemos un promedio ponderado del gradiente de error local de la capa III, lo que hace que **este gradiente pase a través de los pesos que unen la capa III con la capa II, de atrás para adelante.**

=> la modificación queda:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

Por lo tanto:

$$\Delta w_{ji}^{II}(n) = \eta \left[\sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^I(n)$$

ΔW_{ji}^{III} = eta * retroprop. de g. de error de (III) hacia (II) * dd de la f. de activación (II) * entrada a (II)

Generalizando para la capa "p"

$$\Delta w_{ji}^{II}(n) = \eta \left[\sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^I(n)$$

$$\Downarrow$$

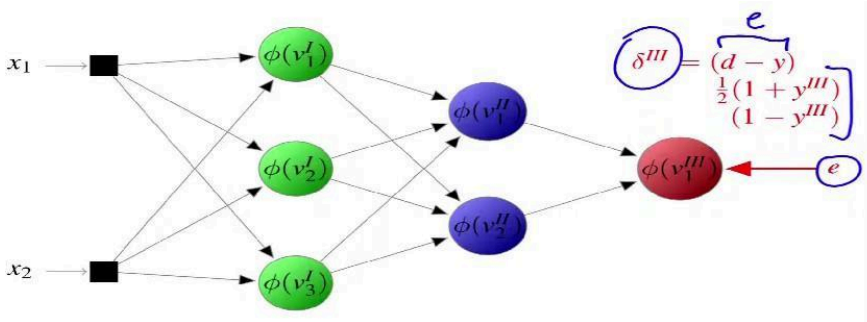
$$\Delta w_{ji}^{(p)}(n) = \eta \left\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \right\rangle (1 + y_j^{(p)}(n))(1 - y_j^{(p)}(n)) y_i^{(p-1)}(n)$$

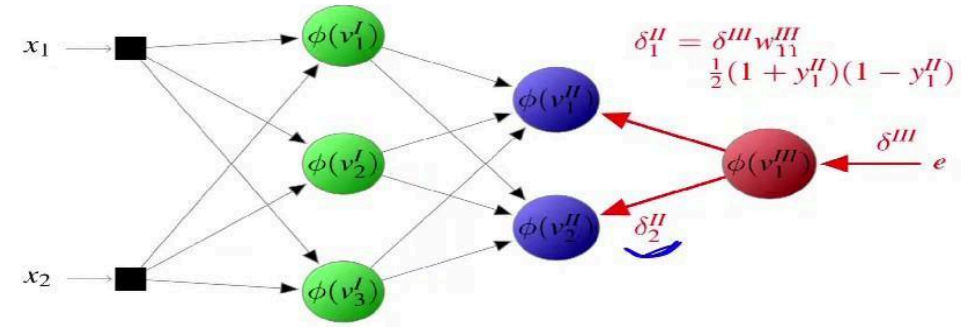
Resúmen del algoritmo de retropropagación (BP)

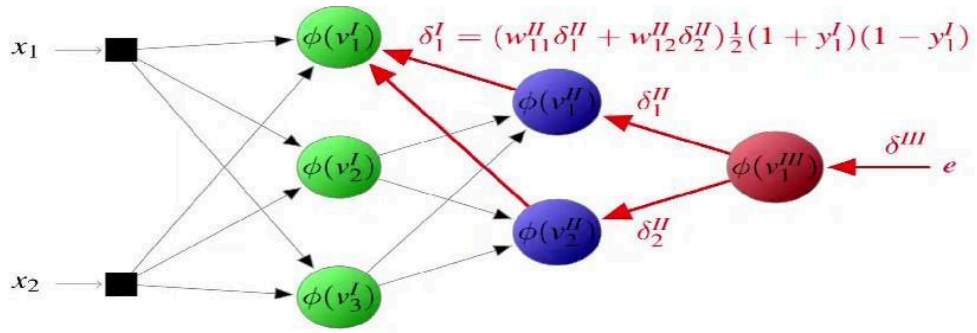
1. Inicialización aleatoria
2. Propagación hacia adelante
3. Propagación hacia atras
4. Adaptación de los pesos
5. Iteración: vuelve a 2 hasta convergencia o finalización

Repetimos (2-5) para todos los patrones (datos del archivo de entrenamiento), y a cada pasada con todos los datos se le llama **ÉPOCA**.

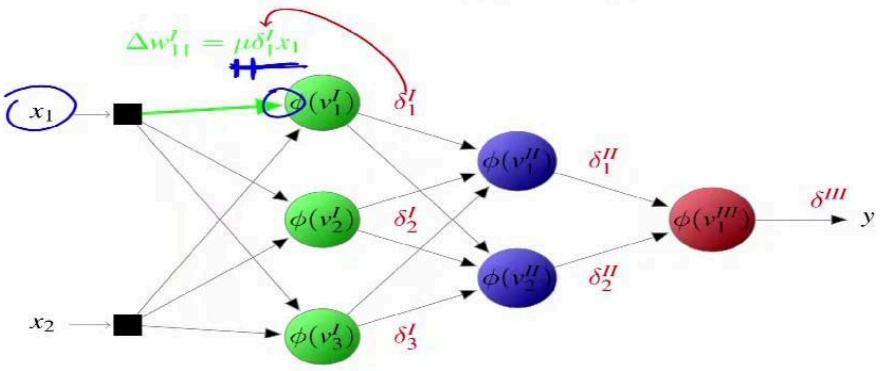
2.







3.

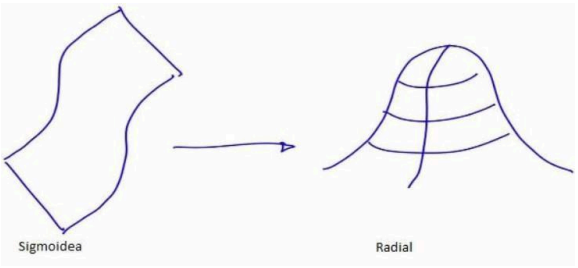


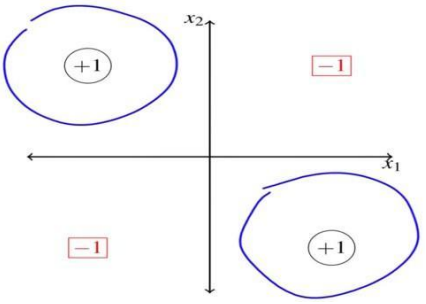
→ Aclaración: No importa el órden en que actualicemos los pesos, ya que ya tenemos calculadas las salidas de las neuronas.

Redes neuronales con funciones de base radial (RBF-NN)

El hecho de cambiar la función de activación nos permite cambiar la forma de las regiones de decisión, por ejemplo, en el caso del XOR:

Regiones radiales:





RBF: Funciones de base radial

Originalmente se usaban para aproximar funciones.

$$\phi : \mathbb{R}^N \rightarrow \mathbb{R}$$
$$d = \phi(\mathbf{x})$$

Aproximación:

$$h(\mathbf{x}) = \sum_j w_j \phi(\|\mathbf{x} - \boldsymbol{\mu}_j\|)$$

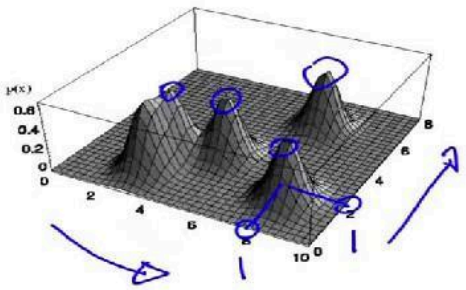
- Pesos $\mathbf{w}_j$ (equivalentes a los pesos sinápticos)
- Alguna función de base radial ϕ por la que se pasa una distancia entre nuestro patrón de entrada y un vector de medias $\boldsymbol{\mu}$.

- Vamos a usar esta **<u>FUNCIÓN GAUSSIANA</u>:**

$$\boldsymbol{\phi}(k) = e^{\left(\frac{-k^2}{2\sigma^2}\right)}$$

Aproximación: ejemplo

$$p(\mathbf{x}) = \sum_j w_j \phi(\|\mathbf{x} - \boldsymbol{\mu}_j\|)$$



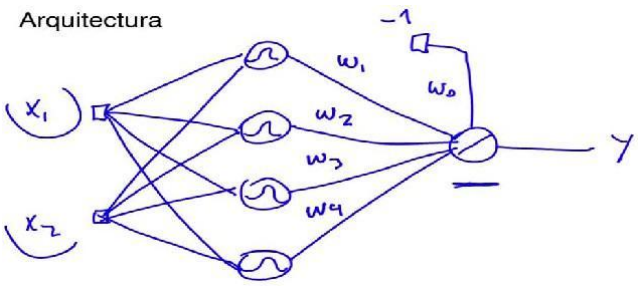
$\mu = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$ → MEDIA mu

RBF-NN: generalidades

- Arquitectura: ——————————————————

  - ***Capa radial:***
    - 1er capa
    - Puede tener muchas neuronas
    - Aquí están las **gaussianas** (en el ejemplo, en R2)
      - Cada una tiene un vector de medias $\boldsymbol{\mu}$ distinto
      - El valor de $\boldsymbol{\sigma}$ para todas
    - Tiene sus pesos sinápticos que conectan a las entradas con ellas
    - NO tienen sesgo/bias. (no tienen w_0)
    - En general se ponen todos pesos = 1.

  - ***Perceptrón de la salida final:***
    - Último perceptrón con una función lineal
    - Una constante que escala la salida
    - Tiene sus pesos sinápticos + el peso $\mathbf{w_0}$
    - Es un perceptrón simple con salida lineal



Como las entradas a las radiales se le ponen pesos todos en 1, se ajusta la relación entre las entradas y la gaussiana con los parámetros de la gaussiana, la *media* $\boldsymbol{\mu}$ y la *desviación estándar* $\boldsymbol{\sigma}$.

- Modelo matemático: ——————————————————

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^{M} w_{kj}\phi_j(\mathbf{x}_\ell)$$

- **k** → recorre las salidas (pueden ser varias)
- **j** → recorre las neuronas de la capa radial
- $\mathbf{w_{kj}}$ → los w de la capa de salida (los otros son todos 1)
- ϕ_j → función de base radial
- $\mathbf{x_l}$ → la entrada

Donde: $\phi_j(x_l) = e^{(-||x_l - \mu_j||^2 / 2\sigma_j^2)}$

→ Para cada **j** tenemos un vector $\boldsymbol{\mu_j}$ distinto

RBF-NN: entrenamiento

Tiene 2 ETAPAS, un entrenamiento mixto:

- **Método 1:**
  - Adaptación no supervisada de las RBF
    - Utilizando el **método k-medias**
    - Utilizando mapas autoorganizativos
    - Otros…
  - Adaptación supervisada de los w_{kj} (LMS)
- **Método 2:**
  - Inicialización por el Método 1
  - Adaptación supervisada de las RBF $\left(\frac{\partial \xi}{\partial \mu_{ji}}, \frac{\partial \xi}{\partial \sigma_j}\right)$ → ajuste de los parámetros de la red

→ En general se adaptan RBF y w_{kj} por separado.

¿Cómo entrenar la capa radial? — ENTRENAMIENTO PARTE 1

- Método **NO-supervisado**! (uno de los más simples)
- Se entrena *SIN NECESIDAD DE CONOCER LA SALIDA CORRECTA*

→ Adaptación de las RBF: k-medias

→ Objetivos:

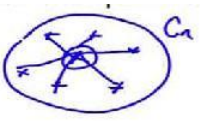
- Encontrar **k** (uno para cada neurona) conjuntos $\mathbf{C_j}$ de patrones de entrada de forma que:
  - Cada conjunto $\mathbf{C_j}$, sea *lo más **diferente*** posible de los demás
  - Los patrones $\mathbf{x_l}$ dentro de cada $\mathbf{C_j}$ sean *lo más **parecidos*** posible entre ellos
- Encontrar el centroide μ_j de cada conjunto $\mathbf{C_j}$ → luego serán los de las gaussianas en la neurona **j** correspondiente

=> lo que tenemos que obtener son los vectores μ_j , en la dimensión que corresponda, que van a ser las medias de las gaussianas ϕ_j , y el promedio de todos los patrones que hayan quedado en los conjuntos $\mathbf{C_j}$

Ecuación de optimización: $\min \left\{ J = \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\}$

→ Minimizar CRITERIO J, compuesto por:

- La distancia entre los patrones que corresponden al conjunto $\mathbf{C_j}$ menos el vector de medias μ de ese conjunto.
- Vamos acumulando todas las distancias anteriores de cada conjunto.



- Hacemos luego una sumatoria de eso para todos los conjuntos.

A. Adaptación de las RBF: k-medias *por lotes*

1. **Inicialización:** se forman los **k** conjuntos $\mathbf{C_j(0)}$ con patrones $\mathbf{x_l}$ elegidos aleatoriamente.
2. **Se calculan los centroides**: → hacemos un promedio (suma de vectores)

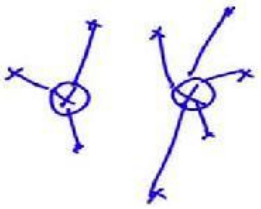
$$\boldsymbol{\mu}_j(n) = \frac{1}{|C_j(n)|} \sum_{\ell \in C_j(n)} \mathbf{x}_\ell$$

3. **Se reasignan los patrones x_l al conjunto C_j más cercano**:

$$\ell \in C_j(n) \Leftrightarrow \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 < \|\mathbf{x}_\ell - \boldsymbol{\mu}_i\|^2 \quad \forall i \neq j$$

el l-ésimo patrón pertenece al conjunto j, sí y solo sí la distancia entre ellos es menos que la distancia entre el patrón y el centroide de todos los demás conjuntos posibles.

Entonces =>
→ Hacemos que cada conjunto tome a los patrones que tiene más cerca, o
→ que cada patrón se vaya al conjunto cuyo centroide tenga más cerca.



4. Volver a **2** hasta que no se realicen reasignaciones.

B. Adaptación de las RBF: k-medias *online*

→ Ajuste patrón a patrón, uno a uno:

Dado un patrón intentamos minimizar este criterio general:

Optimización por método de gradiente:

$$\nabla_\mu J = \nabla_\mu \left\{ \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\} = 0$$

y obtenemos una regla:

$$\boldsymbol{\mu}_j(n+1) = \boldsymbol{\mu}_j(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_j(n))$$

1. **Inicialización:** se eligen **k** patrones aleatoriamente y se usan como centroides iniciales $\boldsymbol{\mu}_j(0) = \mathbf{x}'$ → un patrón al azar.

2. **Selección del centroide ganador:**

$$j* = \arg\min_j \{\|\mathbf{x}_\ell - \boldsymbol{\mu}_j(n)\|\}$$

3. **Adaptación:**

$$\boldsymbol{\mu}_{j*}(n+1) = \boldsymbol{\mu}_{j*}(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_{j*}(n))$$

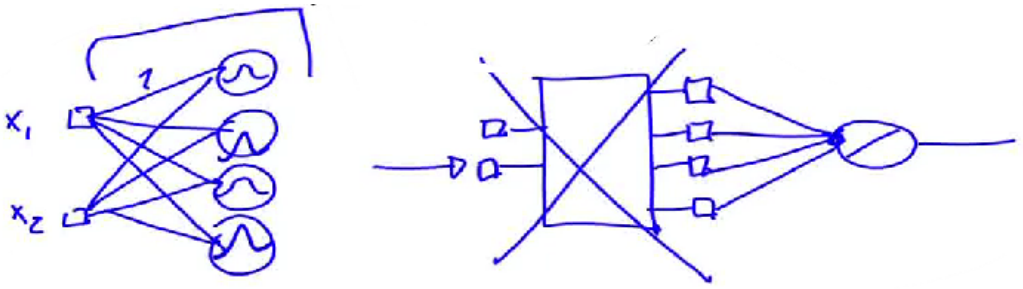
4. **Volver a 2** (por cada patrón) **hasta no encontrar mejoras significativas en J.**

¿Cómo entrenar la capa de salida? — ENTRENAMIENTO PARTE 2

- SÍ ES SUPERVISADO*
- Al entrenar los pesos, las RBF quedan fijas (dejamos fija la capa radial)
- Al estar las RBF fijas se pueden obtener las **salidas intermedias** para cada patrón de entrada:
- Con esas salidas intermedias, que las podemos considerar simplemente como entradas nuevas, se puede entrenar cada perceptrón simple, como ya sabemos:

$$\mathbf{y} = \mathbf{W}\boldsymbol{\phi}(\mathbf{x}_\ell)$$

→ estamos fijando el ϕ



→ **Métodos de entrenamiento para W:**

• pseudo-inversa del vector ϕ**(x$_l$)**

• gradiente descendiente sobre el error cuadrático instantáneo (**LMS**) →

Adaptación de los pesos: método LMS

Recordando la fórmula:

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

- Medimos el error:

$$e_k(n) = y_k(n) - d_k(n)$$

* → necesitamos la salida correcta

k → de la cantidad de salidas (cantidad de neuronas en la capa de salida)

- Lo usamos para el error cuadrático medio:

$$\xi(n) = \frac{1}{2}\sum_k e_k^2(n) = \frac{1}{2}\sum_k \left(\sum_j w_{kj}(n)\phi_j(n) - d_k(n)\right)^2$$

- Calculamos la derivada:

$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = (y_k(n) - d_k(n)) \frac{\partial}{\partial w_{kj}} \left(\sum_j w_{kj}(n)\phi_j(n) - d_k(n) \right)$$

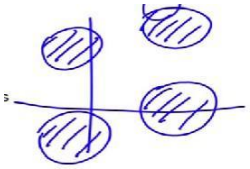
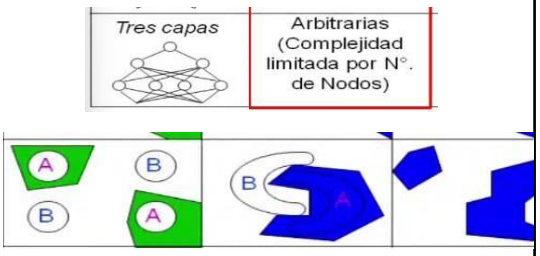
$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = e_k(n)\phi_j(n)$$ salida de la capa radial

Regla de aprendizaje:

$$w_{kj}(n+1) = w_{kj}(n) - \eta e_k(n)\phi_j(n)$$

$$w_{kj}(n+1) = w_{kj}(n) - \eta \left(\sum_i w_{ki}(n)\phi_i(n) - d_k(n) \right) \phi_j(n)$$

Comparación RBF-NN (redes con RBF) vs. MLP (perceptrones multicapa)

RBF-NN	MLP
Con una sola capa oculta puede resolver cualquier tipo de problemas 	Necesita *p* capas ocultas para problemas más complejos 
Tenemos solo las funciones de base radial	Tenemos que hacer combinaciones de hiperplanos sigmoideos
Representaciones locales, zonas acotadas del espacio, sumadas	Representaciones distribuidas, zonas no acotadas del espacio, combinadas
Convergencia más simple, no tenemos retropropagación	Tiene retropropagación
Arquitectura más simple	Arquitectura más compleja
Combinación de aprendizaje supervisado y no supervisado	Solo aprendizaje supervisado

Gaussianas N-dimensionales

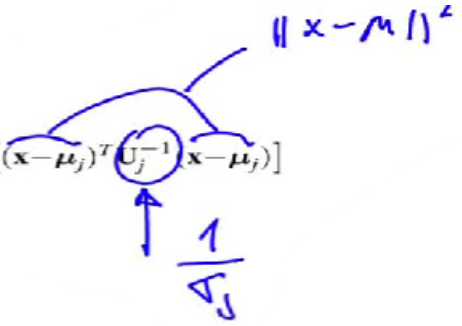
Concepto, interpretación gráfica y forma matricial:

- **x** → vector de entradas
- μ → vector de medias en R^N
- $\mathbf{U_j}$ → **matriz de covarianza** (Compuesto por las <u>desviaciones estándar</u>)
  - que modifican la forma de la campana

FUNCIÓN GAUSSIANA:

Forma general $\rightarrow \mathbf{x}, \boldsymbol{\mu}_j \in \mathbb{R}^N, \mathbf{U}_j \in \mathbb{R}^{N \times N}$:

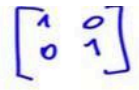
$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}|\mathbf{U}_j|^{1/2}} \cdot e^{-\frac{1}{2}[(\mathbf{x}-\boldsymbol{\mu}_j)^T \mathbf{U}_j^{-1} (\mathbf{x}-\boldsymbol{\mu}_j)]}$$

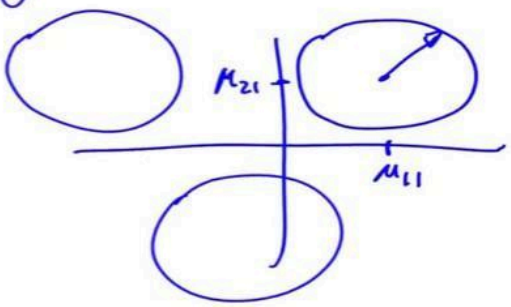


Análisis de distintos casos de U_j:

- **<u>U = matriz identidad</u>**
  - Caso simplificado 3 $\rightarrow \mathbf{U}_j = \mathbf{I}$:

$$\mathcal{N}'(\mathbf{x}, \boldsymbol{\mu}_j) = e^{-\frac{1}{2}\sum_{k=1}^{N}(x_k - \mu_{jk})^2}$$

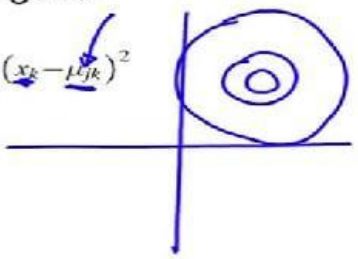




- σ → es = 1, fijo, no lo podemos modificar
- Como la varianza es 1 nos da un círculo, podemos tener varias neuronas pero cada una es un círculo, porque el desvió estándar al estar fijo no nos deja mover.
- Pero luego con los pesos en el perceptrón simple le podemos dar más o menos importancia, evitando así tener que calcular las desviaciones para modificar los círculos.
- Por lo general con este modelo se pueden resolver muchos problemas.

- **<u>U = diagonal con valores iguales, pero no necesariamente 1</u>**
  - Caso simplificado 2 $\rightarrow \mathbf{U}_j \in \mathbb{R}^{N \times N}$, diagonal igual:

$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}\sqrt{N}\sigma} \cdot e^{-\frac{1}{2\sigma^2}\sum_{k=1}^{N}(x_k - \mu_{jk})^2}$$



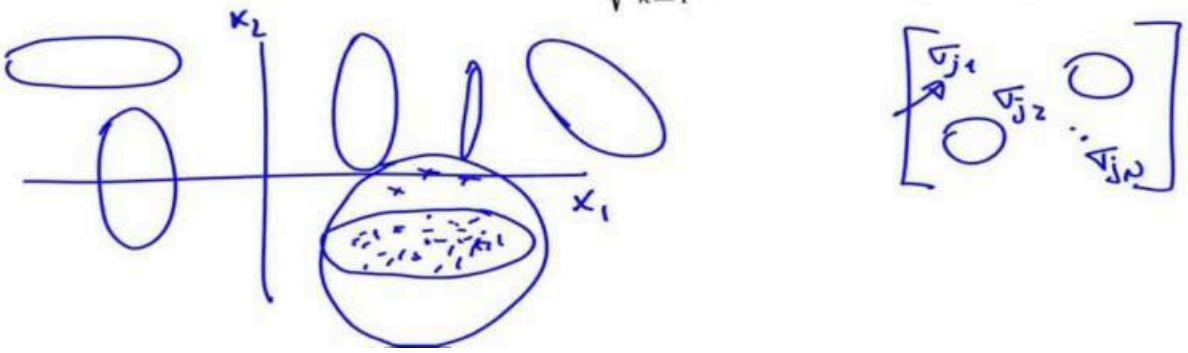
- σ → es un escalar, no depende de k ni j.
- Este nos permite modificar el σ, de manera que un σ grande permite un círculo grande y un σ pequeña un círculo pequeño

  => ¿Cómo calculo ese σ?
  - Si tengo el centroide calculado por el método de k medias, simplemente calculando las desviaciones de los patrones desde el centroide.

- **<u>U = diagonal con valores generales</u>**

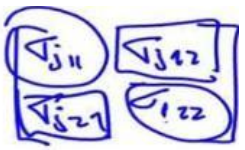
- Caso simplificado 1 → $\mathbf{U}_j \in \mathbb{R}^{N\times N}$, diagonal general:

$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}\sqrt{\sum_{k=1}^{N}\sigma_{jk}^2}} \cdot e^{-\frac{1}{2}\sum_{k=1}^{N}\frac{(x_k-\mu_{jk})^2}{\sigma_{jk}^2}}$$



- σ → depende de **k**, cada dimensión, y depende de **j**, de cada neurona
- <u>Esto nos permite generar elipses, no solo esferas, ya que cada dimensión tiene su varianza independiente de las otras.</u>
- DETALLE: → Solo las podemos poner en vertical u horizontal.

- **<u>FORMA GENERAL, U = matriz COMPLETA:</u>**
  - Podemos tener gaussianas inclinadas, ya que acá sí hay correlación entre las dimensiones, es decir que NO son independientes unas de otras.
  - Es el caso donde tendríamos la matriz COMPLETA:



- Forma general → $\mathbf{x}, \boldsymbol{\mu}_j \in \mathbb{R}^N, \mathbf{U}_j \in \mathbb{R}^{N\times N}$:

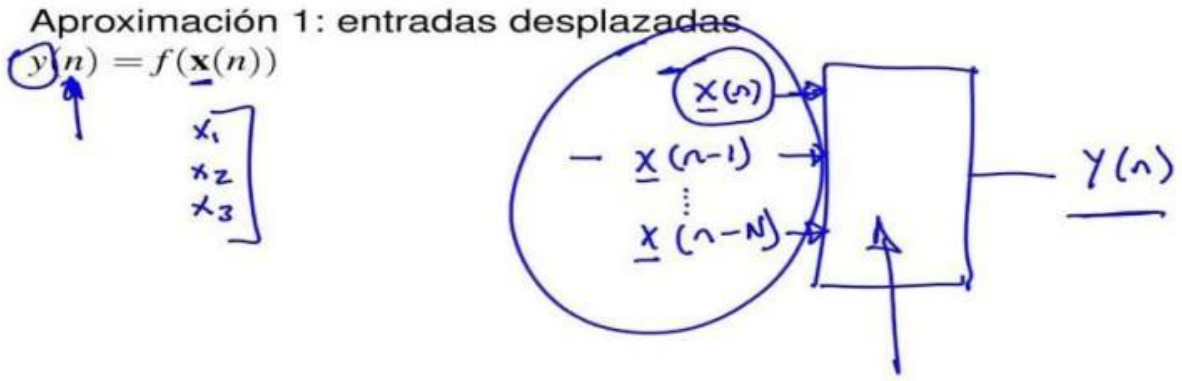
$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}|\mathbf{U}_j|^{1/2}} \cdot e^{-\frac{1}{2}[(\mathbf{x}-\boldsymbol{\mu}_j)^T \mathbf{U}_j^{-1}(\mathbf{x}-\boldsymbol{\mu}_j)]}$$

Redes Neuronales Dinámicas (DNN)

→ Son redes que internamente tienen un comportamiento dinámico que evoluciona a lo largo del tiempo.

¿Por qué dinámica?

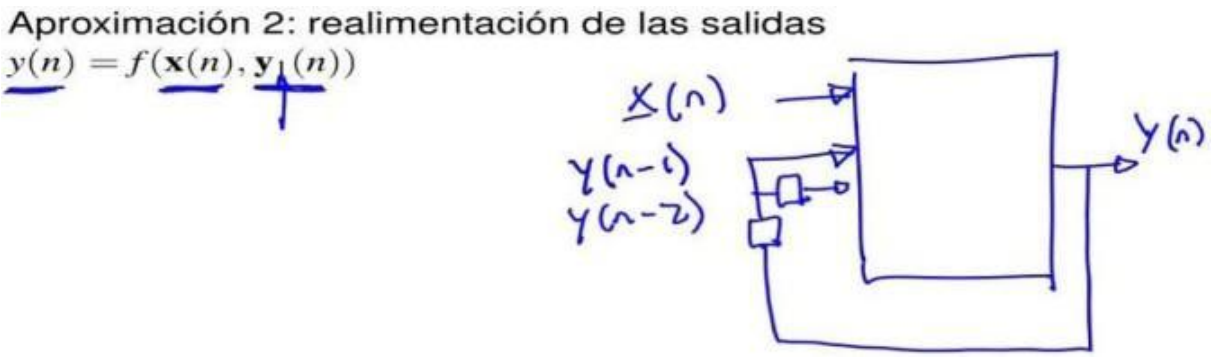
- Hay entradas que no solo contemplan el momento actual sino lo que valieron en instantes anteriores.



- **x** → Varias entradas, una salida (en este caso).
- Incorporamos una **n** = tiempo discreto.
- Planteamos una red <u>neuronal estática</u> cualquiera, y vamos a tener el vector x como entrada, y las entradas desplazadas en los instantes anteriores.

  => tenemos una ENTRADA EXPANDIDA, hasta N:.

  => la red sigue siendo estática, sigue viendo un patrón de entrada cada vez que le pedimos una salida.



Esta segunda aproximación considera las salidas en instantes anteriores, con un retardo, como entradas, aunque también sigue siendo una salida estática.



Esta tercera aproximación considera estados internos de la red neuronal de la capa oculta para alimentar a la red, empezando a tener un comportamiento más dinámico.

Finalmente:

Caso general:

$$y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n), \mathbf{y}_1(n))$$

Considera los 3 tipos anteriores, retroalimentandose con salidas, entradas e intermedios anteriores.

Clasificación de las redes neuronales dinámicas DNN

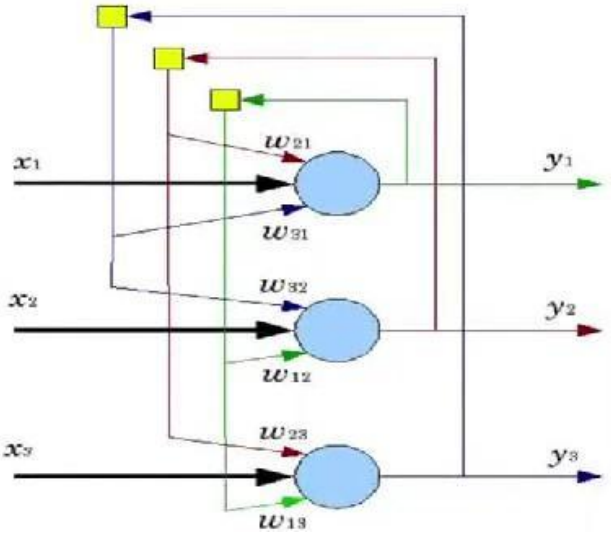
- Redes *con retardo en el tiempo*. (**TDNN**)
  Tienen un comportamiento dinámico, pero las podemos entrenar con los algoritmos que ya conocemos.
- Redes *recurrentes*. (**RNN**)
  Tienen realimentación hacia atrás (la sinapsis puede ir hacia atrás).
  - Redes ***totalmente recurrentes.***
    - ★ Redes de Hopfield (memorias asociativas)
    - ★ Redes de Boltzman (supervisadas)
    - ★ Teoría de la resonancia adaptativa (ART)
  - Redes ***parcialmente recurrentes***. (**PRNN**)
    - ★ Retro propagación a través del tiempo (BPTT)
    - ★ Redes de Elman.
    - ★ Redes de Jordan.

Redes de Hopfield → Redes totalmente recurrentes

- Se hace recurrencia a través de la salida.
  - Pero **no realimenta a la misma neurona de la que sale**, sino a las demás.
- Siempre hay la misma cantidad de entradas, salidas y neuronas.
- No hay capas, solo una.

- Se inicializa con impulsos de entrada, y luego las entradas son las salidas de los instantes anteriores, no vuelven a entrar las entradas originales.
- Tiene función signo → salida +1 y -1
- Los pesos sinápticos son simétricos, y el de la misma neurona es 0.

$$w_{ji} = w_{ij} \quad \forall\, i \neq j$$
$$w_{ii} = 0 \quad \forall i$$



- Modelo Matemático:

$$y_j(n) = sgn(x) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1) - \theta_j\right) \cdots \begin{cases} x > 0 & +1 \\ x = 0 & y_j(n-1) \\ x < 0 & -1 \end{cases}$$

- El comportamiento es bastante parecido al perceptrón, se puede usar la función signo, los pesos se multiplican por la entrada y se suman, hay un umbral o sesgo.
- Pueden tener 3 salidas, pero cuando es 0 la salida es la del instante anterior.

Redes de Hopfield - Generalidades

- Cada neurona tiene un *disparo probabilístico*
- Conexiones *simétricas*
- El entrenamiento es *no-supervisado*
- Pueden utilizarse como *memoria asociativa*

Almacenamiento y recuperación - RdH → Memorias asociativas

Utilizaremos las redes de Hopfield para realizar una memoria asociativa. El proceso de almacenamiento es el entrenamiento de la red y para retomar el contenido solamente se usa o prueba la red.

Almacenamiento (ENTRENAMIENTO)

Dado un conjunto de patrones (memorias fundamentales o datos limpios) x_{k*}:

$$X^* = \{\mathbf{x}_k^* \in \mathbb{R}^N\}$$

El aprendizaje o almacenamiento se denomina Hebbiano:

- Hacemos una especie de promedio de los patrones.

Aprendizaje Hebbiano:

$$w_{ji} = \frac{1}{N} \sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$

- *El peso $\mathbf{w_{ij}}$, que vincula la neurona **i** con la neurona **j**, promedia las entradas de **i** y **j** para todos los patrones, haciendo un producto sobre todos los patrones **k**.*
- ¿Cómo se interpreta esto?:
- Estamos tratando de averiguar qué pasa con la conexión que hay la neurona **i** y la **j**
    - Si ambas siempre son -1 o ambas 1→ el peso se hace positivo y grande
    - Si uno siempre es -1 y otro +1 → el peso se hace negativo y grande
    - Si algunas veces son +1 y -1 o -1 y +1 → el peso va a ser cercano a cero.

  => **Este método está detectando qué neuronas se activan a la vez.**

- **LOS PESOS GUARDAN UNA MEMORIA DE QUÉ TIPO DE RELACIÓN HAY ENTRE LOS PUNTOS/PÍXELES/DIMENSIONES DE LAS ENTRADAS QUE LE VOY MOSTRANDO.**

Observaciones:

- **NO** es un aprendizaje **ITERATIVO.**
- w_{ij} es mayor cuando las neuronas **i** y **j** se tienen que activar juntas (regla de Hebb)
- Hay un límite teórico para la capacidad de almacenamiento:

La cantidad de memorias fundamentales **Pmax** está limitada por la cantidad de neuronas **N**:

$$P_{max} = \frac{N}{2\ln(N)}$$, con un 1% de error.

Prueba (RECUPERACIÓN)

Se le da la misma entrada o una parecida, como la cara de una persona desde otro Angulo y quiero recuperar la imagen limpia.

=> Dado un patrón **x** (incompleto, ruidoso..):

- Inicializar la salida en el instante cero con la entrada:

$$\mathbf{y}(0) = \mathbf{x}$$

- Iteración:
  1. Elegimos al azar alguna de las neuronas:

$$1.\ j^* = rnd(N)$$

  2. Para esa neurona calculamos su salida:

$$2.\ y_{j^*}(n) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1)\right)$$

  3. Volvemos a 1 hasta no observar cambios en las y

     |

     |

     Una vez que hacemos una pasada completa de todas las neuronas y ninguna cambió => **la red converge a una memoria fundamental.**

Observaciones:

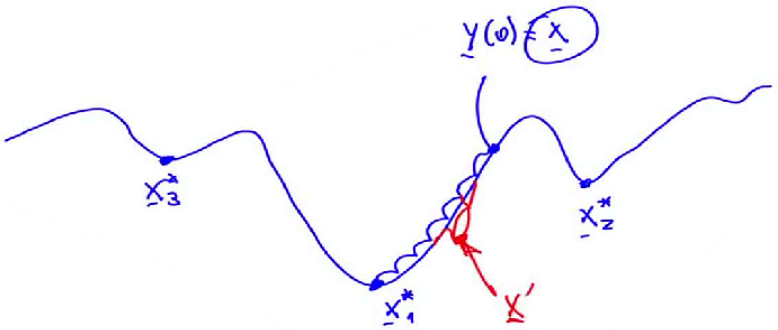
- **ES** un proceso **ITERATIVO** (*dinámico*→ itera hasta estabilizar).
- En general no se utilizan los umbrales θ_j
- Se pueden obtener estados espurios y oscilaciones…*

Estas redes, en el entrenamiento y en la prueba, SON JUSTO LO INVERSO A LAS REDES ESTÁTICAS.

***Campos energéticos de Hopfield**
Se estudia en detalle el caso en que una red queda oscilando, tanto *porque le dimos más memorias fundamentales de las que puede almacenar o porque hay memorias que son muy parecidas.*

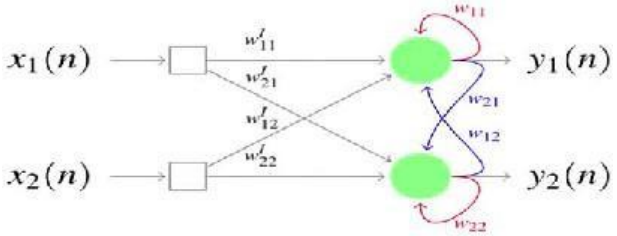
Los campos energéticos son unas distribuciones que podemos observar en una dimensión como zonas de picos y de valles.

- Esa superficie se genera durante el entrenamiento, y, **si todo salió bien**:
  - Las memorias fundamentales se almacenan en los valles.
  - El proceso de recuperación lo que hace es ir bajando, se va acercando hacia una de las fundamentales.
- En caso de que esto **no esté bien hecho**:
  - Puede que se generen valles espudios.
  - Cuando esté bajando caigamos en ellos y veamos una memoria que en realidad no existe.



Retro propagación a través del tiempo (BPTT) → Redes pte recurrentes

- Cualquier cantidad de entradas y salidas, y la arquitectura que sea.
- Hay también pesos sinápticos entre las neuronas, todas con todas, y también consigo mismas.

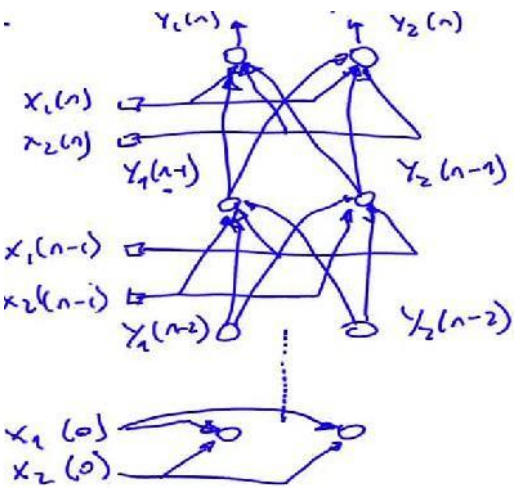


EXPANSIÓN: propagación hacia adelante pura

- En un instante **n**, además de las entradas, a las neuronas van a entrar las salidas de ellas mismas pero de un instante anterior **n-1**.

¿Y cómo se calculan las salidas en el instante **n-1**?

- Podemos pensarlo de la misma manera, expandiendo hasta llegar al instante inicial **n=0**... y obtener:

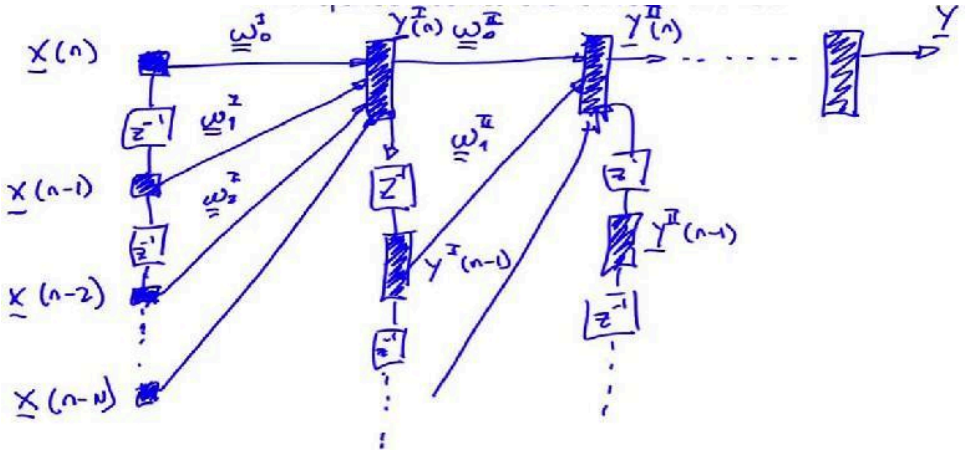


→ Nos quedó una red con muchas capas, pero que <u>ahora es solamente hacia adelante</u> no tiene retroalimentaciones

=> la podemos entrenar por retropropagación como hicimos con el perceptrón multicapa, con ciertos cuidados.

podemos truncar hasta N instantes anteriores.

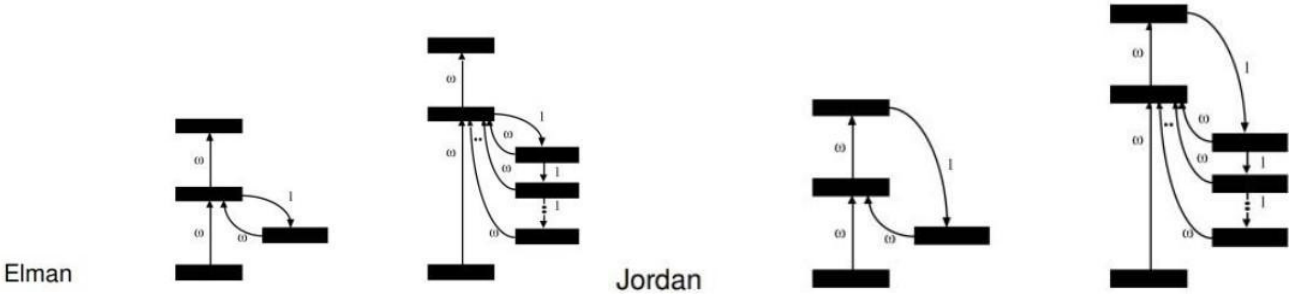
Redes neuronales con retardos en el tiempo (TDNN)



- Tenemos <u>nuevos conjuntos de pesos</u> que sirven para tener en cuenta la historia de lo que fue pasando en el tiempo/espacio.
- <u>Puede ser entrenada con una retropropagación</u> como el perceptrón multicapa.
- Las primeras capas tienen memoria más inmediata, y a medida que avanzan, las capas van teniendo memoria de más largo plazo, ya que son la memoria de la memoria de la memoria…
  - → esto se conoce como clasificación espacio/temporal.

Arquitecturas de Elman y Jordan

- En **Elman** se considera un retardo de la capa intermedia hacia sí misma, o sea que la salida de la capa intermedia se guarda y el instante siguiente se vuelve a ingresar.
- En **Jordan** la realimentación la toman de la salida.



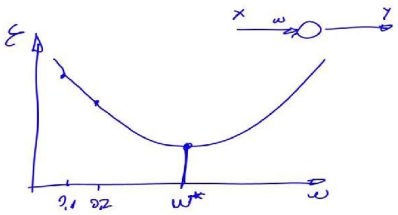
UNIDAD 4 - Aprendizaje Automático

Capacidad de generalización

Superficies de error

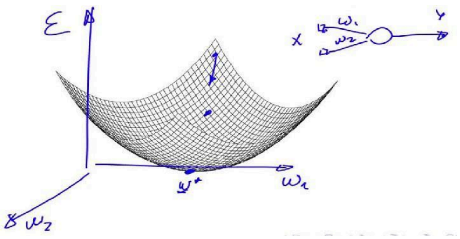
Suponiendo que hay un único parámetro (peso w) a ajustar:

- Podemos medir un error que comete el sistema con distintos valores del parámetro real w.
- => podríamos graficar una curva de error

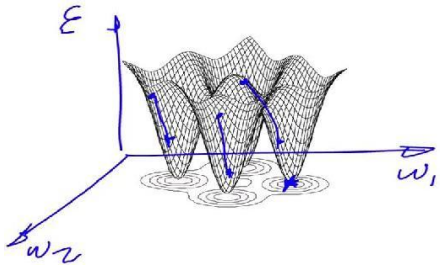


Con 2 dimensiones:

- Podemos graficar una superficie de error:



¿Y si hay muchos mínimos?



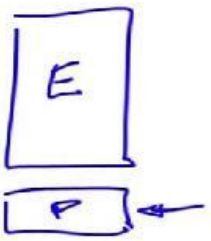
→ dependiendo de dónde arranque el algoritmo de entrenamiento, va a elegir uno u otro, sin encontrar el **mínimo global**

Una opción → inicializar el alg. con varios puntos al azar y quedarnos con el mínimo.

Estos errores los medimos durante el entrenamiento, pero...

- ¿El modelo es *capaz* de funcionar bien en casos más *generales*?
- ¿Cuál será el error en casos nunca vistos durante el entrenamiento?

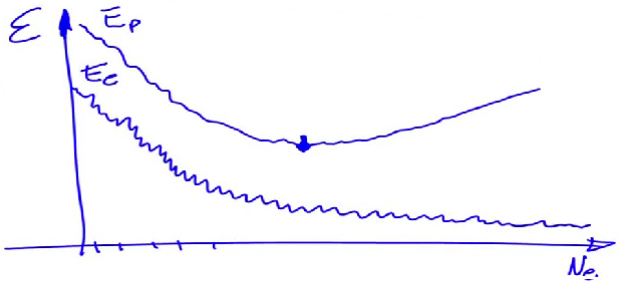
=> para ello hacemos una prueba.



SOBRE-ENTRENAMIENTO:

→ Si un modelo se ajusta demasiado a los datos de entrenamiento y pierde la capacidad de funcionar bien con datos que nunca vió.

- Si observamos ***la cantidad de épocas*** **Ne** contra los errores:

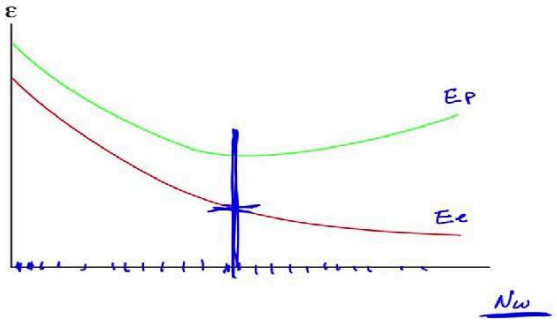


N_e → número de épocas.
E_e → error entrenamiento.
E_p → error prueba.

Vemos que luego de un cierto número de épocas, *el error de la prueba se va a estancar, y luego va a empezar a subir.*

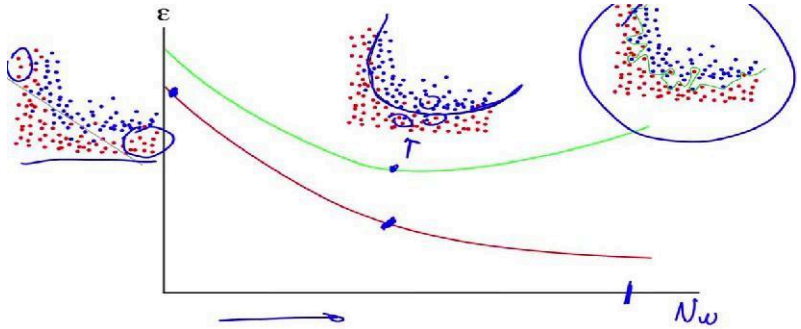
→ Esto se debe a que **el modelo se aprendió detalles de los datos de entrenamiento que no son detalles generales del problema que debe aprender realmente**.

- Si en lugar de la cantidad de épocas, *ahora usamos la* ***cantidad de neuronas*** **Nw**, parámetros libres, que tiene la red:



→ Observamos un comportamiento similar al anterior.

→ Ya que a mayor cantidad de parámetros libres, más capacidad tiene la red para modelar todos los detalles finos que tienen los datos de entrenamiento.



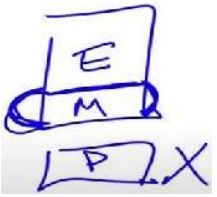
¿Qué puedo hacer para EVITAR EL SOBRE-ENTRENAMIENTO?

→ *Cuando vemos que el error de prueba sube, cortamos el entrenamiento*.

→¿Pero no estamos ajustando según los datos de **prueba** entonces?
→ sí, y en realidad estos no deberán usarse para nada

=> tenemos que tener una estrategia para medir sólo con los de entrenamiento:

easy → nos guardamos una porción de los datos de entrenamiento y le llamamos un subconjunto de monitoreo.



¿Cómo podemos ESTIMAR la capacidad de generalización?

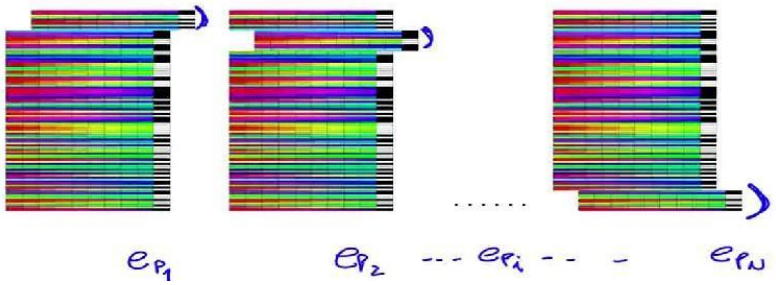
(que tan bien va a funcionar con datos que no son estrictamente los que tengo para entrenarlo y esa es la verdadera tasa de error acierto que quiero conocer)

Validación cruzada

a. Leave-k-out

Rep gráfica: color según las intensidades de los valores

1. Reordenar al azar los datos.
2. Dejo k casos afuera para la prueba, el resto para entrenar, y podemos tener un subset de monitoreo en él para ir midiendo durante el entrenamiento.
3. Hago el entrenamiento.
4. Vuelvo a comenzar de cero y repito este mecanismo tomando otro conjunto de prueba para variar los datos que estaban en cada uno de los 3 conjuntos.
5. Vamos a terminar con N errores de prueba.



6. Finalmente calculamos el **error promedio** como el promedio de ellos:

$$\mathcal{E} = \frac{1}{N} \sum_{i=1}^{N} e_{P_i}$$

y **este va a ser el verdadero error que me interesa**.

→ También es útil medir la **VARIANZA del error**:

- Si es pequeña => las particiones son buenas.
- Si es grande => es porque hay valores de errores muy diferentes entre ellos, lo cual nos indicaría que hay algún problema con los datos.

b. Leave-1-out

- Consiste en entrenar y luego probar con 1 solo ejemplo, lo cual implica, si tenemos N datos, repetirlo N veces.
- El problema es que es pesado.
- Pero como ventaja, tiene un sesgo del error que estima muy pequeño.

c. Solapar las particiones

- Pierdo la independencia estadística de los experimentos

Medidas de desempeño para clasificadores (binarios)

Matriz de confusión

		Predicciones ⊕	Predicciones ⊖	
Clases correctas	⊕	$t_\oplus$	$f_\ominus$	$N_\oplus = t_\oplus + f_\ominus$
	⊖	$f_\oplus$	$t_\ominus$	$N_\ominus = f_\oplus + t_\ominus$
				$N = N_\oplus + N_\ominus$

- Verdaderos positivos → dijo + y era +
- Falsos negativos → dijo - y era +

→ sumados = cantidad de positivos

- Falsos positivos → dijo + y era -
- Verdaderos negativos → dijo - y era -

→ sumados = cantidad de positivos

→ sumando ambas cantidades = total de casos

- **Sensibilidad**

$$s^+ = \frac{t_\oplus}{t_\oplus + f_\ominus} = \frac{t_\oplus}{N_\oplus} \longrightarrow \left\{ \begin{array}{l} \textbf{sensitivity} \\ \text{true positive rate} \\ \text{hit rate} \\ \text{recall} \\ \text{power} \end{array} \right.$$

- **Especificidad**

$$s^- = \frac{t_\ominus}{t_\ominus + f_\oplus} = \frac{t_\ominus}{N_\ominus} \longrightarrow \left\{ \begin{array}{l} \textbf{specificity} \\ \text{true negative rate} \end{array} \right.$$

- **Precisión**

$$p = \frac{t_\oplus}{t_\oplus + f_\oplus} \longrightarrow \left\{ \begin{array}{l} \textbf{precision} \\ \text{positive predictive value} \end{array} \right.$$

$$n = \frac{t_\ominus}{t_\ominus + f_\ominus} \longrightarrow \left\{ \text{negative predictive value} \right.$$

Exactitud y otras medidas compuestas

- **Exactitud**
  Es la más intuitiva y la más utilizada, se suman los verdaderos positivos y los verdaderos negativos y luego lo dividimos por la cantidad de casos.

$$a = \frac{t_\oplus + t_\ominus}{N} \longrightarrow \left\{ \textbf{accuracy} \right.$$

- **F1 score**
  Está focalizada en los positivos, ya que suele ser la que más interesa. Se combina con una media armónica la **sensibilidad** con la **precisión**.

$$F_1 = \frac{2t_{\oplus}}{2t_{\oplus} + f_{\oplus} + f_{\ominus}} = 2\frac{s^+ p}{s^+ + p} \longrightarrow \begin{cases} F_1 \textbf{ score} \\ \text{F-score/measure} \\ \text{harmonic mean} \end{cases}$$

- **Media geométrica**
  También relaciona la sensibilidad y precisión, que también nos da una idea de que también está andando el clasificador.

$$G = \sqrt{s^+ p} \longrightarrow \begin{cases} \textbf{G-measure} \\ \text{geometric mean} \end{cases}$$

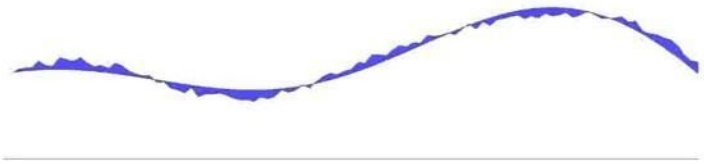
Errores relativos

Tenemos 2 clasificadores, y queremos comparar el desempeño $\mathbf{a_1}$ de uno relativo al $\mathbf{a_2}$ otro:

- $\mathbf{e_r = 1 - a_2}$, $\mathbf{e = 1 - a_1}$
- => el error relativo a la referencia $\mathbf{e_r}$ es: $\delta_e = \frac{e_r - e}{e_r}$

Medidas de desempeño para la predicción en series

La idea es medir con el área entre la referencia y la curva que se predijo:



- Error medio absoluto → $\tilde{\varepsilon}_A = \frac{1}{N}\sum_{i=1}^{N} |e_i|$ (MAE/MAD)
- Error cuadrático medio → $\tilde{\varepsilon}_S = \frac{1}{N}\sum_{i=1}^{N} e_i^2$ (MSE)
- Raíz cuadrada del error cuadrático medio → $\tilde{\varepsilon}_R = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}$ (RMSE)

Aprendizaje automático “no neuronal”

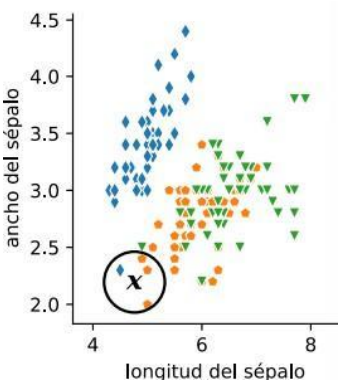
kNN - k-Nearest Neighbors

- Método NO paramétrico
- En realidad no aprende
- Vuelve a buscar los más cercanos con cada nuevo caso a clasificar (basado en instancias).

Clasificador kNN

Supongamos que tenemos:

- **N_k** puntos de la clase **C_k**
- **N** puntos en total $(\sum_k N_k = N)$



Para un nuevo **x**, podemos imaginar una esfera centrada en **x**,

→ Supongamos que esta esfera:

- tiene un volumen **V**
- contiene **K** puntos en total
- de los cuales **Kk** son de la clase **Ck**

La ***densidad de la clase k*** en la esfera sería → $\mathrm{p}(\mathbf{x}|C_k) = \frac{K_k}{N_k V}$

Y *si no condicionamos a la clase* tendríamos → $\mathrm{p}(\mathbf{x}) = \frac{K}{NV}$

Lo que podríamos esperar para cada clase sería → $\mathrm{p}(C_k) = \frac{N_k}{N}$

→ Si con esto usamos el **teorema de Bayes**

$$\mathrm{p}(C_k|\mathbf{x}) = \frac{\mathrm{p}(\mathbf{x}|C_k)\mathrm{p}(C_k)}{\mathrm{p}(\mathbf{x})} = \frac{\frac{K_k}{N_k V}\frac{N_k}{N}}{\frac{K}{NV}} = \frac{K_k}{K}$$

= probabilidad de que el patrón dado que sea de la clase, la probabilidad del patrón / probabilidad de clase

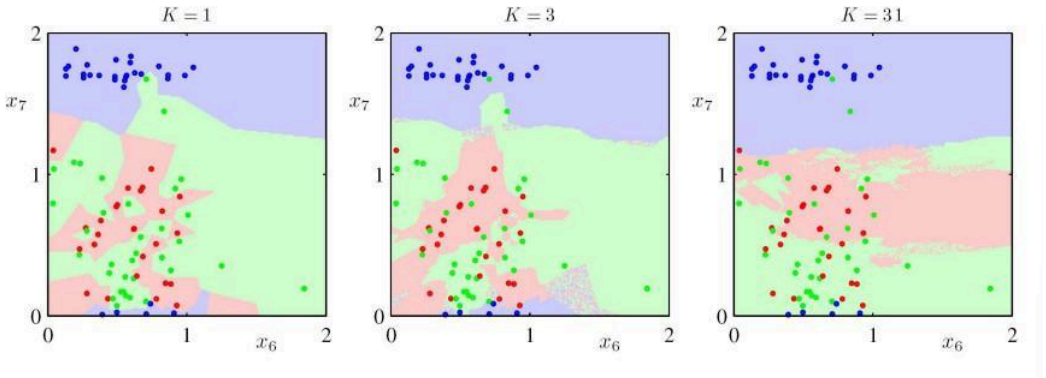
Ventajas:

- ✔ Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- ✔ No hay parámetros entrenables
- ✔ Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento
- ✔ Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

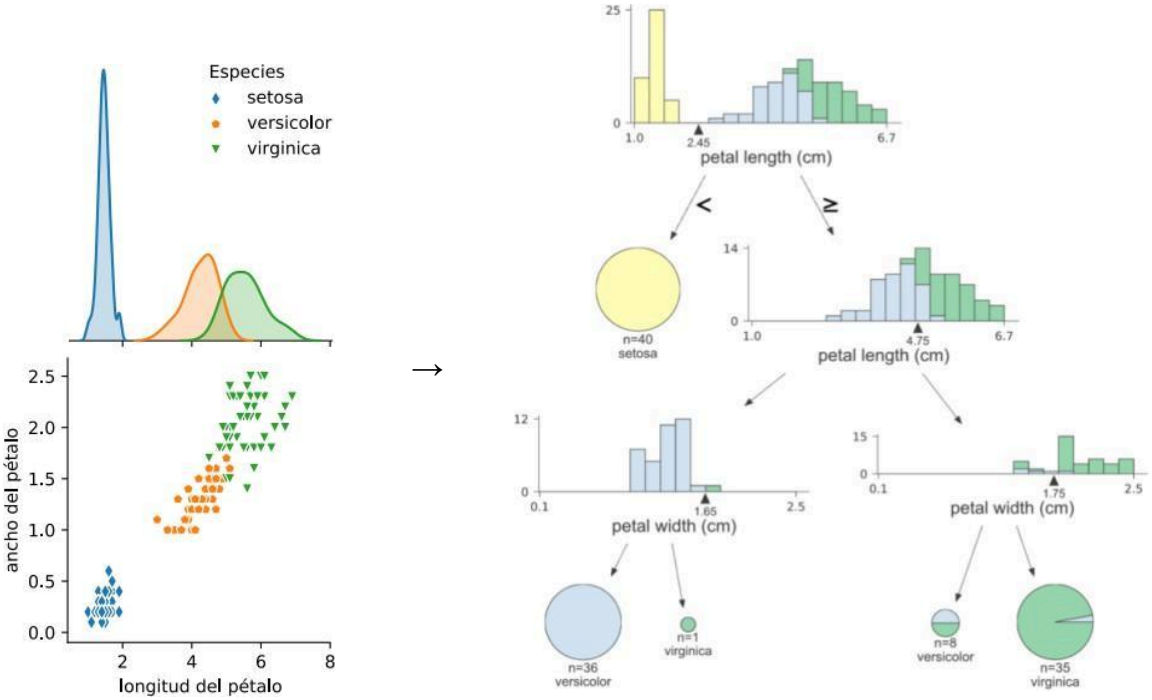
- ✖ Hay que determinar el valor óptimo de k.
- ✖ No es tan claro que distancia usar en cada caso.
- ✖ Para cada dato de prueba se requiere calcular la distancia a todos los datos de entrenamiento.
- ✖ Sensible a outliers y clases desbalanceadas.

A más vecinos, más detalle:



(Voronoi)

Árboles de decisión



- ¿Cómo elegimos por qué variable dividir primero?
- ¿Cómo elegimos el punto de división?

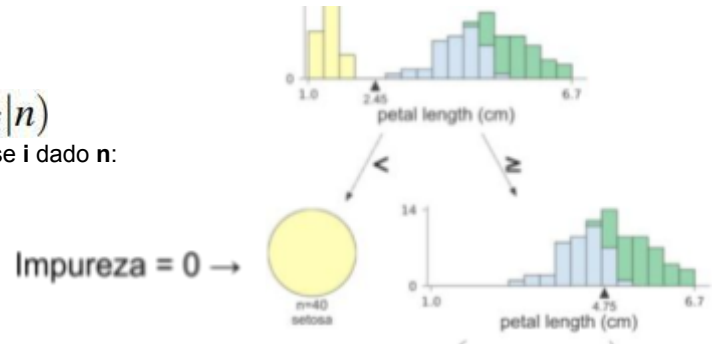
→ Se usan los índices de impurezas:

Medidas de impureza

Impureza de clasificación:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Probabilidad de que **x** pertenezca a la clase **i** dado **n**:



→ Empezamos con la variable que nos de menos impureza, según el n del cálculo.

Impureza de Gini: → Es la que más se usa

$$\mathcal{I}_n^G = \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i|n)\mathsf{P}(\mathbf{x} \in w_j|n)$$
$$= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i|n)$$

= sum(Probabilidad de que **x** pertenezca a la clase **i** dado **n***Probabilidad de que **x** pertenezca a la clase **j** dado **n)**

Impureza de Entropía: *(Entropía → "Desorden")*

$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i|n) \log \mathsf{P}(\mathbf{x} \in w_i|n)$$

→ Si queda todo de un lado, w_i, nos va dar cero (son todos iguales).

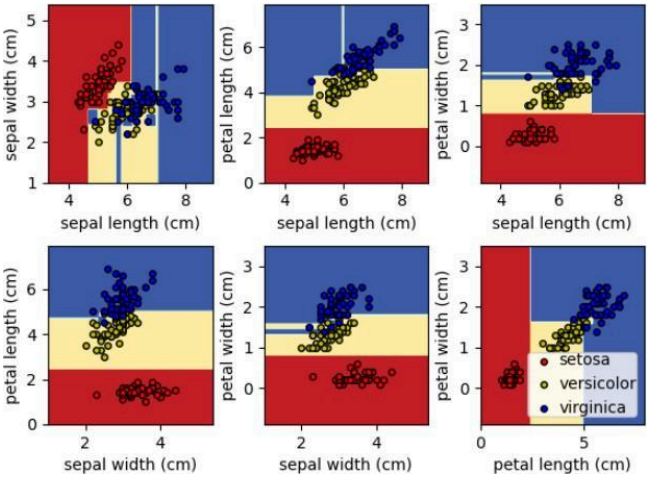
Ganancia de información:

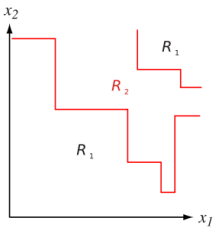
G = $\mathcal{I}$ **p** $-\sum$ **(Nh/N** $\mathcal{I}$ **h)**

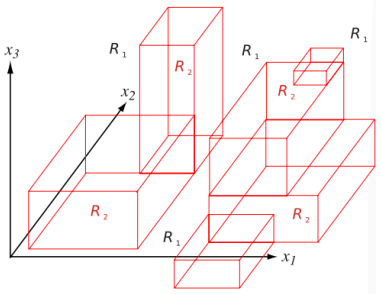
= impureza del padre - impureza de sus hijos que quedan

→ Mide cuánta información gané haciendo esa partición de la información.

Fronteras de decisión:







Ventajas:

- ✔ Proveen una ***explicación simple de la decisión*** con reglas IF-THEN.
- ✔ Trata de forma natural tanto *datos numéricos como categóricos* (y a la vez) sin requerir transformaciones.
- ✔ Tienen *bajo costo computacional* tanto para el entrenamiento como para su uso en producción.

✔ No requieren grandes cantidades de datos para entrenar.
✔ Bastante *robustos a los outliers*.

Desventajas:

✖ *No son robustos a ruido* (pequeños cambios) en los datos.
✖ *Fronteras de decisión ortogonales a las dimensiones* de entrada.
✖ No escalan bien con la complejidad de los problemas a resolver.
✖ *Se sobreajustan* con facilidad.

Clasificadores Bayesianos Ingenuos (Naive Bayes)

Construyamos un clasificador usando el teorema de Bayes:

$$\mathsf{P}(w|\mathbf{x}) = \frac{\mathsf{P}(\mathbf{x}|w)\mathsf{P}(w)}{\mathsf{P}(\mathbf{x})}$$

Probabilidad de que sea la clase w, dado el patrón x

...y supongamos que *las dimensiones o características son condicionalmente independientes* (esta es la parte **más ingenua**):

$$\mathsf{P}(x_i|w, x_j) = \mathsf{P}(x_i|w) \quad \forall i \neq j.$$

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

y usando en cadena esta misma definición:

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, \ldots, x_n) &= \mathsf{P}(x_1, x_2, \ldots, x_n|w)\mathsf{P}(w) \\
&= \mathsf{P}(x_2, x_3, \ldots, x_n|w, x_1)\mathsf{P}(w)\mathsf{P}(x_1|w) \\
&= \mathsf{P}(x_3, x_4, \ldots, x_n|w, x_1, x_2)\mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w, x_1) \\
&= \ldots
\end{aligned}$$

pero con la suposición ingenua:

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, .., x_n) &= \mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w)\ldots\mathsf{P}(x_n|w) \\
&= \mathsf{P}(w)\prod_{i=1}^{n}\mathsf{P}(x_i|w)
\end{aligned}$$

Me quedan las probabilidades de cada columna (entrada), dada la clase, las cuales podemos calcular fácil con los datos.

De esta forma nos queda

$$\mathsf{P}(w|\mathbf{x}) = \frac{\mathsf{P}(\mathbf{x}|w)\mathsf{P}(w)}{\mathsf{P}(\mathbf{x})} = \frac{\mathsf{P}(w)\prod_{i=1}^{n}\mathsf{P}(x_i|w)}{\mathsf{P}(\mathbf{x})}$$

Y considerando la misma probabilidad P(w) para cada clase, y que P(x) no depende de w, la clase más probable se puede obtener mediante:

$$\hat{w}(\mathbf{x}) = \arg\max_{w} \prod_{i=1}^{n} \mathrm{P}(x_i|w)$$

Para cada clase medís la productoria para cada clase,, y te quedas con la que da más grande.

NB: solo nos queda calcular las P(x_i|w)

NB Gaussiano $\mathcal{N}(\mu, \sigma^2)$

$$\mathrm{P}(x_i|w) = \frac{1}{\sigma_w\sqrt{2\pi}} e^{-\frac{(x_i-\mu_w)^2}{2\sigma_w^2}}$$ (caso continuo)

Ventajas:

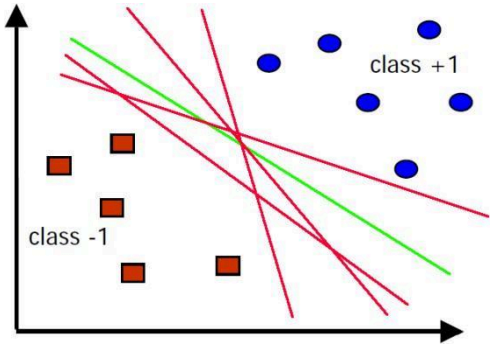
- ✔ Puede usarse directamente en mezclas de datos continuos y categóricos.
- ✔ En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- ✔ El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- ✔ Rápido y simple de implementar.

Desventajas:

- ✖ La hipótesis de independencia no se cumple en muchos datos reales.
- ✖ Suele presentar problemas de estimación durante el entrenamiento.

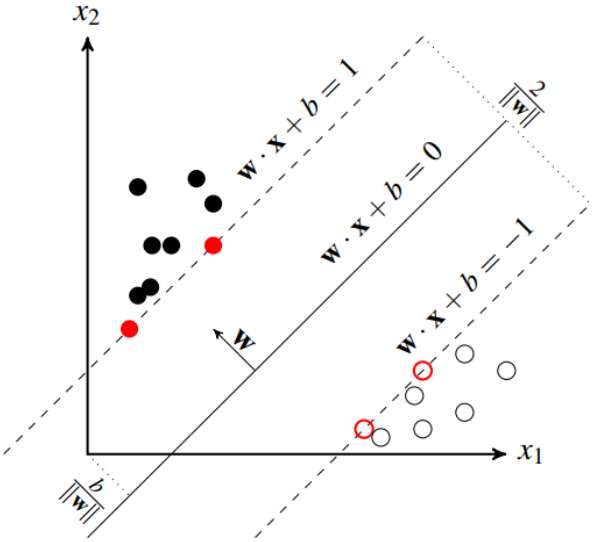
Máquinas con vectores de soporte - (SVM Support Vector Machines)

<u>Margen: ¿cuál sería el mejor clasificador?</u>

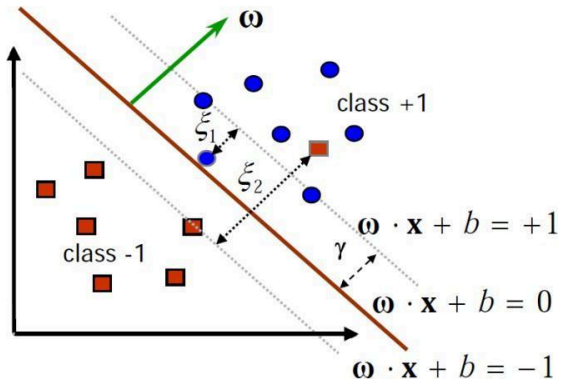


→ recta lo más lejos posible de los patrones con los que empieza la clase

→ Se maximiza el márgen:



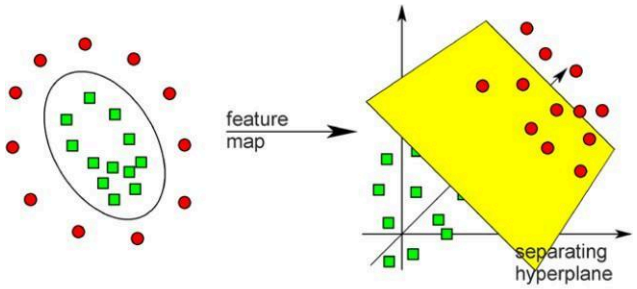
SVM: márgenes blandos



→ Medidas respecto de los nodos que elegimos de soporte.

El truco del kernel

Proyectar los datos a una dimensión superior para que yo pueda atravesar con un hiperplano a los datos.



$$\phi : \mathbb{R}^2 \to \mathbb{R}^3$$

$$\phi : (x_1, x_2) \mapsto (z_1, z_2, z3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$
- Polinomial: $(\mathbf{x} \cdot \mathbf{z} + c)^d$
- Sigmoideo: $\tanh\left(c(\mathbf{x} \cdot \mathbf{z}) + \theta\right)$

Ventajas:

- ✔ *Rápida convergencia* en entrenamiento.
- ✔ Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- ✔ Funciona bastante *bien en altas dimensiones de entrada*.
- ✔ Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- ✔ Suele proveer buena capacidad de generalización. (maximizan el márgen)

Desventajas:

- ✖ El *desempeño es muy dependiente del kernel* que se elija (y sus hiperparametros).
- ✖ Tiene problemas con grandes conjuntos de datos.
- ✖ No maneja bien el solapamiento de clases.
- ✖ No provee buena interpretabilidad de las decisiones.
- ✖ No es directa la extensión a problemas multiclase.

Ensambles de clasificadores

→ *Combinar varios modelos simples en lugar de construir un modelo complejo.*

Bagging - Bootstrap AGGregatING

Entrenar varios modelos simples…
haciendo *bootstrap* sobre el conjunto de datos

1. Entrenar ***B*** modelos separados, cada uno para predecir $y_b(x)$.
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(x)$ (o promedio en caso de regresión).

$$yens(x) = \sum_{b=1}^{B} yb(x)$$

Se puede demostrar que el error del ensamble E_E en relación al promedio de los errores individuales $\tilde{E}_b$ es:

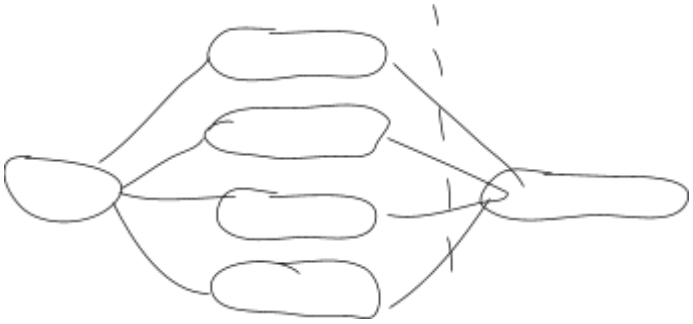
$$\tilde{E}_E = \frac{1}{B}\tilde{E}_b$$

→ Disminuye con la cantidad

(Random Forests)

Stacking

Entrenas varios modelos simples y luego un meta modelo recibe como entrada las salidas de los modelos más simples.



Se entrenan por separado los modelos simples del metamodelo.

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap.
- *Buscamos que la nueva muestra sea más informativa para el nuevo modelo a entrenar.*

Boosting por muestreo 3

Dado un conjunto de entrenamiento ***S*** de ***n*** patrones, para un clasificador **binario**:

1. Se entrena el clasificador ***C1*** usando la muestra de bootstrap **S1** de ***n' < n*** patrones (sin reposición).
2. Se entrena el clasificador ***C2*** en un conjunto de entrenamiento ***S2*** *en el que la mitad de sus patrones hayan sido clasificados correctamente por* ***C1.***
3. Se entrena el clasificador ***C3*** con los patrones en los que la predicción de ***C1*** y ***C2*** difieren.

AdaBoost - ADAptive BOOSTing

La idea clave: *prestar más atención a los patrones difíciles.*

- *Cada patrón de entrenamiento tiene asociado un peso* que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es mal clasificado.
- En test se usa un voto pesado de acuerdo al desempeño previo de cada clasificador del ensamble.

Algoritmo de AdaBoost:

Entrenamiento:

Inicialización: $D_1(i) = 1/n, \quad \forall i.$

para $b = 1, 2, \ldots, B$

1 Muestrear por bootstrap el subconjunto S_b^* de acuerdo a la distribución D_b.

2 Entrenar un clasificador débil con S_b^*, y obtener sus predicciones $h_b : X \to \Omega$

3 Calcular el error de las predicciones h_b:

$$\epsilon_b = \sum_i I(h_b(\mathbf{x}_i), y_i) D_b(i).$$

4 **si** $\epsilon_b \leq \frac{1}{2}$ ← Tiene que ser **menor al "by chance"!**

4.1 Calcular el error normalizado $\beta_b = \frac{\epsilon_b}{1-\epsilon_b}$

4.2 Actualizar la distribución

$$D_{b+1}(i) = \frac{D_b(i)}{Z_b} \times \begin{cases} \beta_b, & \textbf{if } h_b(\mathbf{x}_i) = y_i \\ 1, & \text{otherwise} \end{cases}$$

(Z_b: constante de normalización para las probabilidades).

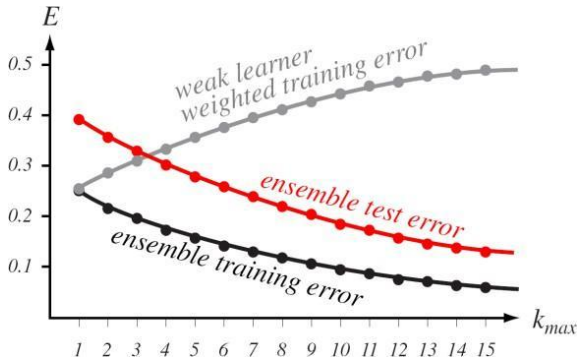
Prueba: Voto por mayoría pesado.

Dado un patrón no etiquetado $\mathbf{z}$

1 Obtener el total de votos recibido por cada clase

$$V_j = \sum_{b:h_b(\mathbf{z})=y_j} \log\left(1/\beta_b\right), \quad \forall j.$$

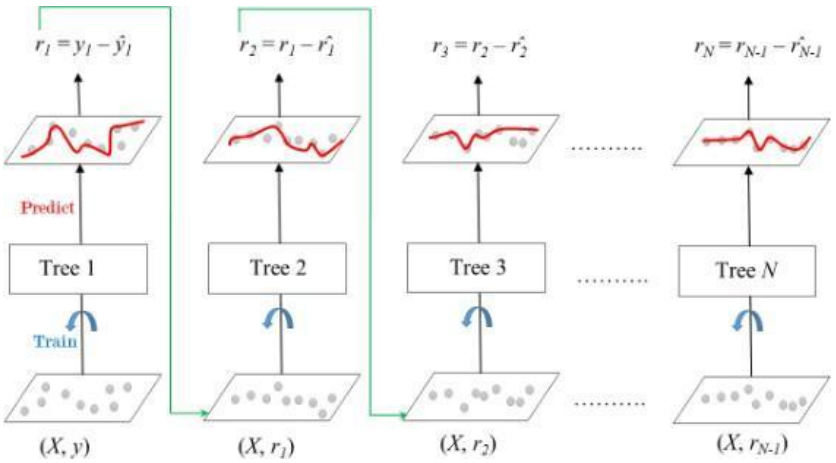
2 Elegir la clase que recibió más votos.



→ aumento cantidad de miembros en el ensamble.

Gradient boosting

- Se usa mucho en **problemas de ML en datos tabulares**
- Es la evolución de los ensambles de los últimos años



En el primer ensamble se hace un pred y a partir de ella se calc un error, que es lo que entra al segundo

→ al segundo ya NO ENTRAN LOS DATOS, lo que estoy tratando de predecir es él es el residuo/error.

Ensambles de clasificadores

Ventajas:

- ✖ *Poco propensos al sobre-entrenamiento* (buena capacidad de generalización).
- ✖ Se adaptan al desbalance de clases.
- ✖ Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
    - Cada tipo de clasificador aprende distintas particularidades del problema.
- ✖ Hacen un mejor aprovechamiento de los datos cuando son escasos.
- ✖ Tratan naturalmente los problemas multiclase.

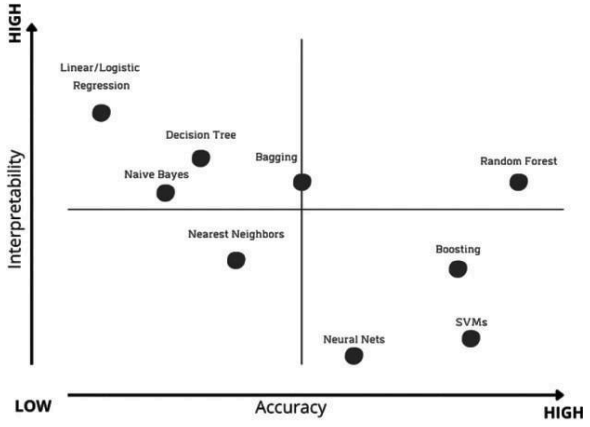
✖ Estables y robustos.

Desventajas:

✖ El entrenamiento puede ser costoso.
✖ No son tan fáciles de interpretar.
✖ Los resultados pueden variar mucho dependiendo de la elección de los modelos débiles, su diversidad y la de los datos.

Interpretabilidad vs exactitud

→ En muchas aplicaciones prácticas, el usuario final del modelo necesita saber el porqué de las salidas.



SOM Mapas auto-organizativos

Auto-organización

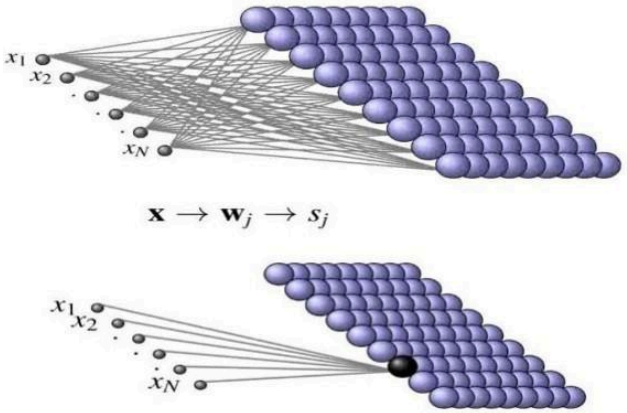
Es el proceso en el cual, *por medio de interacciones locales, se obtiene ordenamiento global.*

- Existen estructuras que se auto-organizan.
- Proceso por el cual los elementos que componen la estructura tienen un comportamiento sencillo y reglas elementales pero emerge un comportamiento global complejo.

Ejemplo: las células del cerebro se auto-organizan en grupos de acuerdo a la información que reciben…

SOM: Arquitectura

- Hay *una sola capa*, y esta organizada en un *plano*.
- Por cada entrada va a haber un peso que conecta a esa entrada con todas las neuronas de la capa, conexión todos contra todos.
    - Tenemos un vector de entradas, una matriz de pesos, y una capa de neuronas.
- Es la más básica y hay muchas variantes.

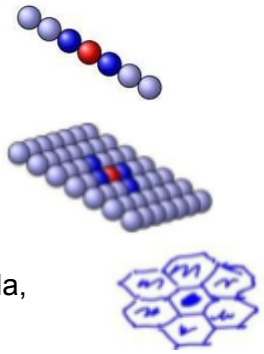


→ Cuando entra un estímulo HAY UNA DE LAS NEURONAS QUE SE ACTIVA, es una **red neuronal de COMPETENCIA**, es decir, se hace una comparación entre la entrada y todos los vectores de pesos, y aquel que tenga menor distancia, o sea el que se parezca más a la entrada, define cuál es la neurona ganadora:

$$j^*(n) = G(\mathbf{x}(n)) = \arg\min_j \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Entornos de influencia o vecindades

➢ Formas básicas de organización:



- **Línea** de neuronas, que cuando una se activa puede afectar a su entorno.
- **Cuadrícula o plano**, que también cuando se activa afecta al resto.
- **Hexagonal** la división del plano que es bastante utilizada, la neurona ganadora tiene 6 neuronas en su vecindad.

➢ Alcance $\mathbf{A_g}$:
A lo largo del entrenamiento estos entornos pueden ser fijos o variables, de radio N.

➢ Excitación asociada al entorno:
→ Las neuronas del entorno pueden estar relacionadas de diferentes formas con la ganadora.
- Excitatorias puras
- Inhibitorias puras
- *Funciones generales* de excitación/inhibición

Funciones de excitación/inhibición lateral

- Uniforme (entorno simple):

$$\Lambda_G(n) = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & \text{si } |G - i| \leq 2 \\ h_{G,i} = 0 & \text{si } |G - i| > 2 \end{cases}$$

→ para un entorno de radio 2, si la distancia entre la neurona ganadora G y una neurona cualquiera del mapa i, es menor que 2, entonces se va a excitar o inhibir con la constante B.
Si la distancia es mayor, no le hace nada

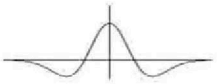
siendo $\beta(n)$ adaptable en función de las iteraciones n.

- Gaussiana:

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

→ si excita o inhibe va a depender del valor de la gaussiana, cuanto más lejos esté, menos la va a influenciar

- Sombrero Mejicano: campos receptivos retina, inhibición lateral



Entrenamiento de un SOM

- Entrenamiento totalmente ***NO-supervisado***.
- **Aprendizaje competitivo**, *en cada iteración no se ajustan los pesos de toda la red neuronal, solo en principio la de la ganadora y algún entorno de ella*.

1. Inicialización:
   - pequeños valores aleatorios con $w_{ji} \in [-0{,}5, \cdots, +0{,}5]$ o también
   - eligiendo aleatoriamente $\mathbf{w}_j(0) = \mathbf{x}_\ell$ $(\ell \in [1, \cdots, L])$
2. Selección del ganador:

$$G(\mathbf{x}(n)) = \arg\min_{\forall j} \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

3. Adaptación de los pesos:

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n)\,(\mathbf{x}(n) - \mathbf{w}_j(n)) & \text{si } y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & \text{si } y_j \notin \Lambda_G(n) \end{cases}$$

→ acercar un poco el valor de los pesos a los valores que trajo el patrón de entrada, según la velocidad de aprendizaje*(diferencia entre entrada y pesos, vector de error), si la neurona y se encuentra en el entorno

4. Volver a 2 hasta no observar cambios significativos.

- <u>Consideraciones prácticas:</u>
  - Generalmente usamos mapas cuadrados.
  - Se suele usar la función $h_{G,i}(n)$ uniforme i, pero con un entorno variable decreciente con **n**.
  - La velocidad de aprendizaje $0 < \eta(n) < 1$ también es variable, decreciente con **n**.

¿Cómo varían $\Lambda_G(n)$ y $\eta(n)$?

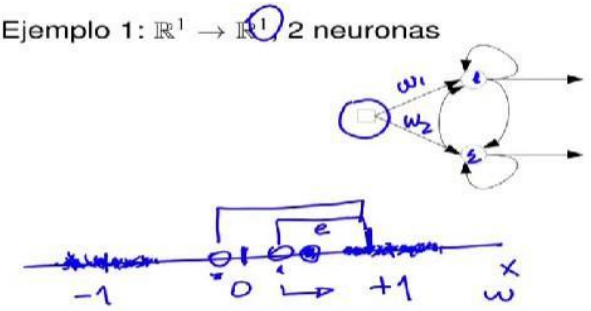
→ Se definen **3 etapas básicas en el entrenamiento**:

1. Ordenamiento global (o topológico)
   - $\Lambda_G(n)$ grande ($\approx$ medio mapa)
   - $\eta(n)$ grande (entre 0.9 y 0.7)
   - Duración 500 a 1000 épocas

2. Transición
   - $\Lambda_G(n)$ se reduce linealmente hasta 1
   - $\eta(n)$ se reduce lineal o exponencialmente hasta 0.1
   - Duración ≈1000 épocas
3. Ajuste fino (o convergencia)
   - $\Lambda_G(n) = 0$ (sólo se actualiza la ganadora)
   - $\eta(n) = cte$ (entre 0.1 y 0.01)
   - Duración: hasta convergencia (≈ 3000 épocas)

Formación de mapas topológicos

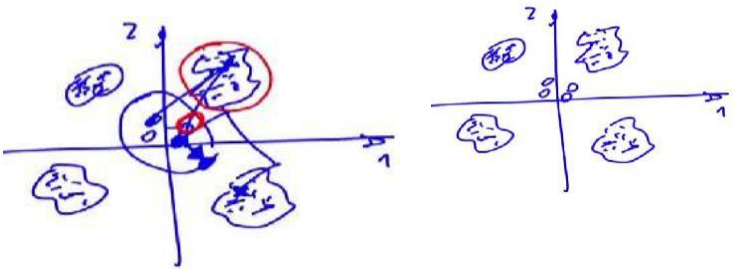
Ejemplo R1 → R1, **sin entorno/relación/vecindad** entre las neuronas.



→ la neurona 1 se va a ir corriendo hacia el +1 y la dos hacia el -1.

=> Aprende la estructura de los datos.

Ejemplo R2 → R1, **sin entorno/relación/vecindad** entre las neuronas. Distribución de los patrones: >
y pesos iniciales

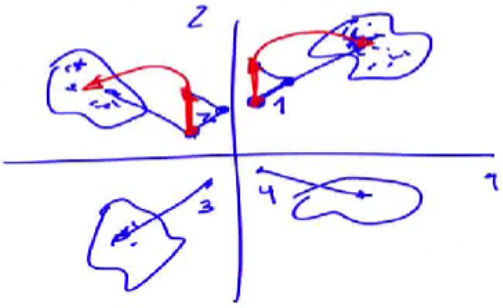


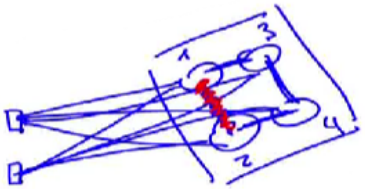
En su mayoría, cada iteración va a hacer que cada neurona se acerque a 1 grupo de patrones en particular, es por eso también que se hacen las 3 etapas básicas de entrenamiento.

=> Aprende la estructura de los datos.

→ **Ahora, sí sí consideramos una vecindad/entorno:**

- ➢ 2 o más neuronas se van a actualizar de forma conjunta, por ejemplo, van a subir juntas.
- ➢ Pero una vez que se inhiba el efecto del entorno, sí van a poder moverse más refinadamente por separado.



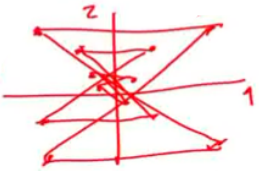


=> Las conexiones topológicas entre neuronas tienen una importancia, dado que **van a hacer que neuronas que están cercanas, representen regiones cercanas del espacio de solución.**

→ y es así cómo se genera el ordenamiento topológico. ← ← ← ← ← ← ← ← ←

Gráficamente: (2D)

→ **Podríamos visualizar el ordenamiento topológico** a lo largo del tiempo si graficamos como puntos a los pesos y trazamos líneas entre las neuronas que están conectadas entre sí.



=> Acá vemos el interés práctico de los SOM, que yo pueda proyectar un espacio de R^N, que no puedo visualizar, en un mapa bidimensional de neuronas en el que las neuronas que están cerca agrupan patrones que también están cerca en N dimensiones. ←←←

- Ideas de cómo entrenarlo:
  - algoritmo k-means etiquetado para clasificación
  - SOM etiquetado como clasificador...
  - algoritmo supervizado LVQ

→ Podríamos decir que los prototipos son los centroides que encontramos con k-medias.
→ Un SOM
→ o LVQ → Utiliza la información de clases durante el entrenamiento.

Algoritmo de aprendizaje LVQ1

1. Inicialización aleatoria
2. Selección:

$$c(n) = \arg\min_i \{\|\mathbf{x}(n) - \mathbf{m}_i(n)\|\}$$

3. Adaptación:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c,d,n)\alpha\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$s(c,d,n) = \begin{cases} +1 & \text{si } c(n) = d(n) \\ -1 & \text{si } c(n) \neq d(n) \end{cases}$$

4. Volver a 2 hasta satisfacer error de clasificación

El factor ***s(c,d,n)*** → nos dice si la neurona o prototipo ganador acertó a la clase o no.

→ Si acertó, acercamos el prototipo/neurona/vector de pesos al patrón que acaba de entrar.
→ Si no acertó, lo alejamos.

→ es casi igual que el SOM, pero:
- **No hay** un concepto de **vecindad**.
- Usamos el factor **s** => **es SUPERVISADO**.
- Cada prototipo se va a ir acercando al centro de la clase que representa más rápidamente, porque no solo acercamos, sino que cuando erra lo alejamos.

Observaciones:
★ No hay arquitectura neuronal
★ Se puede ver al cuantizador como **SOM lineal, sin entorno y supervisado**.
★ La velocidad de aprendizaje no puede ser exactamente la misma durante todo el entrenamiento →

LVQ1: velocidad de aprendizaje

Ecuación original:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

Si aplicamos distributiva del factor s(n) y la vel. de aprendizaje:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n) - s(n)\alpha(n)\mathbf{m}_c(n)$$

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)]\,\mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n)$$

Si expresamos al $\boldsymbol{m_c(n)}$ en función de cómo se calculó el anterior:

$$\mathbf{m}_c(n+1) =$$

$$= [1 - s(n)\alpha(n)] \{\mathbf{m}_c(n-1) + s(n-1)\alpha(n-1)\,[\mathbf{x}(n-1) - \mathbf{m}_c(n-1)]\} + s(n)\alpha(n)\mathbf{x}(n)$$

$\mathbf{x}(n-1)$ es afectado dos veces por α

Siendo $\alpha < 1$ la importancia relativa de los primeros patrones de entrenamiento siempre será menor que la de los últimos.

Si queremos que α afecte por igual a todos los patrones deberemos hacerlo decrecer con el tiempo.

=> hay que cambiarle el alfa de manera que todos los patrones pesen por igual en la posición final que tome el centroide.

=> se debe cumplir que el alfa en 2 iteraciones consecutivas pese lo mismo:

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)]\,\alpha_c(n-1)$$

$\alpha_c(n) = 1\,\alpha_c(n-1) - s(n)\,\alpha_c(n)\,\alpha_c(n-1)$

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$$

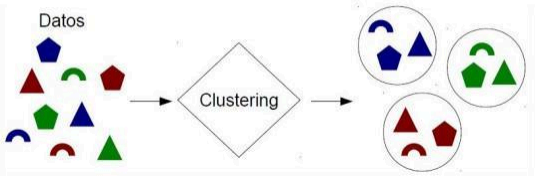
Aprendizaje no supervisado = Clustering (clase adicional)

Clustering

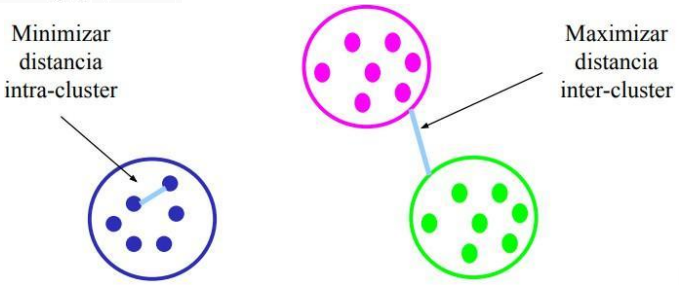
Agrupar objetos similares basados en alguna noción de similaridad. El resultado es una partición de los datos.

Cluster

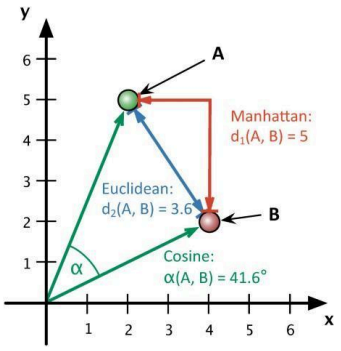
Un conjunto de objetos que son similares entre ellos, mientras que difieren de los objetos en otros clusters.



Objetivos del clustering:

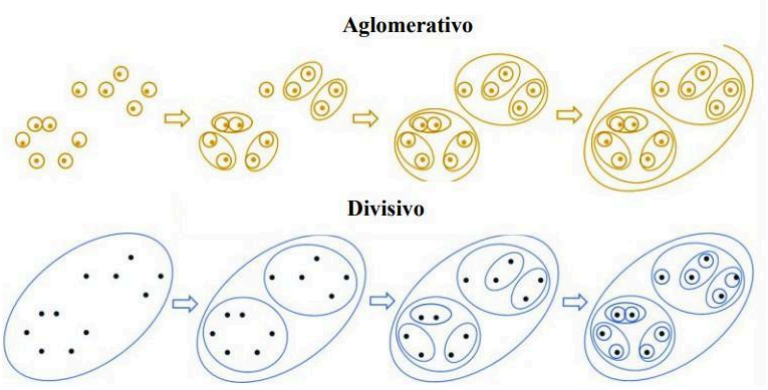


La distancia en el clustering



Clustering - Métodos

Agrupamiento jerárquico:

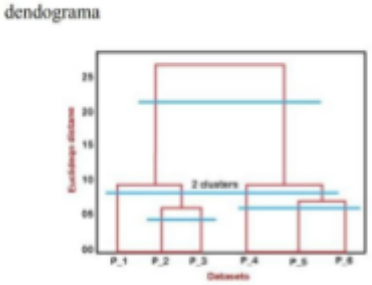


Para aplicarlo necesito una matriz de distancia:

- datos: P1, P2, P3, P4, P5 y P6
- calcular la matriz de distancia entre todos los datos

Divisivo:

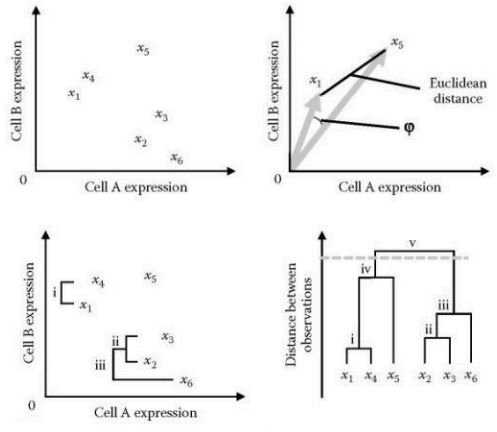
→ Con esto hacemos un:

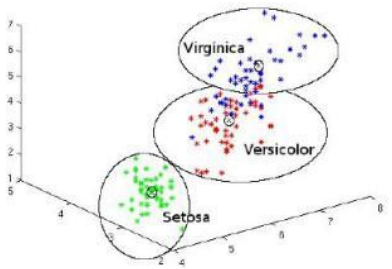


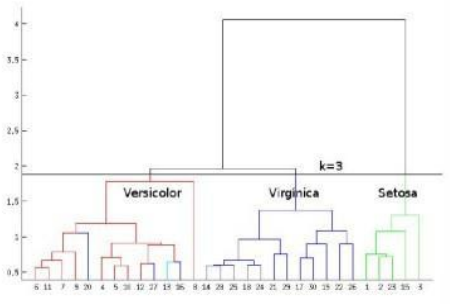
	P1	P2	P3	P4	P5	P6
P1	0.00					
P2	0.24	0.00				
P3	0.22	0.15	0.00			
P4	0.37	0.20	0.15	0.00		
P5	0.34	0.14	0.28	0.29	0.00	
P6	0.23	0.25	0.11	0.22	0.39	0.00

→ Unimos con una rama puntos que están cercanos con ramas, y trazamos luego líneas cuya altura determina la cantidad de clusters.

Aglomerativo:







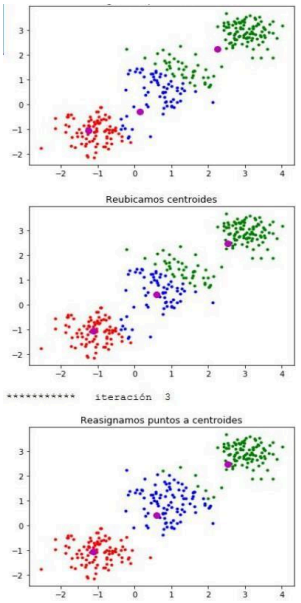
K medias:

- Algoritmo muy popular y muy simple.
- Particiona los datos en k clusters.
- Utiliza centroides (prototipos de los clusters).
- El problema es elegir el k.
- K-medias es un algoritmo que agrupa objetos en k grupos basándose en sus características.

- Minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster.
- Se suele usar la distancia euclídea.

El algoritmo consta de tres pasos:

1) **Inicialización:** una vez elegido el número de grupos k, se establecen k centroides en el espacio de los datos aleatoriamente.
2) **Asignación objetos a los centroides:** cada objeto de los datos es asignado a su centroide más cercano.
3) **Actualización de centroides**: se actualiza la posición del centroide de cada grupo tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo. Se repiten los pasos 2 y 3 hasta que los centroides no se mueven.



- Las principales ventajas del método k-medias son que es un método sencillo y rápido.
- Pero… es necesario decidir el valor de k a-priori
- El resultado final depende de la inicialización de los centroides.
- En principio no converge al mínimo global sino a un mínimo local.

Métricas de clustering

Cada algoritmo de clustering y combinación de parámetros produce particiones diferentes en los datos…

- Por lo tanto, ¿cómo evaluar los resultados obtenidos con varios algoritmos y varias configuraciones?
- ¿Cómo saber si la estructura encontrada por un algoritmo de clustering en los datos es válida?

Medidas internas:

Reciben como entradas los datos y las soluciones de clustering obtenidas, se evalúan según la información intrínseca de los datos

- **Cohesión**
  - Evalúa qué tan compactos u homogéneos son los clusters obtenidos por una solución. Se mide con la varianza intra-cluster:

$$\overline{C}_j = \frac{1}{|\Omega_j|} \sum_{\forall \mathbf{x}_i \in \Omega_j} \|\mathbf{x}_i - \mathbf{w}_j\|_2,$$

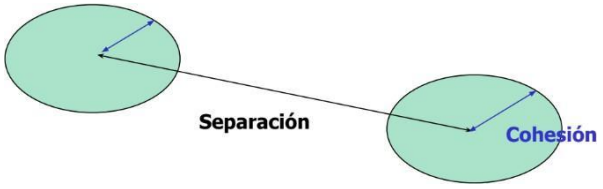
donde $|\Omega_j|$ es el número de datos en el cluster j, $\mathbf{w}_j$ es el centroide del cluster j, $\mathbf{x}_i$ es un dato agrupado en el cluster j.

  - Medida global de cohesión => promedio sobre todos los clusters-
  - Cohesión cercana a 0 => clusters más compactos (mejor)

- **Separación**
  - Se cuantifica el grado de separación entre k clusters individuales. Esta separación puede ser evaluada por la distancia euclídea media, mínima y máxima entre los centroides $\mathbf{w}_i$ y $\mathbf{w}_j$ de los clusters:

$$\begin{aligned} \overline{S} &= \frac{2}{k^2-k}\sum_{i=1}^{k}\sum_{j=i+1}^{k}\|\mathbf{w}_i-\mathbf{w}_j\|_2, \\ S_m &= \min_{0<i\neq j\leq k}\{\|\mathbf{w}_i-\mathbf{w}_j\|_2\}, \\ S_M &= \max_{0<i\neq j\leq k}\{\|\mathbf{w}_i-\mathbf{w}_j\|_2\}, \end{aligned}$$

  - Separación cercana a 0 => indica cercanía entre nodos.
  - Separación cercana a 1=> clusters más separados (mejor)



- **Davies Bouldin**
  - Medida combinada, función de la relación entre la suma de las dispersiones
  - intra-nodo y la separación entre nodos. <u>Índice de Davies-Bouldin:</u>

$$DB = \frac{1}{k}\sum_{i=1}^{k}\max_{j\neq i}\left(\frac{\overline{C}_i+\overline{C}_j}{\|\mathbf{w}_i-\mathbf{w}_j\|_2}\right).$$

cohesión media

separación media

  - Medida del solapamiento entre nodos → privilegia agrupamientos compactos y bien separados entre sí.
  - Valores de DB cercanos a 0 indican poco solapamiento

Medidas externas:

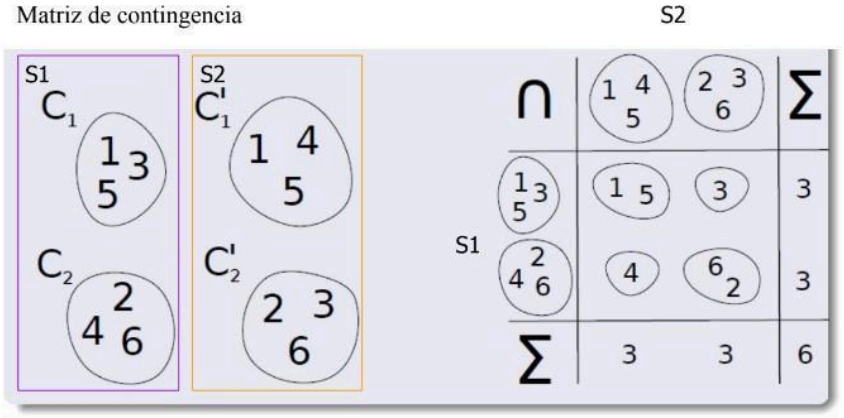
<u>Comparan la similitud existente entre un par de soluciones de clustering (**S1** y **S2**), o contra una referencia (“gold standard”)</u>

- **FM - Fowlkes and Mallows**
  - Se utiliza para determinar la similitud entre dos particiones de clusterings (S1 y S2).

$$\mathrm{FMI} = \frac{\mathrm{TP}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})}}$$

    - **TP**: nro. de pares de puntos que están presentes en el mismo cluster en S1 y en S2
    - **FP**: nro. de pares de puntos que están presentes en el mismo cluster en S1 pero NO en S2
    - **FN**: nro. de pares de puntos que están presentes en el mismo cluster en S2 pero NO en S1

- Puede ser entre un agrupamiento y una clasificación de referencia.
- Un valor más alto (cercano a 1) para el índice de FM indica una mayor similitud entre las soluciones.



- **Rand Index (RI)**
  - Para 2 soluciones S1 y S2, medimos:
    a) nro. de pares de elementos que están en el mismo conjunto en S1 y S2
    b) nro. de pares de elementos que están en conjuntos diferentes en S1 y en conjuntos diferentes en S2
  - El índice de Rand es: $$\mathrm{RI} = \frac{a+b}{C_2^{n_{samples}}}$$

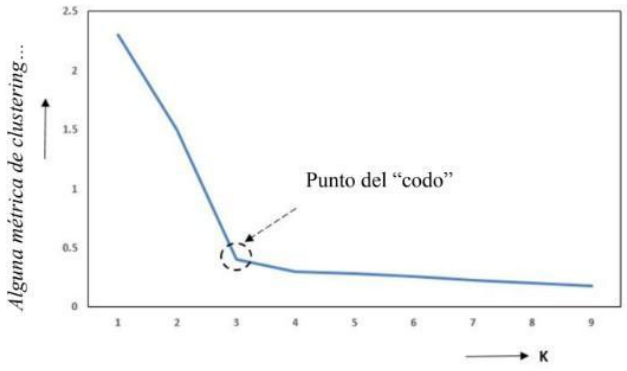
    donde el denominador es el número total de pares posibles en el conjunto de datos.
  - El etiquetado perfecto tiene una puntuación de 1.

- **Jaccard, NMI, …**

El número de clusters (k) - K óptimo

Las métricas de clustering pueden ser usadas para *estimar el número adecuado de clusters para un dataset.*

- Eso involucra **calcular los resultados de una métrica de clustering para un rango de valores de k, y plotear la curva resultante**.
- El mejor nro. de clusters puede determinarse como un **“pico” o “codo” en la curva** de performance obtenida.



Programación diferenciable - ¿Qué pasa si conectamos muchas neuronas?

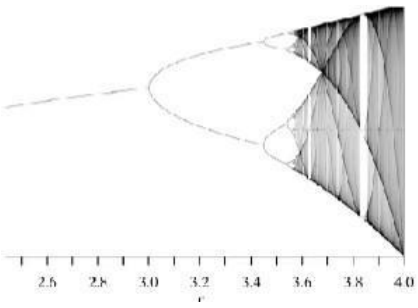
- Necesidad de contar con grandes cantidades de datos para entrenar
  → Revolución de los grandes datos
- Alto costo computacional
  → Unidades de procesamiento gráfico GPUs
- Desvanecimiento del gradiente
  → Pre-entrenamiento no-supervisado,
  → Auto-codificadores con máquinas de Boltzmann,
  → Normalización por lotes,
  → Redes residuales,
  → Otros condimentos: ReLU, dropout, pooling..

- Dificultad para construir y entrenar modelos muy grandes y con diferentes tipos de capas neuronales interconectadas
  → **Programación diferenciable ∂P**

Programación diferenciable ∂P

Es una mezcla entre la diferenciación numérica y la simbólica.

→ Supongamos que tenemos que calcular las derivadas del mapa logístico: $\ell_{n+1} = r\ell_n(1-\ell_n)$, con respecto a x, a lo largo de las iteraciones n.



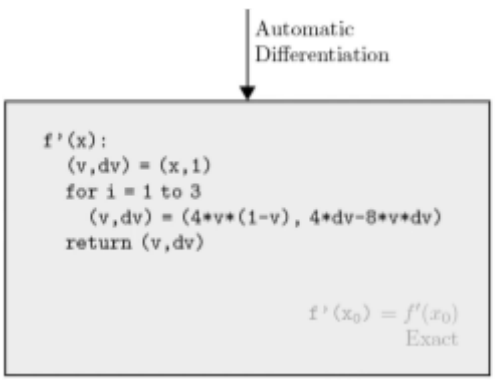
Con $r = 4$, por ejemplo, tenemos

$\ell_{n+1} = 4\ell_n(1-\ell_n), \ell_1 = x$

1	x	1	1
2	$4x(1-x)$	$4(1-x)-4x$	$4-8x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2-16x(1-2x)^2-64x(1-x)(1-2x)$	$16(1-10x+24x^2-16x^3)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2)+64(1-x)(1-2x)^2(1-8x+8x^2)^2-64x(1-2x)^2(1-8x+8x^2)^2-256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1-42x+504x^2-2640x^3+7040x^4-9984x^5+7168x^6-2048x^7)$

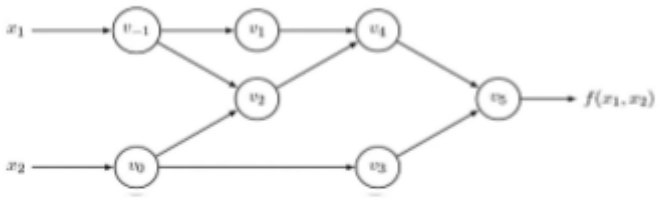
Derivación automática:

- La idea es ir acumulando en un bucle las derivadas de la función, guardo un par de variables, el valor de la función, y el valor de la derivada de la función.
- De esta manera calculamos exactamente la derivada con una mezcla de cómputo simbólico con acumulación de información numérica.



–> MODO HACIA ADELANTE

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

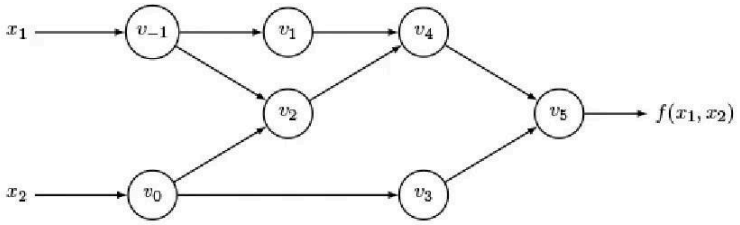


$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

v_{-1}	$= x_1$	$= 2$	$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
v_0	$= x_2$	$= 5$	$\dot{v}_0$	$= \dot{x}_2$	$= 0$
v_1	$= \ln v_{-1}$	$= \ln 2$	$\dot{v}_1$	$= \dot{v}_{-1}/v_{-1}$	$= 1/2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$	$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
v_3	$= \sin v_0$	$= \sin 5$	$\dot{v}_3$	$= \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$	$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$	$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
y	$= v_5$	$= 11.652$	$\dot{y}$	$= \dot{v}_5$	$= 5.5$

–> MODO REVERSO

Hago el de todas las entradas en una sola pasada, con solo saber las derivadas locales.



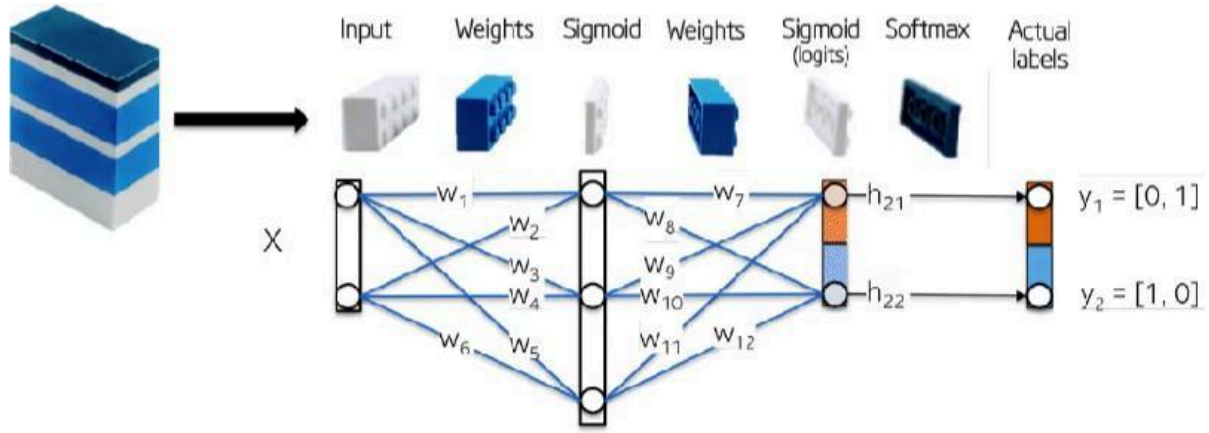
$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

v_{-1}	$= x_1$	$= 2$	$\bar{x}_1$	$= \bar{v}_{-1}$		$= 5.5$
v_0	$= x_2$	$= 5$	$\bar{x}_2$	$= \bar{v}_0$		$= 1.716$
v_1	$= \ln v_{-1}$	$= \ln 2$	$\bar{v}_{-1}$	$= \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}}$	$= \bar{v}_{-1} + \bar{v}_1 / v_{-1}$	$= 5.5$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$	$\bar{v}_0$	$= \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0}$	$= \bar{v}_0 + \bar{v}_2 \times v_{-1}$	$= 1.716$
			$\bar{v}_{-1}$	$= \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}}$	$= \bar{v}_2 \times v_0$	$= 5$
v_3	$= \sin v_0$	$= \sin 5$	$\bar{v}_0$	$= \bar{v}_3 \frac{\partial v_3}{\partial v_0}$	$= \bar{v}_3 \times \cos v_0$	$= -0.284$
v_4	$= v_1 + v_2$	$= 0.693 + 10$	$\bar{v}_2$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_2}$	$= \bar{v}_4 \times 1$	$= 1$
			$\bar{v}_1$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_1}$	$= \bar{v}_4 \times 1$	$= 1$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$	$\bar{v}_3$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_3}$	$= \bar{v}_5 \times (-1)$	$= -1$
			$\bar{v}_4$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_4}$	$= \bar{v}_5 \times 1$	$= 1$
y	$= v_5$	$= 11.652$	$\bar{v}_5$	$= \bar{y}$	$= 1$	

→ Nos permite no tener que sacar la derivada de la ecuación completa, solo conociendo las funciones y sus dd en cada “neurona”

→ Soluciona el desvanecimiento del gradiente (son exactas).

Programación diferenciable - el Lego de las arq. neuronales profundas



→ Permite poder manipular elementos de nuestro modelo sin tener que preocuparnos por volver a hacer cálculos engorrosos.

→ Permitió experimentar mucho más con las redes neuronales.

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
  - Perceptrones simples y de una capa
  - Perceptrón multicapa
  - Redes con funciones de base radial
  - ...
- Redes recurrentes (recurrent/with feedback)
  - Redes de Kohonen
  - Redes de Hopfield
  - Redes de Boltzmann
  - Redes de Elman/Jordan
  - Modelos de resonancia adaptativa
  - ...
- Otros modelos híbridos
  - Parcialmente recurrentes
  - Parcialmente conectados
  - Redes modulares
  - ...

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
  - Deep Belief Network (DBN)
  - Convolutional Neural Network (CNN)
  - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
  - ...
- Redes recurrentes (recurrent/with feedback)
  - Long Short-Term Memory (LSTM)
  - Attention Mechanisms
  - Transformers
  - ...
- Otros modelos y métodos de entrenamiento
  - Generative Adversarial Networks (GAN)
  - Deep Reinforcement Learning (DRL)
  - Bayesian Deep Learning (BDL)
  - ...

Bases estadísticas del reconocimiento de patrones

Aprendizaje Maquinal

Problema: ¿Cómo enseñar a una máquina a que emule las tareas humanas de detección, percepción, identificación, reconocimiento, etc.?

Objetivo: Programar máquinas para:
- ➢ aprender a realizar una *tarea*,
- ➢ mejorando su *desempeño*
- ➢ basado en la *experiencia.*

→ **Restricciones adicionales**:
- mínima intervención humana posible,
- con reducido conjunto de ejemplos,
- etc…

Conceptos básicos de Aprendizaje Maquinal

Podemos considerar dos ramas de la Inteligencia Artificial (IA):

- **IA clásica:**
  - Modelado del proceso de *razonamiento* humano mediante técnicas de la Lógica.
  - Aprendizaje deductivo.

- **Reconocimiento de Formas:**
  - Modelado del proceso de *percepción* humano mediante técnicas de la *Teoría de la Decisión Estadística* y de la *Teoría de los Lenguajes Formales.*
  - Aprendizaje inductivo. → A través de ejemplos

Vista como una disciplina de la IA… Aspectos de Aprendizaje Maquinal:

- ***Adquisición y representación del conocimiento***: conformación y almacenamiento de conjuntos de patrones o prototipos.
- ***Aprendizaje***: algoritmos de aprendizaje inductivo a partir de un conjunto de entrenamiento.
- ***Clasificación*****:** etiquetado de patrones nuevos utilizando el conjunto de clases disponible.
- ***Evaluación***: mecanismos para evaluar la bondad, confianza o error del sistema.

Definiciones

★ **Patrón:**
- ➢ *Objeto de interés que es identificable del resto.*
- ➢ Posiblemente difusos, no bien definidos, no visibles o tangibles.
- ➢ Ej: una huella digital, la voz de una persona, una cara, etc.

★ **Reconocimiento de patrones (RP):**
- ➢ Estudio de los procesos de percepción y razonamiento humanos:
  - Con estas 3 subtareas:

- *capacidad de distinguir y aislar los patrones,*
- *reunirlos en grupos,*
- *asignarles un nombre identificatorio a cada grupo.*

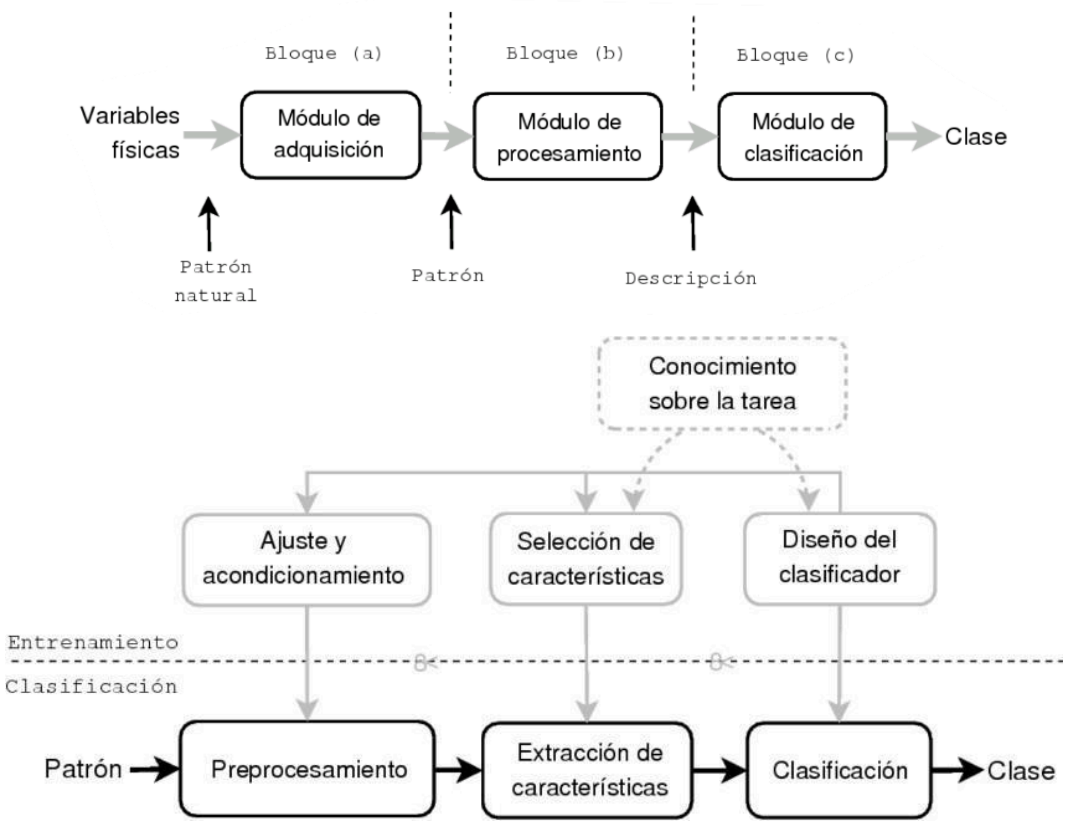
➢ Objetivo del RP: crear sistemas informáticos que imiten el comportamiento descrito.

Paradigma de trabajo conceptual - Diagrama en bloques

(a) **Adquisición**: transducción del mundo real a la representación digital.

(b) **Procesamiento digital**: acondicionamiento y representación alternativa, permite caracterizar los objetos. Como salida tiene la descripción, que es un vector, tiene que ser pequeño y lo más representativo posible.

(c) **Clasificación**: decisión sobre la clase.



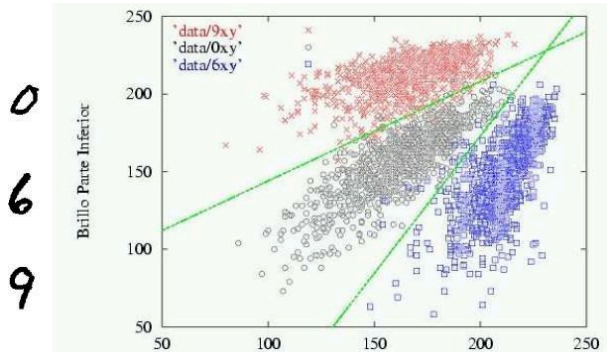
Aproximaciones

- Aproximación geométrica o estadística:
  - Basada en la Teoría Estadística de la Decisión.
  - Representación de *patrones* como ***vectores numéricos.***
  - Representación de *clases* mediante ***patrones prototipo***.
  - Tipos de clasificadores: gaussianos, basados en distancia, etc.

- Aproximación estructural o sintáctica:
  - Basada en la Teoría de Lenguajes Formales.
  - Representación de *patrones* como ***cadenas de símbolos***.

- Utilización de *reglas sintácticas* para especificar los ***patrones válidos de una clase.***
- Tipos de clasificadores: autómatas, gramáticas, HMM, etc

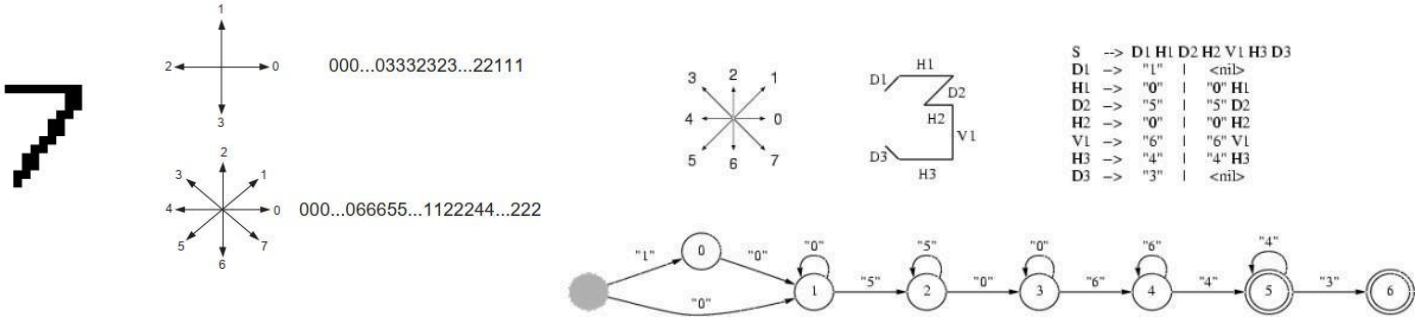
Ejemplo de aproximación geométrica:



Ejemplo de aproximación sintáctica:

Descripción de dígitos mediante cadenas de contorno:

Clasificador sintáctico basado en autómatas:



Conceptos de <u>clasificación estadística</u>

- ***Adquisición y preproceso de datos***: vector numérico que representa al patrón natural. Formalmente, un patrón es una variable aleatoria *n*-dimensional $y = [y1\ y2\ \ldots\ yn]^T$, con
  $y_i \in R$ para $i = 1, 2, \ldots, n$, que representa un punto en el espacio de patrones $P \in R^n$.

- ***Extracción de características:*** información *relevante* para la clasificación.

  Formalmente, dado un conjunto de patrones *n*-dimensionales y se trata de obtener un <u>nuevo conjunto de representaciones</u>

  $$x = [x_1\ x_2\ \ldots\ x_d]^T \text{ , donde } \boldsymbol{d \leq n.}$$

  - ➢ Cambio en el espacio de representación: transformaciones lineales que maximizan la varianza *(d = n)*.
  - ➢ Reducción de dimensionalidad de los datos *(d < n)*.

- ***Espacio de características:*** $E \in R^d$.

- ***Extracción de características:*** *propiedades deseables*
  - ➢ <u>Precisión</u>: representaciones diferentes para objetos diferentes.
  - ➢ <u>Unicidad o determinismo</u>: representación única para cada objeto.

- ➢ Continuidad en el espacio: da robustez al sistema, significa que en un punto en ese espacio que tiene una etiqueta, con cualquier modificación de ruido, el sistema siga identificando esa etiqueta. => Inmunidad al ruido y capacidad de generalización.

(las primeras 2 se pueden relajar)

- ***Clases informacionales:*** *salidas del sistema*
  - ➢ Número de clases: *C*
  - ➢ Conjunto de (etiquetas de) clases: Ω = {ω1, ω2, . . . , ωc}
  - ➢ Conjunto extendido: Ω∗ = {ω1, ω2, . . . , ωc, ω0}, donde ω0 es la *clase de rechazo* → Casos donde el sistema no tiene seguridad sobre la desición que tomó (indica revisar la muestra).

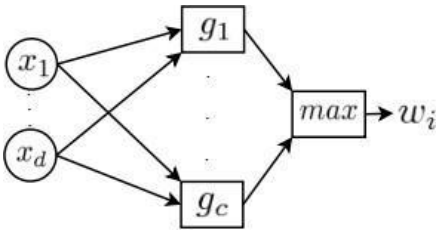
Definiciones de clasificación estadística

Clasificador estadístico: máquina formada por c (funciones) discriminantes

$g_i : E \rightarrow R,\ 1 \leq i \leq c$ → tenemos tantas funciones como clases.

tal que dado un patrón $\boldsymbol{x} \in E$,

x se asigna a la clase ω_i si $g_i(x) > g_j(x)\ \forall j \neq i$



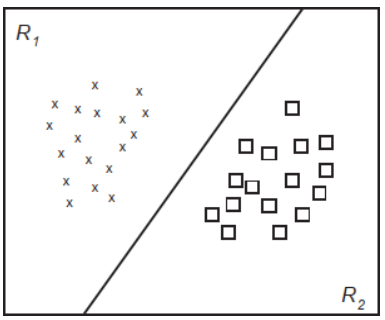
Regiones y fronteras de decisión

- Regiones de decisión: un clasificador divide el espacio en ***c*** regiones de decisión **R1, R2, . . . , Rc**, tal que

$$R_i = \{x \in E : g_i(x) > g_j(x)\ \forall j \neq i\}$$

- Fronteras de decisión: superficies del espacio que separan regiones de decisión contiguas.
  - Son los puntos del espacio donde hay igualdad en la salida de las funciones discriminantes → Causantes de rechazo.

Hipersuperficies definidas por: $g_i(x) - g_j(x) = 0,\ i \neq j,\ 1 \leq i, j \leq c$



Clasificadores estadísticos básicos

Las FD son combinaciones lineales o cuadráticas de las componentes del vector de características.

- **Clasificador lineal:**

$$g(\mathbf{x}) = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^t \mathbf{x} + w_0$$

→ Perceptrón multicapa

Fronteras: hiperplanos.

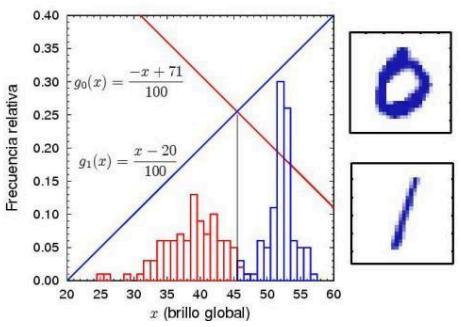
- **Clasificador cuadrático:**

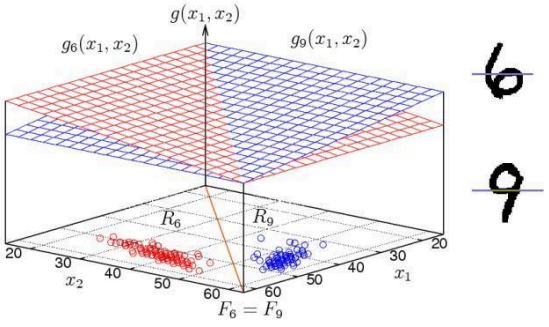
$$g(\mathbf{x}) = \sum_{i=1}^{d} \sum_{j=1}^{d} w_{ij} x_i x_j + \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{x}^t W \mathbf{x} + \mathbf{w}^t \mathbf{x} + w_0$$

Fronteras: hipercuádricas.

Ejemplo: OCR de dígitos manuscritos

- Unidimensional: brillo global.
- Bidimensional: brillo de mitad superior (x_1) e inferior (x_2).





Definiciones de teoría de la decisión

- **Probabilidad a priori P(ωi)** de una clase ωi: probabilidad de que una muestra arbitraria pertenezca a ωi.

"Probabilidad de observar la etiqueta c sin saber qué muestra es"

Puede verse como la **proporción** de muestras de ωi respecto al total (conocimiento que se tiene antes de hacer los experimentos).

Condiciones:
- $0 \leq P(\omega_i) \leq 1$, para $i = 1, \ldots, c$.
- $\sum_{i=1}^{c} P(\omega_i) = 1$.

Clasificador trivial de dos clases basado en probabilidades a priori:
- Decidir por ω_1 si $P(\omega_1) > P(\omega_2)$
- Decidir por ω_2 si $P(\omega_1) < P(\omega_2)$

- **Densidad condicional P(x|ωi)** de una clase ωi: función de densidad de probabilidad que caracteriza la distribución estadística de las muestras de ωi.

"Probabilidad de observar la muestra x sabiendo que la etiqueta es c"

El vector de características x se considera una variable aleatoria d-dimensional de función de densidad **P(x|ωᵢ)** cuando las muestras pertenecen a **ωᵢ**.

Condiciones:

- $P(\mathbf{x}|\omega_i) \geq 0$, para $i = 1, \ldots, c$.
- $\int_E P(\mathbf{x}|\omega_i)\ d\mathbf{x} = 1$.

- **Probabilidad conjunta P (x, ωi)** muestra-clase se define como:

$$P(\mathbf{x}, \omega_i) = P(\omega_i)P(\mathbf{x}|\omega_i)$$

"Probabilidad de observar la muestra x con la etiqueta c"

- **Densidad incondicional P(x)** de las muestras se define como

$$P(\mathbf{x}) = \sum_{j=1}^{c} P(\mathbf{x}, \omega_j) = \sum_{j=1}^{c} P(\mathbf{x}|\omega_j)P(\omega_j)$$

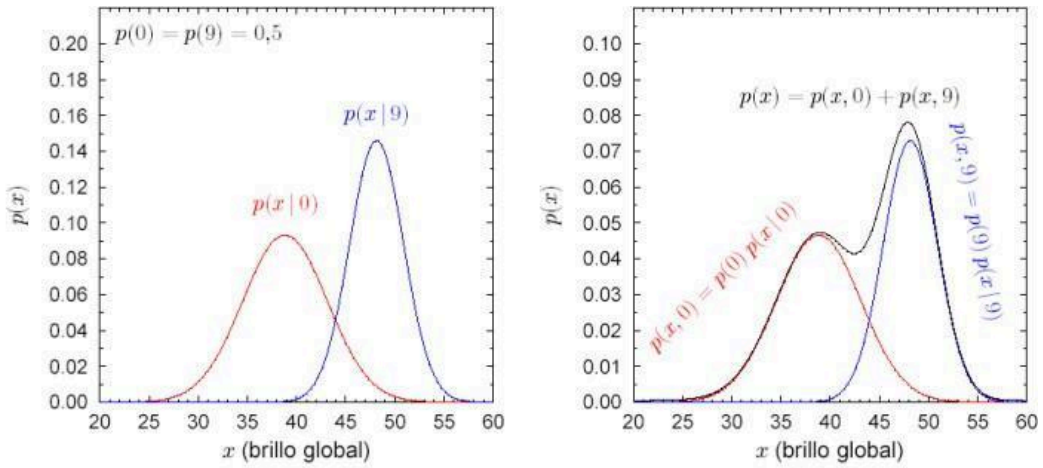
La suma para todas las clases de la probabilidad conjunta

"Probabilidad de observar la muestra x sin saber cuál es su etiqueta"

→ Caracteriza la distribución estadística de las muestras con independencia de las clases a las que pertenecen.

Se cumple que: $P(\mathbf{x}) \geq 0$, y $\int_E P(\mathbf{x})\ d\mathbf{x} = 1$.

Ejemplo:



Significado de: $P(\omega = 0) = 0{,}5;\ P(x = 45|\omega = 0) = 0{,}033;$
$P(x = 45, \omega = 0) = 0{,}016;\ P(x = 45) = 0{,}054.$

Supongamos que tenemos conocidas **P(ωi)**, P(x|ωi) y una medida de **x**. **¿Cómo influye este conocimiento sobre la decisión del clasificador?**

Probabilidad a posteriori P(ωi|x) de una clase ωi: ***probabilidad de que una muestra particular x pertenezca a ωi.***

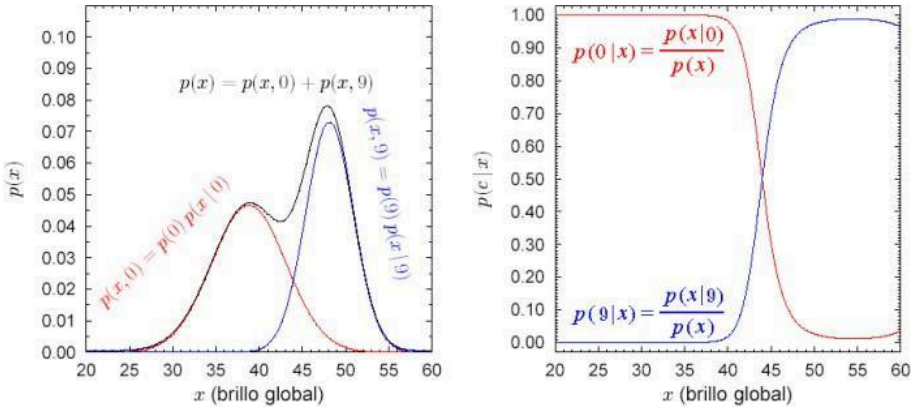
"Probabilidad de observar la etiqueta c sabiendo que la muestra es x"

Se calcula mediante la **regla de Bayes:**

$$P(\omega_i|\mathbf{x}) = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{P(\mathbf{x})} = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{j=1}^{c} P(\mathbf{x}|\omega_j)P(\omega_j)}$$

Se cumple que: $0 \leq P(\omega_i|\mathbf{x}) \leq 1$, y $\sum_{i=1}^{c} P(\omega_i|\mathbf{x}) = 1$.

→ Puede verse como una actualización de P (ωi) después de observar x.



=> Estas probabilidades serían las funciones discriminantes que estamos buscando.

Clasificador de Bayes

- **Regla de clasificación de Bayes:** asignar a x la clase con mayor probabilidad a posteriori.

$$\hat{\omega} = \quad \arg\max_{\omega_i : 1 \leq i \leq c} \quad P(\omega_i|\mathbf{x})$$

- **Clasificador de Bayes:**
  - Utiliza las probabilidades a posteriori como funciones discriminantes.
  - Es de mínimo error basado en FD monótonas crecientes(.)

$$\begin{aligned} g_i(\mathbf{x}) &= P(\omega_i|\mathbf{x}) = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{P(\mathbf{x})} \\ &\equiv P(\mathbf{x}|\omega_i)P(\omega_i) \\ &\equiv \log P(\mathbf{x}|\omega_i) + \log P(\omega_i) \end{aligned}$$

Equivalencia entre clasificadores:

$$(g_1, \ldots, g_c) \equiv (f(g_1), \ldots, f(g_c)) \equiv (g'_1, \ldots, g'_c) \text{ si } R_c = R'_c \ \forall c.$$

- Error de Bayes a posteriori: probabilidad de error puntual dado por

$$P(\text{error}|\mathbf{x}) = 1 - \max_{1 \leq i \leq c} P(\omega_i|\mathbf{x})$$

- Probabilidad media de error del clasificador:

$$P(\text{error}) = \int_E P(\text{error}|\mathbf{x})P(\mathbf{x})d\mathbf{x}$$

→ **Problemas con el cálculo analítico del error**:

- **P(ωi)** y **P(x|ωi)** desconocidas.
- Regiones de integración complejas.

→ **Solución**: *estimación* del error.

Aplicaciones del RP

Reconocimiento de Imágenes

OCR - Análisis de documentos - Rec. de firmas - Identif. de patentes - Detección de defectos en piezas industriales (control de calidad) - Rec. de gestos faciales

Reconocimiento de Habla y Lenguaje

Rec. de palabras aisladas y discurso continuo - Identif. del locutor - Traducción automática - Comprensión

Aplicaciones Biomédicas

Segmentación de tejido en imágenes médicas - Bio Identificación (huellas dactilares, palma, iris, otros)

Economía

Minería de datos - Detección de patrones de fraude

Aplicaciones en astronomía, agricultura, protección civil, etc.

UNIDAD 5 - Inteligencia Colectiva 1

Autómatas Celulares

→ Estructura que tiene entradas **X**, salidas **Y**, un conjunto de estados **E** y un conjunto de reglas **D** que le permiten ir cambiando de estado.

$$\mathbf{A = < X, Y, E, D >}$$

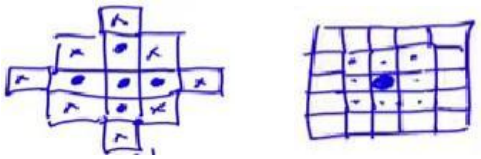
→ Extensión con funciones de salida: $\mathbf{y = \lambda(x, E)}$

→ Los estados con reglas de transición se representan como **grafos**.

→ Las **reglas de transición** pueden ser determinísticas y probabilísticas.

Autómatas celulares

- Definición: **R = < A, T, C >** **(A = autómatas que lo componen)**
  (T = topología)
  (C = conectividad)
- Topologías: triangular, rectangular, hexagonal,...
- Acoplamiento:
  - Tipos y tamaños de vecindad: Von Neumann, Moore,...



(radio 2)

  - Tipos de conexiones: isotrópicas, anisotrópicas,...
    Si son todas iguales en todas las direcciones o tienen un sentido preferencial.

Ejemplos:
Juego de la vida de Conway: reglas básicas

- Vivo con menos de 2 vivos en el entorno → muere
- Vivo con más de 3 vivos en el entorno → muere
- Vivo con 2 o 3 vivos en el entorno → vive
- Muerto con 3 vivos en el entorno → nace

→ Mirek's Cellebration
Crecimiento de plantas, bacterias,...
Poblaciones: colonias de hormigas, enjambres de abejas, modelos presa predador,...
Tejidos biológicos: cardíaco, nervioso,...
Fluidos

Agentes

Un *agente* es un sistema que...

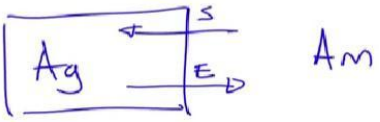
- está situado en un *ambiente*
- es capaz de realizar *acciones automáticas*
- para cumplir sus *objetivos* de diseño

Un *agente* inteligente debe...

- ser proactivo (realizar acciones sin un requerimiento específico)
- ser reactivo (reaccionar ante una demanda o un cambio en el ambiente)
- y poseer habilidad social (poder interactuar con otros agentes)

Un *agente* es todo aquello que...

- percibe su ambiente mediante *sensores*
- y responde o actúa sobre el ambiente mediante *efectores*



Todo agente debe poseer *autonomía*, es decir, capacidad de...

- ***aprender*** de la experiencia
- modificar comportamiento en tiempo de ejecución

Un agente *racional* es aquel que...

- realiza acciones correctas

Un *agente racional* ideal debe ser capaz de...

- percibir
- conocer
- decidir
- actuar

Sistemas multi-agente

Son cooperativos

- por interacción
- por contratos
- por negociación

Un agente posee racionalidad social...

- si puede realizar acciones que generan un beneficio a *todos*
- si ese beneficio es más grande que las pérdidas:

$$\text{utilidad esperada} = f(\text{utilidad individual}) + f(\text{utilidad social})$$

Inteligencia Colectiva - Generalidades

Denominaciones y relaciones:

- Computación evolutiva
- Inteligencia de colonias
- Inteligencia de enjambres
- Inteligencia colaborativa
- Inteligencia social
- ...

Características generales:

- **Auto-organización** (relación con SOM...)
- **Estigmergia**: “colaboración a través del medio físico”
- **Comportamiento emergente**: (comportamiento que uno no define de antemano)

→ inteligencia distribuida, da robustez al sistema

- **Fuerte interacción local** (vecindades)
- **Organización social altamente estructurada** (con roles y jerarquías)
- **Colaboración versus competencia** (siempre hay un compromiso entre colaborar con los demás hacia un fin común o competir y lograr cada uno lo suyo)
- **Componentes estocásticas** (aparecen mucho y son importantes)
- **Bio-inspiración** (casos inspirados en la naturaleza para resolver otros problemas)

Ejemplos de inteligencia colectiva:

• Bandadas de pájaros
• Colonias de hormigas
• Paneles/enjambres de abejas
• Cardúmenes de peces
• Rebaños de ovejas o cabras
• Manadas de predadores

Elementos individuales:

- "Boids": partículas, objetos, elementos...
- Autómatas: redes de autómatas celulares
- Agentes: en estructuras de multi-agentes
- Neuronas…

Algoritmos de inteligencia colectiva:

- ➢ Algoritmos evolutivos
- ➢ Colonias de hormigas
- ➢ Enjambre de partículas
  - • Difusión estocástica
  - • Formación de ríos
  - • Búsqueda gravitacional
  - • Sistema inmune artificial
  - • Algorítmos meméticos
  - • …

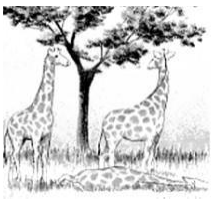
Principales aplicaciones:

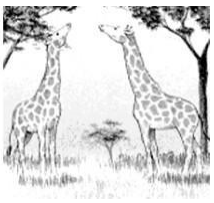
• Optimización: aproximación de funciones, entrenamiento, estimación, identificación, planificación,...
• Búsqueda, ruteo
• Agrupamiento no supervisado, clasificación
• …

Computación Evolutiva: Algoritmos Genéticos

Hace 150 años...
La idea de la evolución genera un cambio de paradigmas tan grande que hasta hoy, incluso en computación, estamos hablando de **Charles R. Darwin**.

Variación y selección natural: si hay variabilidad en la longitud del cuello de las jirafas, las de cuello corto tendrán menos probabilidades de sobrevivir y procrear. Así…





...en la próxima generación habrá menos jirafas de cuello corto.

Y...
¿Qué tiene que ver todo esto con la computación?
¿Y con la "inteligencia" computacional?
¿Podremos ver las ideas de Darwin como un algoritmo? → SÍ
¿Podremos usar estas ideas para resolver problemas con la computadora?

La evolución como un algoritmo

- Poblaciones versus individuos
- Mejores versus "adaptados" → La evolución busca la mejor adaptación a las condiciones actuales, no al mejor "super individuo".
- Aleatoriedad en la selección natural → Los mejores adaptados no tienen asegurada la supervivencia, y viceversa.
- Diversidad → si no hay diversidad la población corre un gran riesgo, porque es casi imposible entonces que se pueda adaptar a las condiciones de contexto => **hay que mantener la diversidad en nuestros algoritmos con operadores de variación en la población.**

→ **Población:** población de *potenciales soluciones a algún problema que queramos resolver, de cualquier tipo, que se sepa que tiene solución.*

Algoritmo:

```
Inicializar(Población)
MejorAptitud ← Evaluar(Población)
mientras MejorAptitud < AptitudRequerida
    Progenitores ← SelecciónNatural(Población)
    Población ← ReproducciónVariación(Progenitores)
    MejorAptitud ← Evaluar(Población)
fin
```

Elementos de un algoritmo evolutivo

1. **Representación de los individuos** → poder codificar las soluciones de nuestro problema.
2. **Función de aptitud** → cómo medir qué tan bueno es un individuo para resolver el problema, con un número.
3. **Estrategia de selección** → decir cómo se eligen los padres de la nueva generación, teniendo en cuenta una componente aleatoria.
4. **Operadores de variación** → cómo cruzar el material genético para generar los hijos.
5. **Reproducción y reemplazo generacional** → cómo realizar la reproducción y generar la nueva población.

Algoritmos genéticos: representación de los individuos

¿Cómo traducir el problema práctico que tenemos en esta representación especial que necesitamos para los algoritmos genéticos?

→ **CROMOSOMAS**

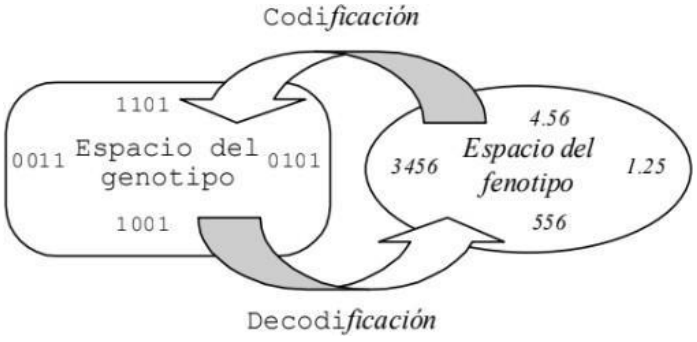
→ que adentro tienen **GENES**

→ que pueden tomar distintos valores llamados **ALELOS**

En cada posición del cromosoma va a haber un gen y ese gen podrá tomar el valor 0 o 1

* → **GENOTIPO**: la información genética que posee un organismo en particular, en forma de ADN. El genotipado se usa para determinar qué variaciones específicas existen en el individuo.
* → **FENOTIPO**: la expresión del genotipo en función de un determinado ambiente. Los rasgos fenotípicos cuentan con rasgos tanto físicos como conductuales

→ **GENOTIPOS informáticos**: dos posibilidades, 0 y 1.
→ **FENOTIPOS informáticos**: problema a resolver, los parámetros/pesos que podemos representar del problema.



→ **Proceso de codificación**: proceso en el cual se convierte de genotipo a fenotipo.

- Lo necesitamos ya que si tenemos el material genético en estado puro, en ceros y unos, no vamos a poder saber si esa es una buena solución o no a nuestro

problema, entonces tenemos que convertirla al dominio del fenotipo donde realmente podemos evaluar qué tan buen solución es para el problema el individuo

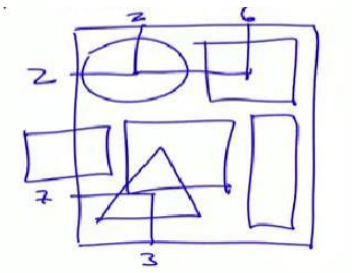
→ **Proceso de decodificación**: proceso inverso. Aunque no es tan crucial, también vamos a tener intenciones de ver, dado una determinada solución del problema, a qué genotipo se corresponde.

→ Vamos a tener muchos individuos, cada uno va a tener sus cromosomas, 1 o más.
→ Cada individuo es una posible solución al problema → codificada.

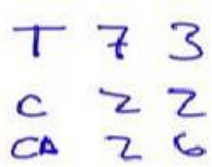
1) Representación de los individuos: ejemplos

- **Ubicación de figuras para el llenado de un área,** aprovechando lo mejor posible el espacio.

Solución: **(pos. figuras)**



Solución codificada:
(fenotipo)



(fig posx posy)

Cromosoma binario: ***(genotipo)***
(3 figuras, 10 bit en total por cada una) [fig posx posy]

[00 0000 0000][01 0000 0000]...
[10 0000 0000]

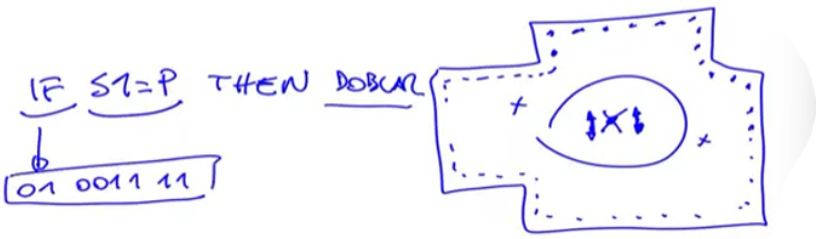
→ Nos queda un individuo de 30 bits de largo → **MATERIAL GENÉTICO DE UN INDIVIDUO**

- **Entrenamiento de una red neuronal**

Fenotipo:
Genotipo:
1- Cant neuronas capa 1 [1enbits 2enbits 3enbits 4enbits ….]
2- Cant neuronas capa 2
3- Cant neuronas
capa 3 (4-N) - pesos

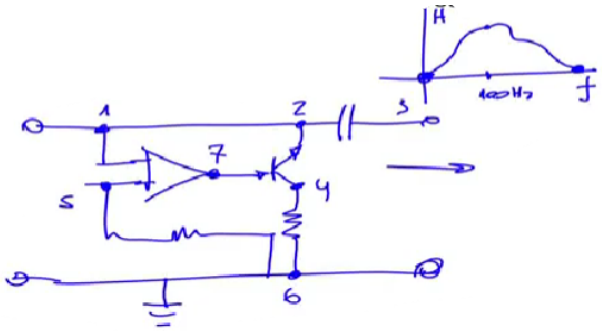
[1 2 3 4 ….. N]

- **Programación de un robot** (debe recorrer el borde de la habitación si chocar las paredes) Vamos a poder construir un programa escrito en código, asignando códigos binarios a las instrucciones

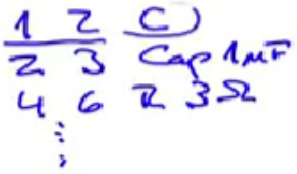


Llegamos a representar en un cromosoma **[instrucción argumentos]**, agregando bits si es necesario → Luego el programa será una serie/cadena de cromosomas

- **Circuito para un filtro multibanda**



FENOTIPO:
Decimos qué hay entre cada nodo:
[N1 N2 cable/capacitor/etc valor]



GENOTIPO: codificación en bits…

- **Problema del agente viajero…(...)**

2) Función de Aptitud (fitness)

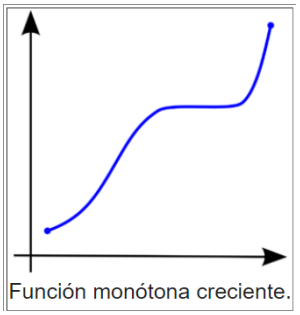
→ Función que mida qué tan bien soluciona el problema un individuo.
→ Nos sirve para elegir a los padres de la próxima generación.
→ Está muy correlacionada con el problema a resolver, pero todas tienen:

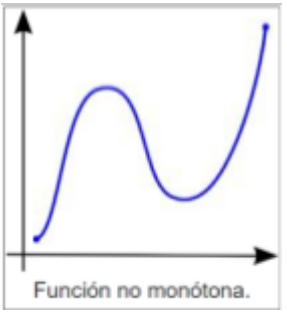
- **Características generales**

➢ ***Monotonicidad***



=>





➢ ***Precisión***

La función debe poder diferenciar individuos que sean parecidos y distinguirlos con números diferentes.

➢ ***Suavidad regulable*** (no es completamente necesaria)

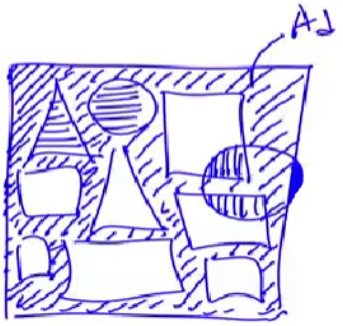
Poder graduar con algún parámetro qué tan escarpada es la forma.
Ej: función sigmoidea → la suavidad del salto la puedo regular con el parámetro alpha.

➢ **Penalización de complejidad**
Ante dos soluciones buenas, que elija la más simple, penalizando la complejidad en la función de aptitud.

- Algunos ejemplos típicos de funciones:
  - Promedios de error: cuadrados medios, desviación media absoluta, error relativo medio,...
  - Estadísticas: estimación de la varianza, validación cruzada, verosimilitud, predicción de error,...
  - Medidas de información: criterio de Akaike, criterio de información Bayesiano, descriptor de mínima longitud, información mutua, minimización del riesgo empírico
  - Otras: correlaciones, distancias,...

Ejemplos de funciones de aptitud para los ejemplos anteriores de individuos

- **Ubicación de figuras para el llenado de un área**



→ Podríamos definirla como el área desperdiciada **Ad** en negativo
=> cuanto menos área halla el valor de aptitud crece.
→ Más el área ocupada **Ao**
→ Menos las áreas solapadas **As**
→ Menos el área que queda afuera **Aa**.

f = - Ad + Ao - As - Aa

→ Podríamos agregarles coeficientes a los términos y darles más o menos pesos.

- **Entrenamiento de una red neuronal**

→ tasa de acierto o error

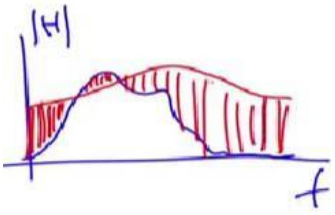
- **Programación de un robot**

→ Definir cerca de las paredes un conjunto de objetivos, y luego contar cuántos de ellos alcanzó, y definir:

f = objetivos_alcanzados - distancia_recorrida

- **Circuito para un filtro multibanda**

→ Medir el error entre la salida del circuito y la esperada



$$f = \frac{1}{1+c}$$

- **Problema del agente viajero**
  → distancia total recorrida + otros criterios de costos asociados a “hotelería” u otros gastos.

*** → Representación de individuos + Función de aptitud se definen MUY en función al problema particular que queremos resolver.

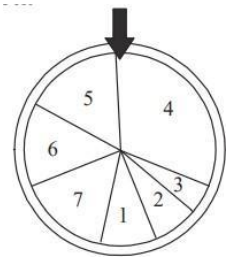
→ una vez hecho esto, lo que resta es más estándar.

Operadores Genéticos

3) Estrategias de selección

→ ***todos los individuos tienen la posibilidad de ser elegidos, pero todos tienen distinta probabilidad****, y esa probabilidad tiene que estar* ***en función de qué tan buenos son.***

a. Método de la ruleta

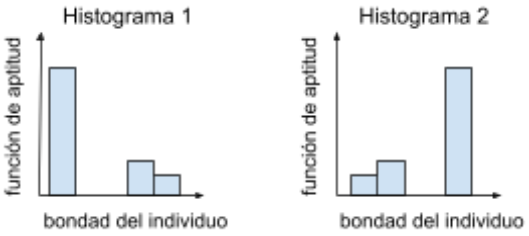


→ cada individuo tiene una porción de la ruleta que es proporcional a su fitness.

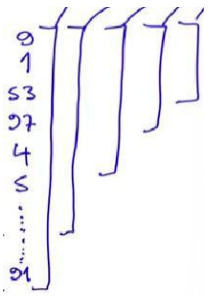
Tiene 2 inconvenientes:

1. Mar de los mediocres (va a caer siempre en los malos porque son muchos más).
2. Mar de los virtuosos (termina eligiendo al azar porque todos tienen una porción similar).

→ en ambos casos hay un *problema con la función de fitness*.



b. Ventanas



→ Utiliza el fitness para ordenar a los individuos, pero descarta el valor absoluto de él.

Primero elige una ventana grande, y saca uno al azar, luego una más pequeña siempre desde arriba hacia abajo, y elige otro,...

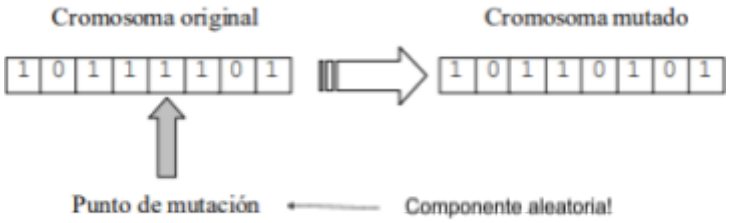
=> de esta manera le estamos dando menos probabilidad de ser elegidos a los últimos que tienen menos fitness, pero la probabilidad no es cero porque en la primera ventana estaban contemplados.

c. **Competencias**

→ Se eligen k individuos cualquiera y el que tenga mayor fitness queda.

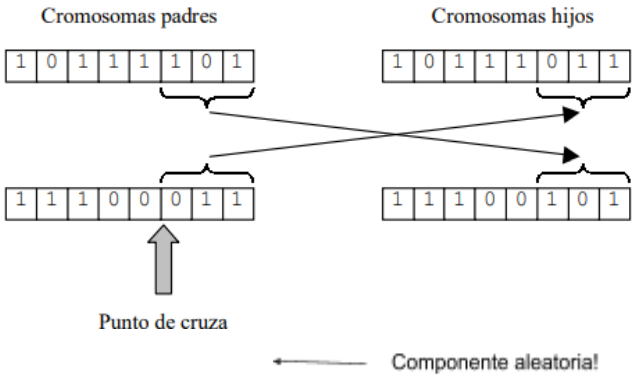
4) Operadores de variación/cruza/mutaciones

Mutación: Elegir un **punto al azar**, y cambiar ese bit:



→ Se definen *tasas de mutación*, con las cuales hay que tener cuidado y especificar *sobre qué se definen*, a nivel de individuos, cromosomas o genes.

Cruzas simples: Elegir un **punto a partir del cual cruzar al azar** e intercambio el material genético.



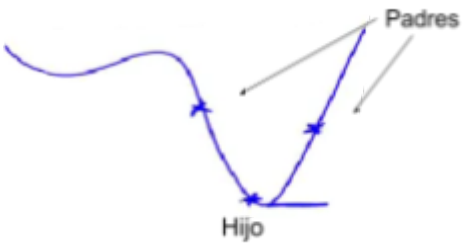
→ la idea es combinar las partes buenas de 2 padres en un hijo para que sea mucho mejor que los padres.

¿Qué rol cumple cada operador en la búsqueda?
Mutación:

★ Sirve para aumentar la diversidad y explorar regiones lejanas del espacio de búsqueda. → tal vez en una zona alejada encontremos el verdadero mínimo global.

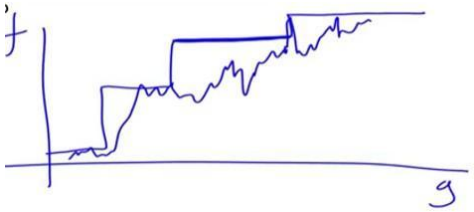
Cruza:

★ Hace una exploración más fina de una zona del espacio.



5) **Reproducción y reemplazo generacional**

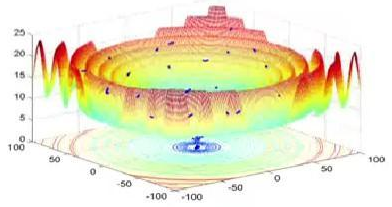
- **Reemplazo total** → Reemplazo la población por todos los hijos que generé.
- **Reemplazo con brecha generacional** → dejamos vivos algunos padres y le sumamos a los nuevos hijos generados
- **Elitismo** → se toma el mejor individuo de la población y *se lo pasa directamente a la próxima generación.*
    - la consecuencia de esto en el fitness en función de las generaciones pega saltos, cuando se encuentra uno mejor, sino, variaría todo el tiempo.

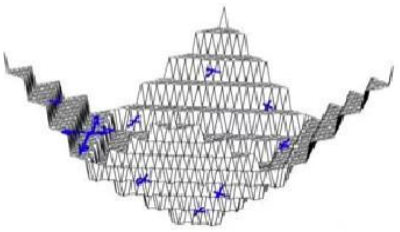


Algoritmos evolutivos: características principales

- ***Búsqueda en un espacio codificado de parámetros***, el genotipo, si bien voy midiendo la función de aptitud sobre el espacio verdadero.
    + Ventaja → en este espacio se asegura la convergencia.

- ***Búsqueda en múltiples puntos del espacio de soluciones*** ←←←←
  Si tenemos 100 individuos, cuando comienza la evolución es como si tomáramos un puñado de semillas y lo tiramos sobre la superficie de error. En cada generación, con cruzas y mutaciones, ellos se van a ir ubicando en otras regiones, y de a poco se van a ir acercando al mínimo global ya que ganan los de mejor fitness, sin quedarse estancados.





- ***Pocos requisitos sobre la función objetivo,*** al contrario del método del gradiente, donde sí necesitamos conocer la derivada y la forma analítica de la función.

- ***Algoritmo de naturaleza estocástica*****,** pero no es completamente aleatorio, combina parte determinística con parte aleatoria.
- ***La estructura de los operadores los hace muy efectivos al realizar búsquedas globales.***
- ***Múltiples objetivos***, pueden perseguir múltiples objetivos a la vez con la función de fitness.

Algunas desventajas:

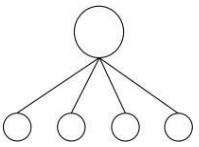
- Son lentos.
- Mucho costo computacional (evaluación de la función de fitness).

Comparación con otros métodos

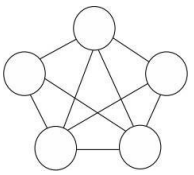
Métodos tradicionales	Algoritmos evolutivos
Trabajan con los propios parámetros a optimizar	Emplea una codificación de los parámetros*
Utilizan información de las derivadas de la función objetivo u otro conocimiento adicional	Utilizan la información de la función objetivo en forma directa
Reglas de transición deterministas	Reglas de transición combinando componentes determinísticos y probabilísticos
Exploran el espacio de soluciones a partir de un punto	Exploran el espacio de soluciones en múltiples puntos a la vez

Paralelismo

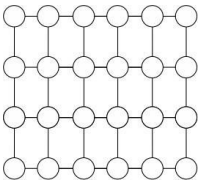
- **Esquema Maestro-Esclavo:**
  - Podemos hacer las cruzas y mutaciones en el servidor, y distribuyó una función de fitness para cada nodo del sistema de paralelismo que tenga con N individuos c/ una.



- **Esquema conexiones:**
  - Formar pequeñas colonias, con N individuos, cada una trabajando en un procesador o grupo de procesadores, y cada tanto se hacen intercambios, todos contra todos, para hacer que evolucione en forma conjunta.



- **Esquemas tipo celdas**
  - Las conexiones no son todos contra todos, sino que se definen regiones donde se intercambia, lo cual requiere cambios en el algoritmo evolutivo en sí mismo.



Computación Evolutiva: Variantes

Parámetros que controlan la evolución

→ Hay distintos parámetros que podrían variar, o modificarse automáticamente: *no ser fijos, o no ser los mismos para todos los individuos, o para distintas partes dentro de un cromosoma*.

- **Probabilidad de mutaciones**
- **Probabilidad de cruzas**
- **Tamaño de la población**
- **Brecha generacional**
- **Elitismo**

  → dan el surgimiento de los algoritmos llamados *"estrategias de evolución"*

Estrategias de evolución

- Representación "**fenotípica**": → poner directamente en los cromosomas los valores de las variables, sin la conversión a binario.
  - Variables objetivo → las que estuvimos evolucionando.
  - Variables de control (o estratégicas) → las que controlan la evolución.
- **Fitness** ... igual que en algoritmos genéticos.
- **Operadores**:
  - Mutación → *lo mutamos y lo guardamos sólo si no empeora.*
  - Cruza (optativa)
- **Selección**: aleatoria, *combinada con mutación.*
- **Reproducción**: determinística, dos mecanismos:
  - ➢ Mecanismo (μ + λ)-ES: se generan **μ** padres que producen **λ ≥ 1** hijos.
    → *Próxima generación*: **{μ** ∪ **λ}** eliminando los peores de **λ.** (tam. pobl. cte.)
  - ➢ Mecanismo (μ, λ)-ES: se generan **μ** padres que producen **λ > μ** hijos.
    → *Próxima generación*: **subconjunto** con los mejores de **λ.** (tam. pobl. cte.)

Representación de los individuos

Terminología: *algoritmo/programación/estrategia...*

- **Genético** : representación **BINARIA**
  - ➢ Muchos genes con pocos alelos (0 o 1): convergencia asegurada por el teorema de esquemas.
  - ➢ Epistasis: *un gen incorrecto invalida a todo el cromosoma.*
  - ➢ Representación lejana al dominio del problema (ej: viajero con enteros)
  - ➢ Gran cantidad de soluciones inválidas en la población.

- **<u>Evolutivo:</u>** representación **REAL** o "**fenotípica**"
  - ➢ Pocos genes con muchos alelos: representación fenotípica.
  - ➢ Convergencia muy dependiente de los operadores.
  - ➢ *Necesidad de redefinición de operadores no "biológicos".*

Otras representaciones: Cromosomas de longitud variable, Árboles, Grafos…

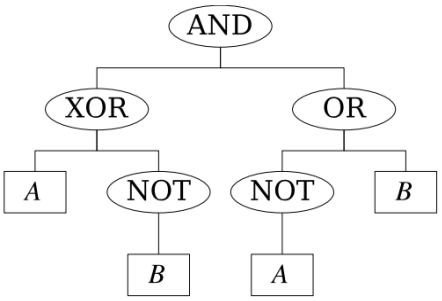
Programación genética

"...generación automática de programas..."

Elementos básicos de un programa:

- Variables y constantes
- Operadores aritméticos y lógicos
- Funciones matemáticas
- Condicionales
- Bucles
- Recursiones
- ...

Ejemplo sencillo con operadores lógicos: $((A)\text{XOR}(\text{NOT}(B)))\text{AND}((\text{NOT}(A))\text{OR}(B))$

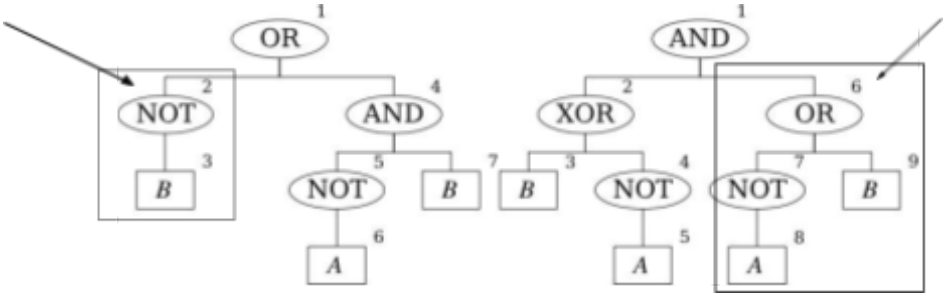


Cruzas en PG

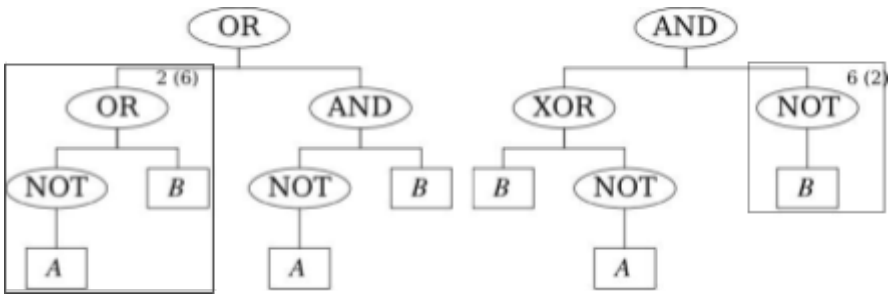
1. <u>Numeración de nodos</u>

   → En este caso primero vamos en profundidad y después vamos recorriendo las ramas

2. <u>Selección de los puntos a cruzar</u> ej 2 y 6



3. <u>Cruza en base al intercambio de ramas</u>



→ Me permitió hacer cruza pero manteniendo la función central de cada árbol, sin hacer cambios tan abruptos.

Mutaciones en PG

- Numeración de nodos
- Selección de la rama a mutar
- Generación de un árbol al azar (con las variables correspondientes)
- Mutación en base al reemplazo
  → el punto clave es que necesitamos una función generadora de árboles al azar.

¿Cómo incorporar en el algoritmo las restricciones del problema real/dominio de la aplicación?

¿Cómo se pueden considerar las restricciones del problema ***durante la evolución***?

- Redefinición de la representación de forma de que **siempre se generen fenotipos válidos.**
- Rechazo o eliminación de individuos inválidos → no es del todo deseable.
- Reparación del material genético.
- **Modificación de los operadores de variación** → para que no hagan individuos inválidos.
- Esquemas de penalización en la función de aptitud.

UNIDAD 6 — Inteligencia Colectiva 2

Inteligencia de enjambres

1. Colonia de hormigas:

La inspiración biológica

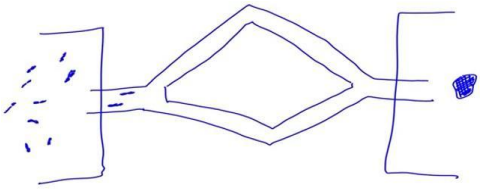
- Hay 10^{16} hormigas en la tierra (y 6×10^{9} humanos).
- Igual peso total.
- 30 millones por colonia.
- ***Feromonas:*** *la búsqueda del camino más corto a la comida.*
  1. Comportamiento inicial aleatorio
  2. Cuando encuentran una fuente de comida se organizan y comienzan a seguir el mismo camino
     - **2.1.** Mecanismo de reclutamiento: mayormente por **feromonas**, liberadas al regresar (algunas las liberan en proporción a la cantidad de alimento encontrado).
     - **2.2** Si otras encuentran feromonas siguen el rastro (con más probabilidad, tratando de alejarse del hormiguero si no llevan comida).
     - **2.3** El camino **se refuerza** al ser seguido por más y más hormigas.

• Notar:
    • **Comunicación indirecta**
    • Modificación del entorno físico: **estigmergia**.

Experimento del puente binario

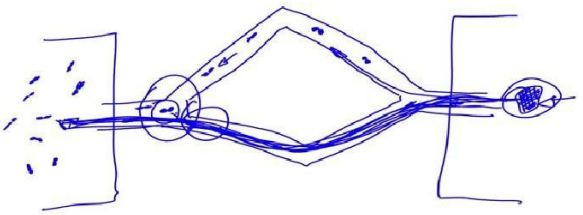
→ dió origen a los algoritmos que surgieron después.

Tenemos 2 mesas a cierta distancia, vistas de arriba, y entre medio de ellas tenemos un puente que tiene 2 brazos, uno más largo y otro más corto.
En una de las mesas tenemos hormigas que se empiezan a mover aleatoriamente hasta que algunas empiezan a transitar por el puente. Vamos a suponer que, con alguna probabilidad, 2 hormigas que van juntas exploran un camino cada una, el de arriba y el de abajo.



La hormiga que vaya por el más corto va a descubrir la comida primero. Una vez que llegó, y lo descubrió, va a volver por donde vino, y la otra que también llegó pero más tarde hará lo mismo.

Cuando una nueva hormiga quiera transitar por el puente, va a llegar a la bifurcación y tiene que tomar la decisión de ir por arriba o por abajo. *Muy probablemente* (la preferencia no es absoluta), perciba que *hay más feromonas en el camino más corto, lo cual va a seguir reforzando este camino*, hasta llegar al punto de que la *decisión de las hormigas en la bifurcación sea directa*.



=> encontraron el camino óptimo.

Colonia de hormigas: algoritmos

Definiciones:

- ***G = (V, E):*** grafo con *vértices o nodos* ***V*** (puntos donde va pasando la hormiga) y *matriz de conexiones* ***E*** entre esos puntos.
- σ_{ij}: cantidad de feromonas en la conexión entre **i** y **j**
- ***k = 1, 2, . . . , N***: hormigas
- ***Ni***: nodos disponibles a partir del nodo **i**, para cada nodo.
- $p^k(t)$***:*** camino de la hormiga **k** (lista de nodos por los que fue pasando), va a ir cambiando con el tiempo.

Notar:

- **t → t + 1:** se incrementa una vez que ***todas*** las hormigas encuentran el alimento y vuelven al origen.

- $\mathbf{N}^k$: denota los nodos disponibles *para la hormiga* ***k***. En algunos casos el entorno se limita a los nodos que *no haya visitado previamente la hormiga* ***k***.

Algoritmo 1: Colonia de hormigas simple (sACO)

1. $t = 0$, Inicializar feromonas con valores pequeños al azar $(\sigma_{ij}(t) \rightsquigarrow U(0, \sigma_0))$
2. Ubicar N hormigas en el nodo origen
3. **repetir**
   - 3.1 **para** cada hormiga $k = 1, 2, \ldots, N$
     - 3.1.1 $\mathbf{p}^k(t) = \varnothing$ → Lista de nodos por los que pasó la hormiga = vacío
     - 3.1.2 **repetir**
       - seleccionar el próximo nodo según la *probabilidad* $p_{ij}^k(t) = \cdots$

$$p_{ij}^k(t) = \begin{cases} \dfrac{\sigma_{ij}^{\alpha}(t)}{\sum_{\forall u \in N_i} \sigma_{iu}^{\alpha}(t)} & \text{si } j \in N_i \\ 0 & \text{en otro caso.} \end{cases}$$

→ Incorporamos el exponente alpha, que nos permite potenciar el efecto de esa cantidad de feromonas

       - agregar un paso (i, j) al camino $\mathbf{p}^k(t)$

       hasta *alcanzar el destino*

     - 3.1.3 eliminar los ciclos en $\mathbf{p}^k(t)$
     - 3.1.4 calcular la longitud del camino econtrado $f\left(\mathbf{p}^k(t)\right)$

   - 3.2 **para** cada conexión (i, j)
     - reducir por evaporación la cantidad de feromonas: $\sigma_{ij}(t) \leftarrow (1 - \rho)\sigma_{ij}(t)$

   Tasa de evaporación de la feromona

   - 3.3 **para** cada conexión (i, j)
     - depositar feromonas proporcionalmente a la bondad de la solución

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \in \mathbf{p}^k(t)} 1/f\left(\mathbf{p}^k(t)\right)$$

→ inversamente proporcional al camino de esa hormiga

→ sumando los aportes de todas las hormigas a esa intersección

   - 3.2 **para** cada conexión (i, j)
     - reducir por evaporación la cantidad de feromonas: $\sigma_{ij}(t) \leftarrow (1 - \rho)\sigma_{ij}(t)$
   - 3.3 **para** cada conexión (i, j)
     - depositar feromonas proporcionalmente a la bondad de la solución

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \in \mathbf{p}^k(t)} 1/f\left(\mathbf{p}^k(t)\right)$$

   - 3.4 $t \leftarrow t + 1$

   hasta que *todas* las hormigas sigan el mismo camino
4. **devolver** el camino más corto $(\downarrow f\left(\mathbf{p}^k(t)\right))$

Algoritmo 2: Sistema de hormigas (AS)

1. $t = 0, \sigma_{ij}(t) \rightsquigarrow U(0, \sigma_0)$
2. Ubicar N hormigas en el nodo origen
3. **repetir**
   - 3.1 **para** cada hormiga $k = 1, 2, \ldots, N$
     - 3.1.1 $\mathbf{p}^k(t) = \varnothing$
     - 3.1.2 **repetir**
       - seleccionar el próximo nodo según la *probabilidad*

$$p_{ij}^k(t) = \begin{cases} \dfrac{\sigma_{ij}^{\alpha}(t)\eta_{ij}^{\beta}(t)}{\sum_{\forall u \in N_i^k} \sigma_{iu}^{\alpha}(t)\eta_{ij}^{\beta}(t)} & \text{si } j \in N_i^k \\ 0 & \text{en otro caso.} \end{cases}$$

       → Deseo de moverse inverso al costo entre nodos: $\eta_{ij} = 1/d_{ij}$.

       → Lista de nodos vecinos con tabú: N_i^k — lista sin nodos por los que ya pasó la hormiga, evita que se generen ciclos

       - agregar un paso (i, j) al camino $\mathbf{p}^k(t)$

       hasta *alcanzar el destino*

     - 3.1.3 calcular la longitud del camino econtrado $f\left(\mathbf{p}^k(t)\right)$
   - 3.2 **para** cada conexión (i, j)
     - reducir por evaporación la cantidad de feromonas: $\sigma_{ij}(t) \leftarrow (1 - \rho)\sigma_{ij}(t)$
     - depositar feromonas proporcionalmente a la bondad de la solución

$$\Delta\sigma_{ij}^k(t) = \begin{cases} Q/f\left(\mathbf{p}^k(t)\right) & \text{global.} \\ Q & \text{uniforme.} \\ Q/d_{ij} & \text{local.} \end{cases}$$

Alternativas de incremento de feromonas

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \in \mathbf{p}^k(t)} \Delta\sigma_{ij}^k(t)$$

   - 3.3 $t \leftarrow t + 1$

   hasta que *todas* las hormigas sigan el mismo camíno
4. **devolver** el mejor camino

2. <u>Enjambre de partículas:</u>

La inspiración biológica

- Bandadas de pájaros.
- Cada individuo vuela/navega el espacio *ajustando su posición en base a su* ***experiencia propia*** *y a la información de los individuos de su entorno, la* ***experiencia de su entorno***.
- <u>Emular el éxito de los vecinos y propio</u>.
- Tenemos 2 variables fundamentales:
  - $x^k(t)$: <u>posición</u> de la partícula **k**, en el tiempo **t** (espacio R^N)
  - $v^k(t)$: <u>velocidad</u> de la partícula **k**, en el tiempo **t**
- Regla básica: $\mathbf{x}_k(t+1) = \mathbf{x}_k(t) + \mathbf{v}_k(t)$

Definiciones:

- y_k: mejor posición personal de la partícula k (la mejor que visitó desde t = 0 hasta la actualidad) → aquella con menor error para el problema a resolver.
- ***f(x)***: *función de error* a minimizar.

Mejor global (gEP):

- *Topología **estrella***: usa *info. de todo el enjambre* → ***información social global***
- $\hat{y}$: mejor posición visitada por todo el enjambre

Algoritmo 3: Enjambre del mejor global (gEP)

1. Inicializar $\mathbf{x}_{ki}(0) \rightsquigarrow U(\mathbf{x}_i^{min}, \mathbf{x}_i^{max})$ → Para cada variable i de los vectores posición x
2. **repetir**
   - 2.1 **para** cada partícula $k = 1, 2, \ldots, N$
     - **si** $f(\mathbf{x}_k(t)) < f(\mathbf{y}_k) \rightarrow \mathbf{y}_k = \mathbf{x}_k(t)$ → Ver si la posición actual es mejor que su mejor posición individual
     - **si** $f(\mathbf{y}_k) < f(\hat{\mathbf{y}}) \rightarrow \hat{\mathbf{y}} = \mathbf{y}_k$ → Ver si la posición actual es mejor que la mejor posición global
   - 2.2 **para** cada partícula $k = 1, 2, \ldots, N$
     - $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}\left[y_{ki} - x_{ki}(t)\right]$ → experiencia personal
       $+ c_2 r_{2i}\left[\hat{y}_i - x_{ki}(t)\right]$ → experiencia social

       velocidad del individuo k en la dimensión i

       $r_{1i}, r_{2i} \rightsquigarrow U(0,1)$

       c → constante de aceleración

       r → constante aleatoria entre 0 y 1.
     - $\mathbf{x}_k(t+1) = \mathbf{x}_k(t) + \mathbf{v}_k(t+1)$

   hasta cumplir con la condición de finalización
3. **devolver** la mejor partícula encontrada

Variante - Enjambre de partículas

Definiciones:

- y_k: mejor posición personal de la partícula k (la mejor que visitó desde t = 0 hasta la actualidad) → aquella con menor error para el problema a resolver.
- ***f(x)***: *función de error* a minimizar.

Mejor global (gEP):

- *Topología **anillo***: usa *info. del entorno cercano a la partícula* → ***información social global acotada a un entorno***
- Selección: basada en los índices de las partículas (caso más simple)

$$\mathcal{E}_k = \{\mathbf{y}_{k-n}, \mathbf{y}_{k-n+1}, \ldots, \mathbf{y}_{k-1}, \mathbf{y}_k, \mathbf{y}_{k+1}, \ldots, \mathbf{y}_{k+n}\}:$$ vecindad

- $\hat{\mathbf{y}}_k = \arg\min_{\forall \mathbf{y}_u \in \mathcal{E}_k} \{f(\mathbf{y}_u)\}$ mejor posición visitada por el entorno de la partícula k → para todas las mejores posiciones individuales que alcanzaron las partículas vecinas de k, las recorremos y buscamos el mínimo, y la definimos como la mejor local.

Algoritmo 4: Enjambre del mejor local (ℚEP)

1. Inicializar $\mathbf{x}_{ki}(0) \rightsquigarrow U(\mathbf{x}_i^{min}, \mathbf{x}_i^{max})$
2. **repetir**
   2.1 **para** cada partícula $k = 1, 2, \ldots, N$
   - **si** $f(\mathbf{x}_k(t)) < f(\mathbf{y}_k) \rightarrow \mathbf{y}_k = \mathbf{x}_k(t)$
   - **si** $f(\mathbf{y}_k) < f(\hat{\mathbf{y}}) \rightarrow \hat{\mathbf{y}} = \mathbf{y}_k$

   2.2 **para** cada partícula $k = 1, 2, \ldots, N$
   - $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i} [y_{ki} - x_{ki}(t)] + c_2 r_{2i} [\hat{y}_{ki} - x_{ki}(t)]$

     $r_{1i}, r_{2i} \rightsquigarrow U(0,1)$

     $\hat{\mathbf{y}}_k = \arg\min_{\forall \mathbf{y}_u \in \mathcal{E}_k} \{f(\mathbf{y}_u)\}$

     → Aparece k porque ahora la información social depende de cada partícula k, no es la misma para todas.
   - $\mathbf{x}_k(t+1) = \mathbf{x}_k(t) + \mathbf{v}_k(t+1)$

   hasta cumplir con la condición de finalización
3. **devolver** la mejor partícula encontrada

UNIDAD 7 - Lógica Borrosa

Lógica Borrosa

→ Surge a partir de la idea de cómo el ser humano piensa las cosas de las que no tiene certeza total, es decir a partir de reglas que no están completamente definidas, no son verdadero o falso, no son cosas absolutas.

- Nosotros usamos reglas lingüísticas
  - Tendríamos que pensar en un:

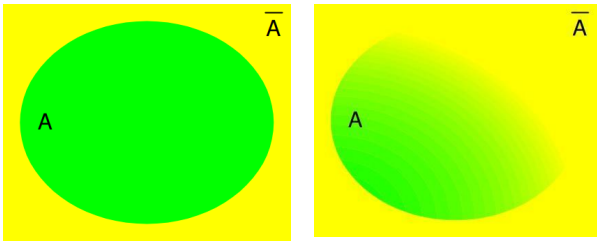
```
if-then borroso?
   si la temperatura es ALTA
   entonces activar el acondicionador a nivel MEDIO
```

- Incerteza vs aleatoriedad:
  - **Incerteza** → Sistemas borrosos, trata de UN objeto.
  - **Aleatoriedad** → Estadística de las probabilidades, trata de MUCHOS objetos.

En un objeto que se acerca desde lejos tenemos incerteza.
En el número que saldrá al tirar los datos tenemos aleatoriedad.

Conceptos matemáticos borrosos

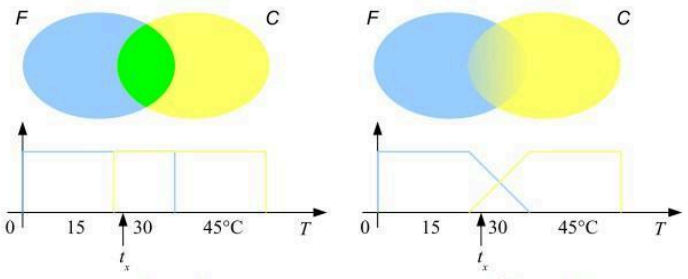
Conjunto binario vs Conjunto borroso



→ Tiene ciertos grados de pertenencia a los conjuntos, con ciertos grados de incerteza.

Ejemplos:

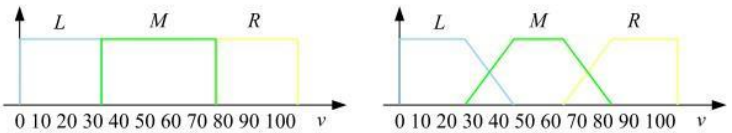
- Temperaturas (caso contínuo)



$t_x \in F \rightarrow 1{,}0$
$t_x \in C \rightarrow 1{,}0$

$t_x \in \tilde{F} \rightarrow 0{,}8$
$t_x \in \tilde{C} \rightarrow 0{,}2$

- Velocidades (caso discreto)



→ [0 10 20 30 40 50 60 70 80 90 100]

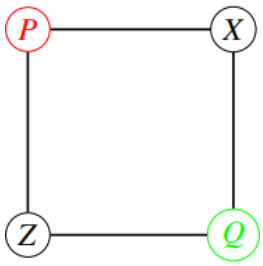
$\mu_L = 1\,1\,1\,1\,0\,0\,0\,0\,0\,0\,0$
$\mu_M = 0\,0\,0\,0\,1\,1\,1\,1\,0\,0\,0$
$\mu_R = 0\,0\,0\,0\,0\,0\,0\,0\,1\,1\,1$

$\mu_{\tilde{L}} = 1\,1\,1\,0{,}6\,0{,}3\,0\,0\,0 \quad 0 \quad 0\,0$
$\mu_{\tilde{M}} = 0\,0\,0\,0{,}3\,0{,}6\,1\,1\,0{,}6\,0{,}3\,0\,0$
$\mu_{\tilde{R}} = 0\,0\,0\,0 \quad 0 \quad 0\,0\,0{,}3\,0{,}6\,1\,1$

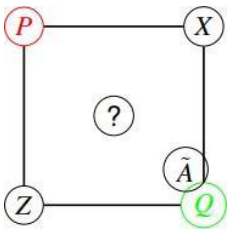
Forma de representar a los conjuntos borrosos

Simplificación a conjuntos de 2 elementos (R^2):

Conjuntos binarios $P = (0;1)$
y $Q = (1;0)$
Conjuntos universo $X = (1;1)$
y vacío $\Phi = (0;0)$



Esto nos permite colocar objetos dentro del cuadrado de manera que no pertenezca totalmente a P o Q:



Vamos a utilizar **funciones de membresía**:

→ En las cuales los grados de pertenencia ya no son 0 o 1, sino que pueden tener también cualquier valor intermedio $\mu_{\tilde{A}} : x \rightarrow [0, 1]$

Operaciones con conjuntos borrosos

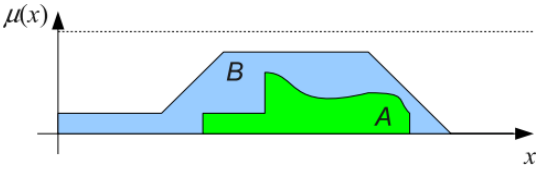
Subconjunto borroso

Sea **E** un conjunto enumerable y **x** un elemento de **E**. Un subconjunto borroso **A˜** de **E** es un conjunto de pares ordenados (elemento, grado de pertenencia):

$$\tilde{A} = \{(x_i, \mu_A(x_i))\}; \quad x_i \in E$$

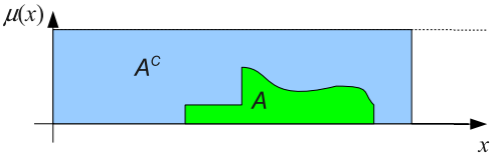
donde $\mu_A(x_i)$ es el grado de membresía de **x** en **A˜**.

Inclusión: $\tilde{A} \subset \tilde{B} \Leftrightarrow \mu_A(x) \leq \mu_B(x) \quad \forall x$

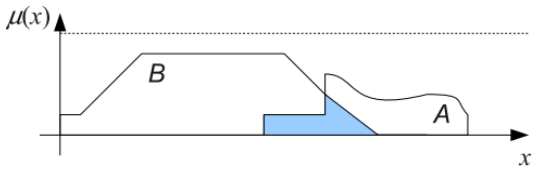


Igualdad: $\tilde{A} = \tilde{B} \Leftrightarrow \mu_A(x) = \mu_B(x) \quad \forall x$

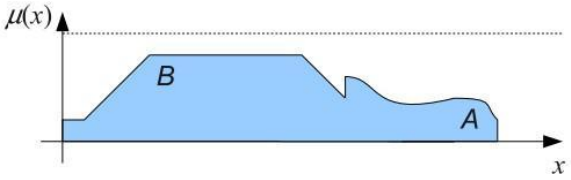
Complemento: $\tilde{B} = \tilde{A}^c \Leftrightarrow \mu_B(x) = 1 - \mu_A(x) \quad \forall x$



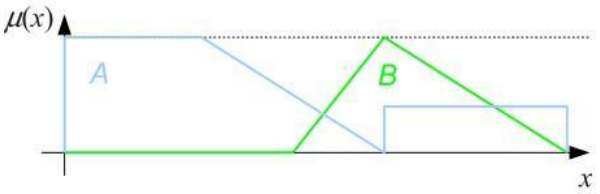
Intersección: $\tilde{A} \cap \tilde{B} \Rightarrow \mu_{A \cap B}(x) = \min\{\mu_A(x), \mu_B(x)\} \quad \forall x$



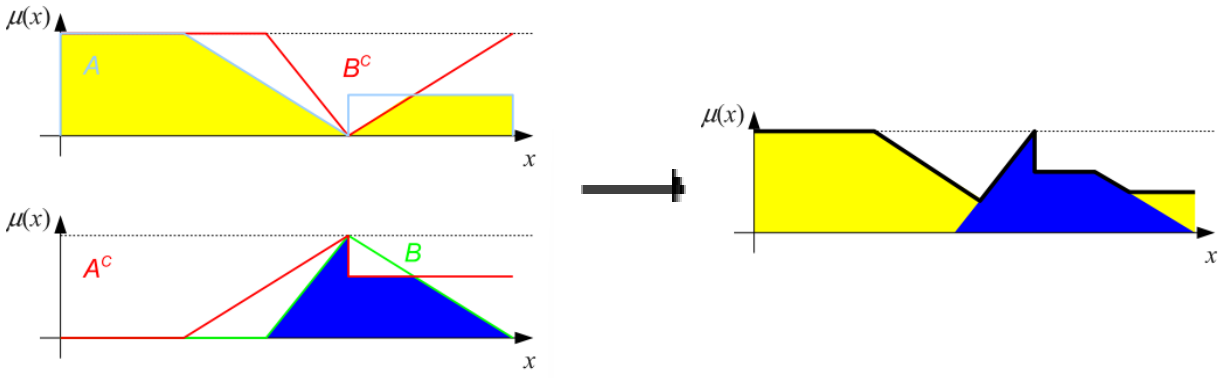
Unión: $\tilde{A} \cup \tilde{B} \Rightarrow \mu_{A \cup B}(x) = \max\{\mu_A(x), \mu_B(x)\} \quad \forall x$



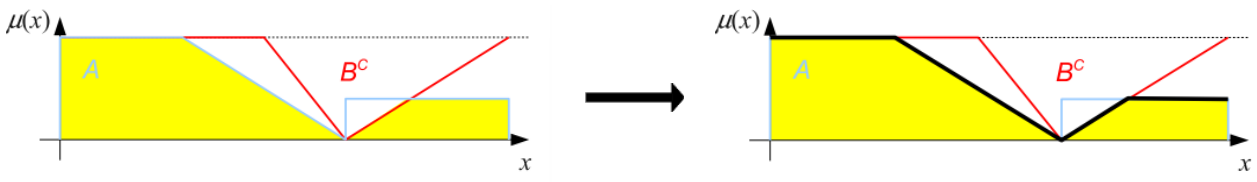
Operaciones más complejas…



Suma disyuntiva: $\tilde{A} \oplus \tilde{B} = (\tilde{A} \cap \tilde{B}^c) \cup (\tilde{A}^c \cap \tilde{B})$



Diferencia: $\tilde{A} - \tilde{B} = \tilde{A} \cap \tilde{B}^c$



Medidas de distancia entre conjuntos borrosos

Distancia de Hamming: $d(\tilde{A},\tilde{B}) = \sum_{i=1}^{n} |\mu_A(x_i) - \mu_B(x_i)|$

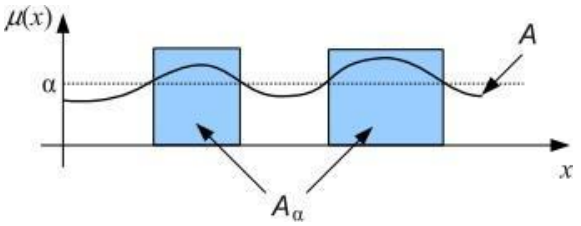
Distancia de Hamming relativa: $\delta(\tilde{A},\tilde{B}) = \frac{d(\tilde{A},\tilde{B})}{n}$ (no depende de la cant de elem)

Distancia de Euclídea: $e(\tilde{A},\tilde{B}) = \sqrt{\sum_{i=1}^{n} |\mu_A(x_i) - \mu_B(x_i)|^2}$

Distancia de Euclídea relativa: $\epsilon(\tilde{A},\tilde{B}) = \frac{e(\tilde{A},\tilde{B})}{\sqrt{n}}$

Caracterización de los conjuntos borrosos

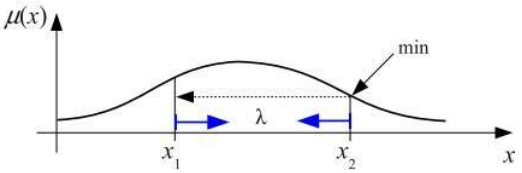
Conjunto binario de nivel α: $A_\alpha = \{x / \mu_A(x) \geq \alpha \quad \forall x \in A\}$



→ Conjunto estándar, no borroso, cuyo grado de membresía al conjunto original A borroso es mayor a una constante **α**.

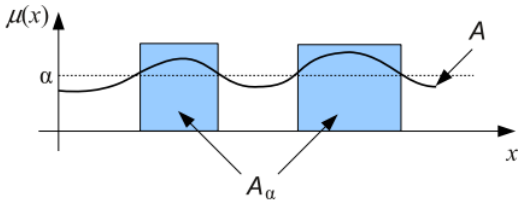
Conjunto convexo:

1. $\mu_A(\lambda x_1 + (1-\lambda)x_2) \geq \min\{\mu_A(x_1), \mu_A(x_2)\}$
   $\forall \lambda \in [0,1], \forall x_1, x_2 \in \mathbb{R}$



→ El grado de membresía del **x** resultante del cálculo $(\lambda x_1 + (1-\lambda)x_2)$, debe ser mayor o igual que el mínimo entre la membresía de ambos, para cualquier $\mathbf{x_1}$ y $\mathbf{x_2}$ que yo tome dentro del conjunto **A**, y cualquier **λ** dentro de [0,1].

2. $\tilde{A}$ es convexo $\Leftrightarrow$ A_α es convexo $\forall \alpha \in [0,1]$ ← Definición más simple de calcular.

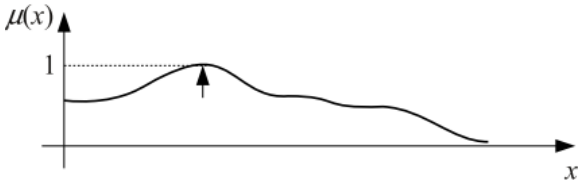


→ Probamos para todos los alfas, y con encontrar 1 para el cual haya un rango de x que no pertenezcan al A_{alfa}, ya sabemos que no es convexo.

¿$\tilde{A}$ es convexo?

Conjunto Normal:

$$\max\{\mu_A(x)\} = 1$$



Tamaño de un conjunto borroso:

→ Lo calculamos sumando las membresías de todos los elementos.

$$|\tilde{A}| = \sum_{i=1}^{n} \mu_A(x_i)$$ → Funciona también para conjuntos binarios.

Otras propiedades curiosas $\tilde{A} \cup \tilde{A}^c =^? X$

$\tilde{A} \cap \tilde{A}^c =^? \Phi$

Ejemplo:

$A = (0{,}2; 0{,}8)$

$A^c = (0{,}8; 0{,}2)$

$A \cup \tilde{A}^c =^? (0{,}8; 0{,}8)$ $A \cap \tilde{A}^c =^? (0{,}2; 0{,}2)$

El conjunto borroso medio

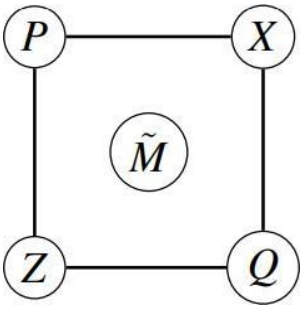
→ El conjunto en el que todos sus elementos tienen grado de membresía 0.5, es decir la máxima incertidumbre.

$$\tilde{M}/\mu_M(x) = \frac{1}{2} \quad \forall x \in E$$

Lo que podemos observar, es que en este caso, la intersección entre él y su complemento, da lo mismo que el conjunto medio, que es lo mismo que la unión, que es lo mismo que el complemento.

$$\tilde{M} = \tilde{M} \cap \tilde{M}^c = \tilde{M} \cup \tilde{M}^c = \tilde{M}^c$$

=> Este conjunto medio es el peor caso que podemos tener:



¿Cómo medimos la borrosidad?

Entropía borrosa

→ La *distancia* entre el conjunto borroso en cuestión, y el *extremo más cercano*, dividido la *distancia* entre el conjunto borroso y el *extremo más lejano*:

$$S(\tilde{A}) = \frac{d(\tilde{A}, I^n_{min})}{d(\tilde{A}, I^n_{max})}$$

Donde los extremos se definen como n-uplas, es decir, combinaciones de ceros y unos, que serán los vértices de un cuadrado (n=2) o un cubo (n=3), etc:

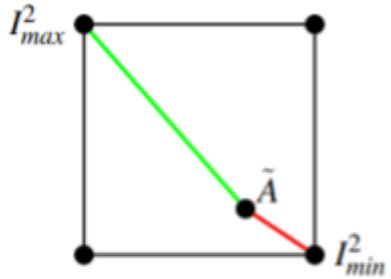
$$I^n = \{0,1\}^n$$

$$I^n_{min} = I^n_{j*} \Leftrightarrow d(\tilde{A}, I^n_{j*}) < d(\tilde{A}, I^n_j) \quad \forall j \neq j*$$

$$I^n_{max} = I^n_{i*} \Leftrightarrow d(\tilde{A}, I^n_{i*}) > d(\tilde{A}, I^n_i) \quad \forall i \neq i*$$

Ejemplos:

$A = (0{,}7; 0{,}2)$



$d_{max} = |0{,}7 - 0{,}0| + |0{,}2 - 1{,}0| = 1{,}5$ ← Distancia de Hamming, medida en las 2 coord.

$d_{min} = |0{,}7 - 1{,}0| + |0{,}2 - 0{,}0| = 0{,}5$

$S(\tilde{A}) = \frac{0{,}5}{1{,}5} = \frac{1}{3}$ ← Tiene entropía borrosa de ⅓, es ⅓ borroso.

$S(\tilde{M}) = 1{,}0$ ← El conj.borroso medio tiene entropía de 1, es el conjunto más borroso posible.

$S(\tilde{I}^n) = 0{,}0$

Memorias Asociativas Borrosas (FAM)

→ Intentan codificar las reglas borrosas.

Regla simple (local FAM)

$$\texttt{if } X = \tilde{A} \texttt{ then } Y = \tilde{B}$$

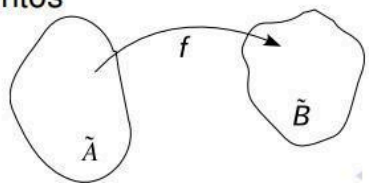
$$\left\{ \begin{array}{l} \tilde{A} \in I^n \\ \tilde{B} \in I^p \end{array} \right\} f : [0,1]^n \rightarrow [0,1]^p$$

Diagrama en bloques



f: x→y

Diagrama de conjuntos

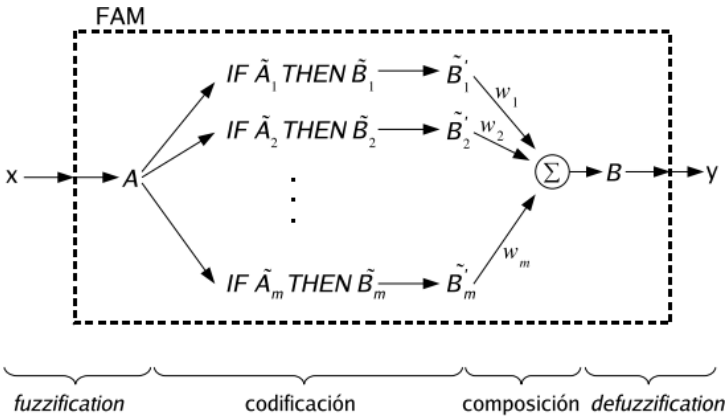


Composición de reglas (global FAM)

→ Modelo aditivo estándar (SAM)

Cada regla tiene un peso asociado, y la salida de esa regla es lo que se va acumulando (suma de conjuntos).

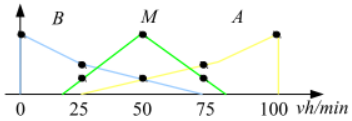
$$\tilde{B} = \sum_{j=1}^{m} w_j \tilde{B}'_j$$



Fuzzyfication: un ejemplo sencillo

Control automático de un semáforo por nivel de tráfico

- Conjuntos borrosos de entrada:



tráfico *BAJO*, *MEDIO* y *ALTO*

- Activaciones para $x = 75$ vehículos/minuto
  - $\mu_{BAJO} = 0{,}00 \Rightarrow$ IGNORAR reglas tráfico *BAJO* no activada $(a_1(x) = 0{,}00)$
  - $\mu_{MEDIO} = 0{,}25 \Rightarrow$ ACTIVAR reglas tráfico *MEDIO* activada con $a_2(x) = 0{,}25$
  - $\mu_{ALTO} = 0{,}50 \Rightarrow$ ACTIVAR reglas tráfico *ALTO* activada con $a_3(x) = 0{,}50$

FAM
FAM: codificación

➢ Discretización de los conjuntos de entrada

v/min	→	0	25	50	75	100	
$\tilde{A}_1 = BAJO$	= (	1.00	0.50	0.25	0.00	0.00	)
$\tilde{A}_2 = MEDIO$	= (	0.00	0.25	1.00	0.25	0.00	)
$\tilde{A}_3 = ALTO$	= (	0.00	0.00	0.25	0.50	1.00	)

➢ Conjuntos borrosos de salida:

tiempo de espera *CORTO*, *NORMAL* y *LARGO*

➢ Discretización de los conjuntos de salida

seg.	→	0	30	60	90	120	
$\tilde{B}_1 = CORTO$	= (	1.00	0.25	0.00	0.00	0.00	)
$\tilde{B}_2 = NORMAL$	= (	0.00	0.50	1.00	0.50	0.00	)
$\tilde{B}_3 = LARGO$	= (	0.00	0.00	0.00	0.25	1.00	)

Codificación de una regla borrosa

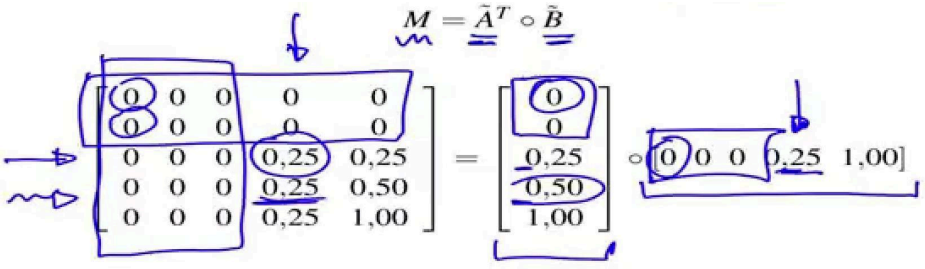
Notación: ejemplo para una única regla

- `if` x es *ALTO* `then` y es *LARGO*
- `if` *ALTO* `then` *LARGO*
- $(ALTO; LARGO)$
- (antecedente, consecuente)

Fuzzy Hebb FAMs: codificación *correlación-mínimo*

$$M = \tilde{A}^T \circ \tilde{B} \rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0{,}25 & 0{,}25 \\ 0 & 0 & 0 & 0{,}25 & 0{,}50 \\ 0 & 0 & 0 & 0{,}25 & 1{,}00 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0{,}25 \\ 0{,}50 \\ 1{,}00 \end{bmatrix} \circ [0 \;\; 0 \;\; 0 \;\; 0{,}25 \;\; 1{,}00]$$

→ Obtenemos cada elemento de M con el mínimo de las coordenadas en A y en B



Prueba: $\tilde{A} \circ M = \tilde{B}$?? *("el máximo del mínimo")* → para probar, agarro A contra cada columna de M, busco el mínimo elem a elem, y me quedo con el máx de esos mínimos.

M codifica la regla `if` $\tilde{A}$ `then` $\tilde{B}$

Fuzzy Hebb FAMs: codificación *correlación-producto*

$$M = \tilde{A}^T \tilde{B} \quad \rightarrow \quad \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0{,}0625 & 0{,}25 \\ 0 & 0 & 0 & 0{,}125 & 0{,}50 \\ 0 & 0 & 0 & 0{,}25 & 1{,}00 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0{,}25 \\ 0{,}50 \\ 1{,}00 \end{bmatrix} [0 \;\; 0 \;\; 0 \;\; 0{,}25 \;\; 1{,}00]$$

→ Solo es un poco más compleja computacionalmente.

Prueba: $\tilde{A}M = \tilde{B}$?? *("el máximo del producto")*

Teoremas de bidireccionalidad (NN&FS 310 y 313) --> ver bibliografía.

Codificación: consideraciones prácticas

- Si la etapa de fuzzification es directa y sólo admite una de las entradas discretas, los conjuntos de entrada siempre tendrán la forma de un δi
- Siguiendo con el ejemplo de x = 75 vehículos/minuto, el conjunto de entrada sería un conjunto binario clásico: A 0 = (0.00 0.00 0.00 1.00 0.00)
- Con este tipo de conjuntos binarios de entrada, el producto matricial para la regla k se simplifica como:

$$A' \circ M_k = A' \circ (\tilde{A}_k^T \circ \tilde{B}_k) = (A' \circ \tilde{A}_k^T) \circ \tilde{B}_k = a_{ki} \tilde{B}_k$$

- Cuando el conjunto de entrada es un conjunto borroso realmente, ya sea por:
  - provenir de un real proceso de fuzzification.
  - ser el resultado de un sistema borroso en una etapa previa.

  → vuelve a tener sentido el producto matricial para cada regla A' ◦ Mk.
- El conjunto borroso de entrada podría ser: A˜0 = (0.00 0.00 0.25 0.50 0.25)

FAM: composición

Composición de reglas

→ combinar todas las matrices y obtener el resultado con un solo cálculo.

Superposición de reglas codificadas en matrices:

$$M = \max_{1 \leqslant k \leqslant m} M_k$$

(Eficiente, pero no recomendado…, a medida que incremento la cantidad de reglas, la matriz tiende a aplanarse, con todos 1)

Modelo aditivo estándar (SAM): → generar UNA sola matriz que combine todo.

Componemos la salida de cada regla, en lugar de las reglas, con algún peso. (mejor)

$$\tilde{B} = \sum_{j=1}^{m} w_j \tilde{B}'_j$$

SAM con factor de activación:

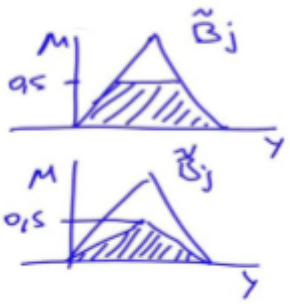
Incorporamos con qué grado se activa cada regla, por separado:

$$\tilde{B} = \sum_{j=1}^{m} w_j a_j(x) \tilde{B}_j$$

→ Factor de activación de reglas (ej: a(x) = 0.5)

- mínimo
- escala

→ Representación gráfica:



→ Combinación de conjuntos

- máximo
- suma
- centroides (compuesta con defuzzyfication)

FAM: defuzzyfication

→ La idea es convertir el conjunto borroso de salida en un número en caso de que nuestra aplicación práctica necesite un número real para operar.

$$x \overset{fuzz}{\rightarrow} \tilde{A} \rightarrow \underbrace{REGLAS}_{if\ldots then} \rightarrow \underbrace{SAM}_{\sum w_j B'_j} \rightarrow \tilde{B} \overset{??}{\rightarrow} y$$

De-fuzzyfication: un ejemplo sencillo ...para el caso discreto

- Fuzzification (sigue ejemplo control de un semáforo):
  - La entrada x = 75 nos dió:

$$x \overset{fuzz}{\rightarrow} \left\{ \begin{array}{l} a_1(x) = 0{,}00 \\ a_2(x) = 0{,}25 \\ a_3(x) = 0{,}50 \end{array} \right\}$$

- Aplicación de las reglas:
  - `if` *BAJO* `then` *CORTO* $\rightarrow \tilde{B}_1 = \tilde{A}_1 \circ M_1$
    no activada $(a_1(x) = 0{,}00)$
  - `if` *MEDIO* `then` *NORMAL* $\rightarrow \tilde{B}_2 = \tilde{A}_2 \circ M_2$
    activada con $a_2(x) = 0{,}25$
  - `if` *ALTO* `then` *LARGO* $\rightarrow \tilde{B}_3 = \tilde{A}_3 \circ M_3$
    activada con $a_3(x) = 0{,}50$

- Composición de las reglas activadas (simple escala-suma):
  - $\tilde{B} = \sum_{j=1}^{m} w_j a_j(x) \tilde{B}_j$
  - $\tilde{B} = 0{,}0B_1 + 0{,}25B_2 + 0{,}5B_3$
  - $\tilde{B} = 0{,}25[0 \;\; 0{,}5 \;\; 1{,}0 \;\; 0{,}5 \;\; 0] + 0{,}5[0 \;\; 0 \;\; 0 \;\; 0{,}25 \;\; 1{,}0]$
  - $\tilde{B} = [0 \;\; 0{,}125 \;\; 0{,}25 \;\; 0{,}25 \;\; 0{,}5]$
    → Obtenemos un conjunto discreto de salida.

- ¿Cómo obtenemos y, la salida no borrosa? (segundos de semáforo en verde)

seg.	→	0	30	60	90	120	
$\tilde{B}_1 = CORTO$	= (	1.00	0.25	0.00	0.00	0.00	)
$\tilde{B}_2 = NORMAL$	= (	0.00	0.50	1.00	0.50	0.00	)
$\tilde{B}_3 = LARGO$	= (	0.00	0.00	0.00	0.25	1.00	)

y = 0, 30, 60, 90 o 120

→ 2 alternativas:

Defuzzyfication

- **Maximum-defuzzyfication** $y = \arg\max_{y_j} \mu_B(y_j)$

  y = [0, 30, 60, 90, 120]
  B = [0, 0.125, 0.25, 0.25, 0.5] → max = 0.5 => y = 120seg.

- **Centroid-defuzzyfication**
  → Pesar a cada una de las posibles velocidades por el grado de membresía, divido los grados de membresía.

$$y = \frac{\sum_j y_j \mu_B(y_j)}{\sum_j \mu_B(y_j)}$$

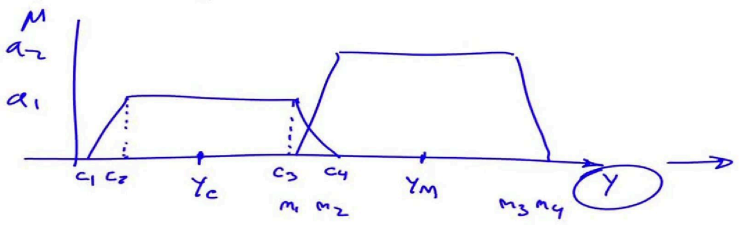
FAM: extensiones

- Conjuntos de pertenencia continuos.
- Representación de dos antecedentes por consecuente.
- Composición de dos antecedentes.

→ **Conjuntos de pertenencia continuos:**

- Los conjuntos son funciones continuas
- No existen las matrices M
- Método simplificado para guardar los trapecios
- Evaluación de reglas (una a una...)
- Composición de los conjuntos de cada regla y defuzzyfication → en un solo paso.
  → Método de los centroides

Composición de los conjuntos de cada regla y defuzzyfication para los centroides trapezoidales (y_c, y_m)



→ Queremos calcular el centroide de estos 2 conjuntos unidos, de coordenadas trapezoidales c1,c2,c3,c4 y m1,m2,m3,m4, y ese centroide de la figura completa va a ser la salida defuzzyficada de la combinación de estos 2 conjuntos:

- Cálculo: $y = \frac{\sum_j y_j B_j}{\sum_j B_j} = \frac{y_c B_c + y_m B_m}{B_c + B_m}$

→ Sumamos el producto de las posiciones y de los conjuntos por sus **áreas**, y lo dividimos por la suma de sus áreas.

- Suponiendo $c_2 - c_1 = c_4 - c_3$ y $m_2 - m_1 = m_4 - m_3$:

→ **Área** = ancho * altura(activación **a**)

$$y = \frac{y_c a_1[(c_3 - c_2) + (c_2 - c_1)] + y_m a_2[(m_3 - m_2) + (m_2 - m_1)]}{B_c + B_m}$$

$$\boxed{y = \frac{y_c a_1 (c_3 - c_1) + y_m a_2 (m_3 - m_1)}{a_1 (c_3 - c_1) + a_2 (m_3 - m_1)}}$$

→ **Representación de dos antecedentes por consecuente**

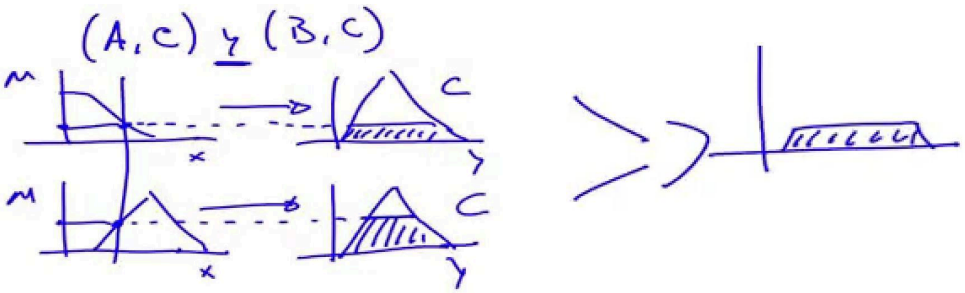
Tabla de doble entrada y en cada fila y columna poner los conjuntos:

A \ V	B	M	A
B	N	N	P
M	N	P	M
A	P	M	M
MA	P	M	MM

← Armamos 12 reglas que dependen de 2 antecedentes, velocidad y aceleración, y que tienen un consecuente, que es cuánto apretamos el freno.

→ **Composición de 2 antecedentes**

Tenemos 2 antecedentes y un mismo consecuente:



La entrada activa en cierta cantidad a A y B. Ante ambos casos tenemos un asociado un conjunto de salida y. Lo que va a hacer el grado de activación es truncar el conjunto C en ambos casos. Luego, como lo combinamos con el "y", nos quedamos con el mínimo.

Otras alternativas.. ver bibliografía:

- FAM de entrada salida binaria (BIOFAM) (NN&FS 317)
- FAM adaptativa (NN&FS 327)

Comparación Redes Neuronales (NN) vs Sistemas Borrosos (FS) vs Sistemas Expertos (ES) (I)

★ Entradas incompletas o corruptas

- **NN** Reconstrucción automática → pueden seguir funcionando.
- **FS** Reconstrucción automática → pueden seguir funcionando.
- **ES** Reglas específicas → el sistema se va haciendo cada vez más complejo e inmanejable.

★ Visibilidad del modelo

- **NN** Numérico oculto (caja negra) → la idea es que aprendan solas.
- **FS** Simbólico-numérico visible → tienen algo intermedio, ya que las reglas son claras.
- **ES** Simbólico visible → visibilidad total.

★ Estructuración del conocimiento almacenado

- **NN** No estructurado → no tienen una estructura que se adecúa específicamente al problema que estoy resolviendo, no está hecha a medida.
- **FS** Numérico estructurado → tenemos una mezcla de numérico con simbólico.
- **ES** Simbólico altamente estructurado → tenemos reglas bien definidas, y la estructura es completamente dependiente del problema.

★ Aprendizaje

- **NN** Automático, requiere gran cantidad de datos.
- **FS** Experto + algoritmos de adaptación a los datos → caso intermedio.
- **ES** Experto humano.

★ Tamaño de la representación

- **NN** Relativamente pequeña → pueden resolver muchos problemas con poca información almacenada en los pesos.
- **FS** Proporcional a la dimensión del problema → a medida que el problema tiene más variables tengo que ir incorporando más reglas, pero no el tamaño de los conjuntos borrosos.
- **ES** Crecimiento exponencial de reglas.